1.5em

Otto-von-Guericke University Magdeburg

Department of Software Engineering

Master Thesis

Leveraging Uncertainty Estimation in Semantic Segmentation: A Step Towards Robust Vision Models

Author:
Bhavana Malla

February 11, 2025

Advisors:
Prof. Dr. Frank Ortmeier
Department of Software Engineering

Malla, Bhavana:
Leveraging Uncertainty Estimation in Semantic Segmentation: A Step Towards Robust Vision Models
Master Thesis, Otto-von-Guericke University
Magdeburg, 2025.

Abstract

Semantic segmentation assigns predefined labels to every pixel in an image and is crucial for detailed visual analysis. However, deploying Deep Neural Networks (DNNs) for semantic segmentation in safety-critical applications is challenging due to their tendency to be overconfident, particularly with Out of Distribution (OOD) data, leading to unreliable predictions and potential risks in domains like autonomous driving. Uncertainty estimation is, therefore, crucial for ensuring the reliability and robustness of semantic segmentation models. While existing models often report predictive uncertainty during inference, they typically do not leverage this knowledge during training. To bridge this gap, our thesis explores leveraging various uncertainty estimation techniques as guiding signals during the training of semantic segmentation models with the aim of improving segmentation performance, reliability, and robustness. We investigate uncertainty estimation techniques such as Monte Carlo Dropout (MCD), Online Hard Example Mining (OHEM), and Evidential Deep Learning (EDL), along with their finetuned variants, and comprehensively analyzed their impact on segmentation performance, OOD detection, and model calibration. Extensive experiments on the three standard benchmark datasets, Cityscapes, Fishyscapes, and LostandFound, reveal nuanced trade-offs between uncertainty estimation methods. While some models showed limited improvement in segmentation performance compared to the conventional baseline, EDL based models consistently outperformed all approaches in distinguishing between In-distribution (ID) and OOD samples, highlighting their superior ability to represent and leverage uncertainty information. A key contribution of this work lies in demonstrating the significant impact of uncertainty guidance training on model calibration and OOD detection. Despite these advantages, the trade-offs between segmentation accuracy and OOD detection highlights the need for further research on optimizing uncertainty integration, possibly through adaptive uncertainty modulation or novel loss functions.

Acknowledgements

I would like to express my most sincere appreciation to M.Sc. Venkatesh Thirugnana Sambandham for his invaluable guidance and support. His expertise and insightful advice have been crucial in shaping this thesis. I am immensely thankful for his trust in my abilities, his mentorship, and the creative freedom he consistently provided throughout this journey. His encouragement and insights have not only broadened my horizons but also inspired me to push beyond my limits and ensured a truly impactful learning experience.

I also extend my warmest thanks to Prof. Dr. Frank Ortmeier, without whose support this thesis would not have been possible. I am deeply grateful for the opportunity he provided me to pursue my thesis in the Department of Software Engineering at Otto-von-Guericke University. His valuable feedback and suggestions throughout the course of the thesis have been truly invaluable.

Finally, and most importantly, I want to express my heartfelt thanks to my husband, who has been my steadfast moral support and motivator throughout my academic journey. His unconditional love and encouragement have been a source of strength. I am also profoundly grateful to my family for their unwavering belief in me and their constant support throughout this academic endeavor. Your love and encouragement mean the world to me.

Contents

Abstract

1 Introduction and Motivation
1.1 Motivation . . . 6
1.2 Aim of this thesis . . . 8
1.3 Structure of this thesis . . . 9

2 Background
2.1 Overview of Computer Vision and its Core Tasks . . . 11
2.2 Semantic Segmentaion . . . 12
2.3 Deep Neural Networks . . . 13
2.3.1 Encoder Decoder Model . . . 15
2.3.2 Transformers . . . 17
2.4 Uncertainty . . . 22
2.4.1 Aleotoric Uncertainty . . . 22
2.4.2 Epistemic Uncertainty . . . 23
2.5 Confidence Calibration . . . 23
2.6 Out-of-Distribution Detection . . . 24
2.7 Softwares and Tools . . . 25
2.7.1 Python . . . 25

3 Related Work
3.1 Monte Carlo Dropout . . . 30
3.2 Online Hard Example Mining . . . 32
3.3 Evidential Deep Learning . . . 35
3.4 Expected Calibration Error . . . 40
3.5 Reliability Diagrams . . . 42

4 Methodology
4.1 Datasets . . . 45
4.1.1 Cityscapes . . . 45
4.1.2 Fishyscapes . . . 48
4.1.3 LostAndFound . . . 49
4.2 Baseline Model . . . 51
4.2.1 Encoder: MiT-B3 Encoder . . . 51

4.2.2 Decoder: Feature Pyramid Networks 55
4.3 Regularization Techniques 57
4.3.1 Data Augmentation Preprocessing 57
4.3.2 Transfer Learning 58
4.4 Proposed Methodology 59
4.4.1 MCD based Uncertainty signal training 59
4.4.2 OHEM based Uncertainty signal training 61
4.4.3 EDL based Uncertainty signal training 63
4.5 Model Training 64
4.5.1 Training Process and Optimization 64
4.5.2 Computational Resources and Configuration 65
4.5.3 Mixed-Precision Training with Gradient Scaling 66

5 Evaluation and Results
5.1 Evaluation Metrics 67
5.1.1 Segmentation Metrics 67
5.1.2 Calibration Metrics 71
5.1.3 OOD Detection Metrics 71
5.2 Experiments and Results 73
5.2.1 Vanilla Models Segmentation Performance 73
5.2.2 Vanilla Model Varaints 75
5.2.3 Per Category Performance of Vanilla Models 77
5.2.4 Finetuned Models Segmentation Performance 79
5.2.5 Per Category Performance of Finetuned Models 80
5.2.6 Calibration Performance 85
5.2.7 OOD Performance evaluation 93
5.2.8 Additional Results 103

6 Discussion and Limitations
6.1 Discussion 106
6.2 Limitations 109

7 Conclusions and Future Work
7.1 Conclusions 111
7.2 Future Work 112

A List of Figures

B List of Tables

C Bibliography

1 Introduction and Motivation

1.1 Motivation

Since the advent of digital technologies, visual data has become a cornerstone of information dissemination across various domains, including healthcare, transportation, and communication. This rapid growth of visual data has significantly increased the demand for precise and reliable methods to analyze and interpret images. Among these methods, image segmentation has emerged as a foundational technique for breaking down complex visual data into meaningful regions, enabling detailed analysis and decision-making.

Semantic segmentation, a specific branch of image segmentation, focuses on assigning a predefined label to every pixel in an image, offering a fine-grained understanding of visual content. This pixel-level precision is essential for safety-critical applications such as autonomous driving, medical imaging, and remote sensing, where accurate and cautious decision-making is essential. Deep Neural Networks (DNNs) have revolutionized the field of computer vision (LeCun et al., 2015) and also have demonstrated remarkable performance in semantic segmentation tasks, delivering impressive results across various domains(Minaee et al., 2020). However, deploying these models in such critical applications remains challenging due to their inherent lack of robustness and reliability (de Jorge et al., 2023). One of the most significant challenges with DNNs is their tendency to exhibit overconfidence in their predictions, where models may assign high confidence even to incorrect outputs (Guo et al., 2017). This is a kind of miscalibration where the predicted confidence does not accurately reflect the true probability of correctness, i.e., the models confidence in its prediction can be greater or less than the models accuracy.

This issue becomes especially problematic when models encounter Out of Distribution (OOD) data where inputs deviate substantially from training data (GOODFELLOW et al., 2015), posing considerable risks in scenarios where accuracy is paramount. Overconfidence in unrecognizable or irrelevant inputs can obscure the models limitations, leading to catastrophic outcomes.

Addressing uncertainty in model predictions is, therefore, a valuable and essential tool. Failing to estimate uncertainty while dealing with OOD instances can result in dangerous misclassifications, especially in critical domains such as healthcare, autonomous systems, and public safety. For instance, in medical imaging, an unrecognized OOD data point could result in a critical misdiagnosis, compromising patient care and outcomes. In autonomous driving, incorrect interpretation of OOD objects, such as an animal or pedestrian in an unexpected position or a new type of vehicle, can jeopardize safety and cause severe accidents.

In a constantly evolving data landscape, it is crucial for models to recognize and flag new, previously unseen data while maintaining consistent performance on familiar data. Instead of assuming all predictions are accurate, models should include mechanisms to express uncertainty, enabling them to navigate unfamiliar scenarios cautiously and reliably. Therefore, estimating the uncertainty in model predictions has become a key focus in advancing semantic segmentation systems (LEIBIG et al., 2017) (LEE et al., 2018) (LANDGRAF et al., 2023b) and is just as crucial as achieving high accuracy in tasks, particularly when handling OOD instances.

Although uncertainty estimation in deep learning models for image segmentation has been intensively studied (ABDAR et al., 2021), (GAWLIKOWSKI et al., 2022), most existing methods rely solely on task objectives (e.g., cross-entropy, IoU, or Dice) to train the network and then investigate different inference strategies such as sampling multiple predictions to compute variance [(GAL und GHAHRAMANI, 2016), (LAKSHMINARAYANAN et al., 2017), (MONTEIRO et al., 2020)], implicitly estimating uncertainty through feature density (VAN AMERSFOORT et al., 2020), or designing evidence-based layers to predict uncertainty (SENSOY et al., 2018). While these approaches are effective to a certain extent, they often suffer from a lack of a dedicated uncertainty estimation metric during both the model train-

ing and estimation phases. This absence results in indirect calibration of model confidence and an unclear prior, which can affect the quality of estimated uncertainty, especially in ambiguous regions or OOD cases.

In light of these challenges, our thesis focuses on leveraging various uncertainty estimation techniques as guiding signals for training semantic segmentation models. Our main goal is to enhance the reliability and calibration of segmentation systems. To achieve this, we examine existing uncertainty estimation methods. Through a series of experiments, we demonstrate that straightforward training mechanisms can lead to more reliable predictions by effectively identifying OOD instances and improving the confidence calibration of the segmentation systems.

1.2 Aim of this thesis

The primary objective of this thesis is to investigate the integration of uncertainty signals as a guiding mechanism in training semantic segmentation models, with the goal of enhancing their performance, reliability, and robustness, particularly in detecting Out of Distribution (OOD) instances. Rather than introducing a novel methodology, our goal is to effectively leverage existing uncertainty estimation techniques during training to improve the segmentation model reliability. We aim to explore how guidance signals, such as uncertainty estimations, can serve as valuable additional inputs that significantly boost the reliability of segmentation models. To accomplish this, we employ uncertainty quantification techniques based on Monte Carlo Dropout (MCD), Online Hard Example Mining (OHEM), and Evidential Deep Learning (EDL). These techniques are integrated as supplementary inputs in the segmentation process, with a particular emphasis on enhancing Out of Distribution (OOD) detection, uncertainty estimation, and confidence calibration. Additionally, this thesis examines the impact of this approach on OOD datasets. The culmination of the thesis includes a comprehensive analysis and evaluation metrics, as described in chapter 5. Therefore, the thesis focuses on answering the following research questions:

RQ1: How can uncertainty estimations be effectively incorporated into the training regimes of semantic segmentation models as guidance

signals, and what impact do they have on the segmentation performance of the systems?

RQ2: Does the incorporation of guidance signals in the training process enhance the reliability of semantic segmentation models compared to conventional training regimes, particularly in terms of predictive confidence and reliability?

RQ3: To what extent does employing guidance signals during segmentation model training improve the robustness in detecting Out of Distribution (OOD) objects? How effectively can an uncertainty-guided segmentation model differentiate between ID and OOD instances?

1.3 Structure of this thesis

The rest of our thesis work is structured as follows:

- **Background:** In chapter 2, we provide a comprehensive overview of the fundamental concepts in CV, including Image Segmentation and its various types, Deep Neural Networks, and the associated uncertainties, followed by a discussion on confidence calibration and OOD detection. The encoder-decoder model explained in subsection 2.3.1 serves as the basis for our baseline model architecture in section 4.2. Furthermore, in section 2.7, we describe the software and tools used in this thesis.

- **Related Work:** In chapter 3, we conduct a literature survey and provide an overview of the key topics addressed in this thesis. We discussed several earlier research studies that inspired us to dive more into leveraging uncertainty estimations in training semantic segmentation systems. Furthermore, in section 3.1, section 3.2, and section 3.3, we provide a detailed overview of the different kinds of methods used that aim to improve uncertainty estimations and confidence calibrations in segmentation systems.

- **Methodology:** In chapter 4, we describe the various datasets used to perform the experiments, along with the detailed steps. We start

by detailing how we obtained our guidance signals as additional inputs to the model. Furthermore, in section 4.4, we detail the architectural adjustments made to the baseline model to incorporate these guiding signals. Next, we provide the technical details, such as the hardware and software requirements required to perform the experiments in section 4.5.

- **Experiments and Evaluation:** In chapter 5, first, we provide a detailed explanation of the evaluation metrics used to perform the experiments. Then we provide both the qualitative and quantitative results of our experiments by evaluating the baseline and proposed models.

- **Conclusion and future work:** In chapter 7, we conclude our thesis by highlighting its contributions and identifying avenues for future research in this domain. This section not only reflects on the impact of our work but also outlines the potential for expanding upon these findings in subsequent studies.

2
Background

To better comprehend the thesis work, this chapter offers additional background information on the essential topics. The sections section 2.1 and section 2.2 explore the fundamental concepts of Computer Vision (CV) and Semantic Segmentation. Following this, section 2.3 presents a brief overview of neural network models with a detailed explanation of the encoder-decoder model architecture, which is crucial for our thesis. Additionally, section 2.4 introduces the concept of uncertainty and its various types. In section 2.5, we discuss the idea of confidence calibration and its significance, while section 2.6 focuses on the concept of Out of Distribution (OOD) detection. Finally, the last section, section 2.7, lists the software tools and libraries needed to conduct the experiments.

2.1 Overview of Computer Vision and its Core Tasks

Computer Vision (CV) is a subdiscipline of AI that enables computer systems to analyze and interpret visual information. It encompasses various tasks to mimic the human ability to perceive and understand the visual world. The main tasks in computer vision include image classification, object recognition, and image segmentation. The primary goal of image classification is to identify the main object or scene in an input image. This process involves assigning a label or category to the entire image based on its visual content. For example, a model trained for image classification might determine whether an image contains a cat, a dog, or a car. Object detection goes beyond image classification, which focuses on identifying and localizing objects within an image. This process involves predicting bounding boxes around the objects and classifying them accordingly. Object detection is especially valuable in applications like surveillance, where

detecting and tracking multiple objects in a scene, including people, vehicles, and animals is crucial. Image segmentation is a more granular task in CV; it studies the problem of grouping pixels. The core idea of segmentation is to partition an image into different regions or segments based on the characteristics of the pixels. Different semantics for grouping pixels, e.g., category or instance membership, have led to various segmentation tasks, such as panoptic, instance, or semantic segmentation. Each task serves a distinct purpose and is crucial for various applications, ranging from autonomous vehicles to medical imaging. This thesis solely focuses on semantic segmentation, an analysis task performed at the pixel level.

2.2 Semantic Segmentaion

Semantic segmentation can be seen as an extension of image classification from image level to pixel level. Semantic segmentation aims to assign a specific class label to each pixel within an image (WEI et al., 2016), facilitating a comprehensive pixel-by-pixel classification. By categorizing each pixel into predefined categories, such as "person," "car," or "road," semantic segmentation effectively partitions the image into distinct regions that correspond to particular objects or object parts. By providing such granular information, semantic segmentation serves as a foundation for applications requiring spatial precision and contextual understanding, making it an indispensable tool in fields like autonomous systems, healthcare, and remote sensing. These applications depend on the ability to accurately segment, analyze, and understand the scenes they encounter to make informed decisions.

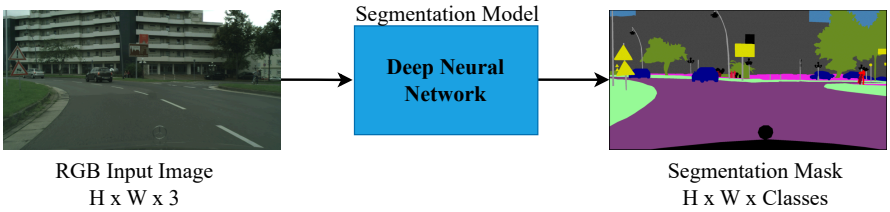


Figure 2.1: The process of semantic segmentation.

Figure 2.1 illustrates the black-box view of this process. It starts with a natural image captured from the real world, which contains a variety of

objects and regions, each with distinct visual characteristics. The segmentation model then analyzes this image, assigning a specific label to each pixel based on its semantic category. The outcome is a segmentation mask or segmentation map, an image where each pixel is color-coded according to its assigned category, transforming the raw image into a rich and clear representation of the scene. This process leverages spatial relationships within the image to group pixels that share similar properties, enabling the delineation of fine boundaries between different regions. For instance, in an urban street scene, the model might identify various elements, such as cars, pedestrians, and traffic signs, while also distinguishing between different parts of the same object, such as separating the road from the sidewalk. This fine-grained segmentation enhances vehicle navigation, ensuring safe and efficient driving.

In medical imaging, semantic segmentation plays a critical role in accurately identifying and segmenting organs, tumors, and other anatomical structures. This capability is invaluable in diagnostics and treatment planning, as it allows for precise delineation of regions of interest. Similarly, in satellite imaging, semantic segmentation helps distinguish between land types, such as urban areas, vegetation, and bodies of water, supporting land-use classification and urban planning. These examples demonstrate the versatility and importance of semantic segmentation in solving complex real-world problems. By partitioning images into semantically meaningful regions, this task enables machines to interpret and understand the visual world with precision, making it a powerful tool across diverse fields.

At the heart of semantic segmentation are Deep Neural Networks (DNNs), which have demonstrated exceptional power in handling such complex tasks.

2.3 Deep Neural Networks

Earlier approaches to semantic segmentation primarily utilized Convolutional Neural Networks to extract features and predict the segmentation class for each pixel. Designed for grid-like data such as images, CNNs employs convolutional and fully connected layers to identify patterns like edges, textures, and shapes (LeCun et al., 1989), (LeCun et al., 2015). This makes them particularly effective in interpreting the structure of objects

within an image. The use of CNNs in semantic segmentation models has evolved with significant diversity (SULTANA et al., 2020). In (FARABET et al., 2013), the authors employed a multi-scale convolutional neural network (CNN) for scene labeling, achieving state-of-the-art results. The R-CNN developed by (GIRSHICK et al., 2014) utilized the selective search algorithm (UIJLINGS et al., 2013) to first extract region proposals, after which CNN was applied to each proposal for the PASCAL VOC semantic segmentation challenge (EVERINGHAM et al., 2007). Meanwhile, (GUPTA et al., 2014) combined CNN with geocentric embedding to process RGB-D images for semantic segmentation.

However, early CNNs faced a notable limitation: their reliance on fully connected layers at the end of the architecture resulted in the loss of spatial information from images. This made it difficult to apply them to tasks like semantic segmentation, where preserving spatial information is essential. A pivotal breakthrough occurred with the development of Fully Convolutional Networks (FCNs) by (LONG et al., 2017), which addressed this issue by eliminating the fully connected layers typically found in conventional CNNs with 1×1 convolutional layers. This modification allowed the networks to retain the spatial information of an image throughout the network. This preservation enables FCNs the production of segmentation maps that align directly with the input image, regardless of its dimensions. Furthermore, the exclusive use of convolutional layers improves computational efficiency through the utilization of shared weights and local receptive fields, resulting in accelerated processing speeds and decreased memory consumption. As a result, Fully Convolutional Networks (FCNs) enabled neural networks to process images of any size and predict a label for each pixel.

On the other hand, (CHEN et al., 2016) combined the atrous algorithm with Conditional Random Fields (CRF) to develop DeepLab. Subsequently, the authors introduced Atrous Spatial Pyramid Pooling (ASPP) in DeepLabv2 (CHEN et al., 2017a). DeepLabv3 (CHEN et al., 2017b) advanced further by employing a cascaded deep ASPP module to integrate multiple contextual information. All three versions of DeepLab have achieved impressive results in semantic segmentation.

Deconvnet, as described by (NOH et al., 2015), employs a convolutional network followed by a hierarchically opposed de-convolutional network

for semantic segmentation. Similarly, (RONNEBERGER et al., 2015) introduced a U-shaped architecture called U-Net, which consists of a contracting pathway and an expansive pathway to tackle semantic segmentation tasks. The contracting pathway is responsible for extracting feature maps while reducing spatial information, much like traditional convolutional networks. In contrast, the expansive pathway takes the contracted feature map as input and performs an up-convolution. This architecture enables precise pixel-level predictions and lays the groundwork for several subsequent innovations in CV, particularly in encoder-decoder models as detailed in subsection 2.3.1. These advancements have further enhanced the model's ability to capture and leverage spatial hierarchies and contextual information effectively. The work by LIN et al. (2017) utilizes the inherent multi-scale, pyramidal structure of deep convolutional networks to create feature pyramids for object detection. This approach has also been applied to semantic segmentation, as demonstrated in subsection 4.2.2. It features top-down architecture with lateral connections designed to generate high-level semantic feature maps across all scales. Inspired from Natural Language Processing (NLP), more recent approaches use transformer-based architectures such as (DOSOVITSKIY et al., 2020), (LIU et al., 2021), (ZHENG et al., 2021a). These models leverage attention mechanisms to effectively capture long-range dependencies within the feature maps, as elaborated in subsection 2.3.2.

2.3.1 Encoder Decoder Model

The encoder-decoder architecture has significantly impacted CV research, offering a strong framework for addressing complex visual tasks like semantic segmentation. Innovations like de-convolutions and skip connections have contributed to the ongoing evolution of the architecture, improving its ability to capture and reconstruct detailed visual information. The general architecture of the model is shown in Figure 2.2, which consists of an encoder and a decoder.

2.3.1.1 Encoder

The encoder is a foundational component of the encoder-decoder architecture, designed to extract essential features from an input image. It can

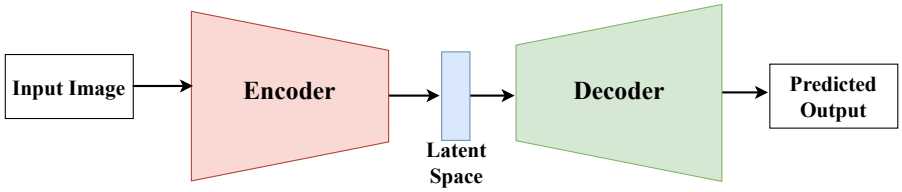


Figure 2.2: Encoder Decoder Architecture

be implemented using CNNs, transformers, or a hybrid approach combining the strengths of both. The encoder processes the input by progressively reducing its spatial dimensions while enriching the depth of its feature representations, transforming the raw data into a high-level, compact feature vector suitable for downstream tasks such as segmentation or reconstruction.

In traditional implementations, the encoder consists of a series of convolutional layers (BADRINARAYANAN et al., 2016), often paired with activation functions like ReLU, followed by pooling layers such as max pooling or average pooling. The convolutional layers detect localized patterns such as edges, corners, and textures, while deeper layers capture more abstract and complex features. Pooling layers reduce the spatial resolution of feature maps, concentrating on the most salient aspects of the input. This approach not only minimizes computational complexity but also helps the model generalize better by discarding less significant details.

In recent advancements, transformers, as detailed in subsection 2.3.2, have been incorporated into the encoder to leverage their ability to model global relationships within the image(CARION et al., 2020). Transformers utilize self-attention mechanisms and feed-forward neural networks, allowing them to capture long-range dependencies and contextual information that CNNs might overlook due to their localized receptive fields. In hybrid architectures, transformers can be integrated with CNN-based layers, where CNNs handle local feature extraction and transformers augment the representation by encoding global context. This combination is particularly beneficial in tasks requiring fine-grained detail and holistic understanding.

Regardless of the underlying architecture, the encoders primary role remains consistent: to convert the input image into a compact, high-

dimensional latent representation. By progressively downsampling and abstracting the input while preserving critical information, the encoder enables accurate and efficient interpretation of complex visual data. This encoded representation is then passed to the decoder, which utilizes it to generate the desired output, such as segmentation masks, reconstructed images, or classification results.

2.3.1.2 Decoder

The decoder plays a crucial role in the encoder-decoder architecture. It is responsible for reconstructing the spatial dimensions of the input image from the encoded representation. Its primary purpose is to interpret the compressed, high-dimensional features generated by the encoder and progressively restore them to the original image's resolution. This process involves upsampling operations that expand the feature maps and refine their details to ensure accurate pixel-level predictions. In semantic segmentation tasks, the decoder produces a set of feature maps, each representing the likelihood of a pixel belonging to a specific class, ultimately resulting in a dense prediction map for semantic segmentation.

To improve the reconstruction process, many encoder-decoder architectures incorporate skip connections. These connections link corresponding layers in the encoder and decoder, allowing the decoder to integrate high-resolution features from the early stages of the encoder with its own upsampled features. This combination of low-level spatial details and high-level semantic information is particularly beneficial for capturing fine-grained structures and resolving ambiguities, making it effective in applications such as semantic segmentation.

2.3.2 Transformers

One of the significant drawbacks associated with CNN-based architectures is the relatively low resolution of the final output segmentation image resulting from multiple convolutional and pooling layers. Additionally, the inherent locality characteristic of CNN methods restricts their ability to capture long-range dependencies within feature maps. To address these issues, researchers have explored the adoption of Transformer archi-

tectures in the field of CV, building on their success in Natural Language Processing (NLP).

Transformers, first introduced by (VASWANI et al., 2023) to handle sequence-to-sequence machine translation, have truly transformed the landscape of NLP through their innovative self-attention mechanisms, which are adept at capturing long-range dependencies and complex contextual relationships. Transformers process input sentences as sets of tokens without any inherent order, which allows for parallel computation and greatly improves computational efficiency. Each token is converted into token embeddings, and positional information is subsequently added to the input embeddings through positional encoding. This process enables the model to differentiate between input tokens based on their relative or absolute positions. This capability is crucial for both NLP and CV tasks, where understanding sequential or spatial relationships is essential.

Central to the functionality of Transformers is self-attention, which allows each element in the input sequence to attend to every other element, dynamically weighing their importance. The attention mechanism allows the models to assign weights to each token in the input sequence in relation to every other token based on their relevance. This enables the model to capture the relationships and dependencies between input tokens regardless of their distance. Self-attention is mathematically expressed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{head}}}\right)V \tag{2.1}$$

Here, Q, K, and V are the input query, key, and value matrices derived from the input features, and (d_{head}) is the dimension of the head. The first step is to multiply the token embeddings with the learned weight matrices to obtain three vectors named the Query vector (Q), Key vector (K), and Value vector (V). Then, they compute a similarity between queries and keys using a dot product (QK^T), normalized by the square root of d_{head} to stabilize gradients. This is then passed via a softmax function to provide a normalized score. A score is now assigned to each token in the sequence, indicating their significance to the input token. Then, to eliminate the unnecessary tokens by giving a low score, they multiply the Value vector (V) by the score, providing a high score to the tokens of relevance. Finally, they aggregate all value vectors together to calculate a score for the input token.

Initially designed for language modeling and machine translation tasks, transformers have since made significant strides in the realm of CV. A highly effective strategy in training NLP models involves pre-training on extensive text corpora followed by fine-tuning on smaller, task-specific datasets. As transformers have high computational efficiency and scalability, it is easier to train on a large set of data(HOCHREITER, 1998). However, this approach posed challenges for traditional DNNs, especially in CV domain. The success of using self-attention in boosting input-output interactions in NLP has paved the way for innovative approaches to combine convolutional architectures with self-attention (RAMACHANDRAN et al., 2019) in CV, particularly in areas like object detection and semantic segmentation, where input-output interaction is highly needed.

Nevertheless, integrating attention mechanisms into convolutional architectures demands considerable computational resources, even though they are theoretically efficient(VASWANI et al., 2023). This is primarily due to the quadratic relationship between the computation of self-attention and the size of the image. Since each pixel attends to every other pixel, the computational cost increases quadratically with the number of pixels, leading to substantial computational costs[(DOSOVITSKIY et al., 2020)]. To tackle this challenge, researchers proposed partitioning images into smaller squares or patches, treating these patches as tokens similar to those in NLP. A significant development in this area is the Vision Transformer (ViT) (DOSOVITSKIY et al., 2020), which processes images as sequences of fixed-size patches. This novel approach has led to substantial advancements in image classification and segmentation. By employing attention mechanisms at the patch level rather than the pixel level, ViT drastically reduces computational complexity while maintaining high performance, demonstrating its effectiveness in handling large-scale image data.

2.3.2.1 Vision Transformer (ViT)

The core idea behind ViT is to leverage the self-attention mechanism of transformers to capture global dependencies and contextual information within images. By processing the entire image in one holistic view, ViT eliminates the need for handcrafted features or spatial hierarchies, offer-

ing a more flexible and adaptive approach to image understanding as presented in Figure 2.3.

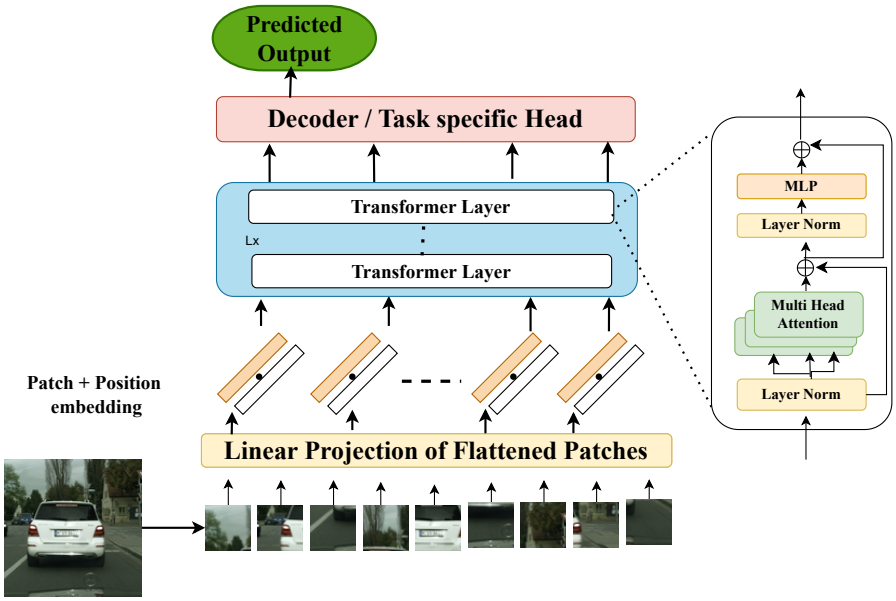


Figure 2.3: Vision Transformer Architecture

ViT works by dividing an input image into fixed-size patches, typically 16x16 or 32x32 pixels, forming a grid-like structure. Each patch is then linearly embedded into a lower-dimensional feature space using a learnable linear projection, resulting in patch embeddings also known as feature vectors. The patch embeddings are then passed through multiple layers of the transformer encoder. Each encoder layer consists of self-attention mechanisms and feed-forward neural networks. In the self-attention mechanism, each patch attends to all other patches in the image, capturing global dependencies and contextual information. The feed-forward neural networks process the attended features, allowing the model to learn complex representations of the input image. Since transformers do not inherently encode spatial information, positional encodings are added to the patch embeddings to provide information about the spatial arrangement of patches within the image. These positional encodings can take various forms, such as sinusoidal functions or learned embeddings, and are added to the patch embeddings before feeding them into the transformer encoder layers(ZHANG und GONG, 2023), (CHU et al., 2023).

The self-attention mechanism is typically implemented as multi-head attention within each transformer encoder layer. This involves splitting the input feature vectors into multiple heads and computing separate attention scores for each head. The outputs of the attention heads are then concatenated and linearly transformed to produce the final output of the self-attention layer. After the self-attention mechanism, the attended feature vectors pass through a Feed-Forward Network (FFN) within each transformer encoder layer. The FFN consists of two linear transformations separated by a non-linear activation function, such as GeLU (Gaussian Error Linear Unit). This allows the model to capture complex patterns and interactions between patches in the image(BAI et al., 2023). Layer normalization and residual connections are applied after each sub-layer (self-attention and feed-forward neural network) within the transformer encoder layers. Layer normalization helps stabilize training by normalizing the activation of each layer, while residual connections facilitate gradient flow and alleviate the vanishing gradient problem(YAO et al., 2021). The output of the final transformer encoder layer is typically pooled across all patches to produce a global representation of the input image. This global representation is then passed through a classification head consisting of one or more fully connected layers, followed by a softmax activation function for image classification tasks. For other complex tasks, such as object detection and segmentation, additional heads may be added to predict bounding boxes, class labels, and other relevant attributes. Many new architectures utilizing ViTs as a backbone to extract features have emerged and been successfully implemented for several segmentation tasks in recent years. (ZHENG et al., 2021b), (LIU et al., 2021), (XIE et al., 2021), (HE et al., 2021).

Building on this foundation, transformers have been successfully integrated into encoder-decoder architectures, further enhancing their capabilities. The SegFormer model, introduced by (XIE et al., 2021), is a prime example of this integration. This thesis utilized the SegFormer's encoder, and the detailed architecture of SegFormer is described in subsection 4.2.1.

2.4 Uncertainty

With the growing prevalence of DNNs in critical applications, confidence in their predictions has become increasingly significant. Humans often overestimate their abilities, a cognitive bias known as the Dunning-Kruger effect (KRUGER und DUNNING, 1999). Intriguingly, DNNs exhibit a similar tendency: they often produce overconfident predictions, even when incorrect (GUO et al., 2017). This overconfidence becomes more pronounced under domain shifts, where model performance typically deteriorates (OVADIA et al., 2019).

To address these challenges, researchers have focused on understanding and quantifying uncertainty in neural network predictions (GAWLIKOWSKI et al., 2022). Uncertainty estimation not only enables models to communicate their confidence levels but also provides a mechanism to improve decision-making in scenarios where incorrect predictions can have severe consequences. For tasks such as semantic segmentation, understanding the reliability of predictions is just as important as the predictions themselves. Uncertainty in machine learning can broadly be categorized into two types: aleatoric uncertainty and epistemic uncertainty.

2.4.1 Aleotoric Uncertainty

Aleatoric uncertainty arises from inherent noise or randomness in the data, such as ambiguous pixel-level labeling in segmentation tasks. For instance, blurry scans or overlapping tissue regions in medical imaging introduce ambiguity that cannot be resolved, no matter how sophisticated the model is. Similarly, in autonomous vehicles, uncertainty can stem from various unpredictable elements in the driving environment. For example, poor weather conditions such as heavy rain or fog may obscure road markings or traffic signs, making them difficult to detect. Shadows cast by trees or buildings can resemble obstacles, confusing the vehicle's perception systems. Another common challenge is the presence of unseen or ambiguous objects on the road, such as debris, fallen branches, or stray animals, which the model may struggle to classify or react to appropriately. This type of uncertainty is data-dependent and irreducible without improving the quality or clarity of the input data.

2.4.2 Epistemic Uncertainty

Epistemic uncertainty arises from a model's lack of knowledge due to limited training data or an incomplete understanding of the underlying data distribution. It can also be caused by model limitations, such as errors in the training process, an inadequate model structure, or insufficient coverage of the training dataset, which may include unknown or underrepresented samples. Unlike aleatoric uncertainty inherent to the data, epistemic uncertainty can be reduced by acquiring additional data, enhancing the model's architecture, or improving the learning process.

2.5 Confidence Calibration

Confidence calibration the problem of predicting probability estimates representative of the true correctness likelihood is important for all DNNs models. In real-world decision-making systems, neural networks must not only be accurate but also should indicate when they are likely to be incorrect. Calibrating deep neural models plays an important role in building reliable, robust AI systems in safety-critical applications. As an example, consider a self-driving car that uses a neural network to detect pedestrians and other obstructions. If the detection network is not able to confidently predict the presence or absence of immediate obstructions, the car should rely more on the output of other sensors for braking. Alternatively, in automated health care, control should be passed on to human doctors when the confidence of a disease diagnosis network is low. Specifically, a network should provide a calibrated confidence measure in addition to its prediction (GUO et al., 2017). In other words, the probability associated with the predicted class label should reflect its ground truth correctness likelihood.

In a supervised multiclass classification setting, the input $X \in \mathcal{X}$ and label $Y \in \mathcal{Y} = \{1, ..., K\}$ are random variables that follow a ground truth joint distribution $\pi(X, Y) = \pi(Y \mid X)\pi(X)$. Let h be a neural network with $h(X) = (\hat{Y}, \hat{P})$, where $\hat{Y}$ is a class prediction and $\hat{P}$ is confidence score associated with the predicted class label (i.e., the estimated probability of correctness). We would like the confidence estimate $\hat{P}$ to be calibrated, which intuitively means that $\hat{P}$ accurately reflects the true probability of the pre-

diction being correct. For example, given 100 predictions, each with a confidence of 0.8, we expect that 80 should be correctly classified. More formally, we define perfect calibration as

$$P(\hat{Y} = Y \mid \hat{P} = p) = p, \quad \forall p \in [0,1] \tag{2.2}$$

Where the probability is over the joint distribution. This equation in Equation 2.2 states that if the model predicts a confidence score of 'p' for a certain class, then the actual probability of that prediction being correct should also be 'p'. In all practical settings, achieving perfect calibration is impossible. Additionally, the probability in Equation 2.2 cannot be computed using finitely many samples since $\hat{P}$ is a continuous random variable.

Calibrated confidence estimates are also crucial for model interpretability. Good confidence estimates provide a valuable extra bit of information to establish trustworthiness with the user, especially for neural networks, whose classification decisions are often difficult to interpret. Reliability diagrams visually represent model calibration, while the expected calibration error serves as a summary statistic, detailed in section 3.5 and section 3.4, respectively.

2.6 Out-of-Distribution Detection

Inputs belonging to the classes or distributions encountered during training are referred to as In-distribution (ID) inputs. Conversely, samples that originate from classes or distributions not observed during training are categorized as Out of Distribution (OOD) inputs. Detecting OOD inputs is a crucial step in building safe and reliable Machine Learning (ML) systems. Identifying anomalous inputs allows systems to initiate fallback policies, such as adopting a conservative response or deferring decisions to human intervention and this capability is especially vital in safety-critical domains where an incorrect prediction can have severe consequences.

OOD detection is crucial for ensuring the safety of machine learning systems, as highlighted by ((HENDRYCKS et al., 2022)). While modern neural networks can generalize well when the training and testing data come from the same distribution ((KRIZHEVSKY et al., 2012), (SIMONYAN und

ZISSERMAN, 2015), (HE et al., 2015)), their reliability significantly deteriorates in real-world scenarios where the input distributions are often unpredictable and uncontrolled. Studies have demonstrated that neural networks can produce high-confidence predictions for completely unrecognizable (NGUYEN et al., 2015) or irrelevant inputs ((GOODFELLOW et al., 2015), (HENDRYCKS und GIMPEL, 2018), (SZEGEDY et al., 2014)). This overconfidence on OOD samples highlights the need for models to accurately estimate uncertainty and distinguish novel inputs from familiar ones ((AMODEI et al., 2016)). The ability to effectively detect OOD examples is particularly important for visual recognition tasks, where maintaining robust performance across diverse testing scenarios is critical (KRIZHEVSKY et al., 2012).

2.7 Softwares and Tools

The following is a list of the tools and libraries we utilized in our thesis for data processing, method implementation, and experimentation, along with a brief description.

2.7.1 Python

In this thesis, we use Python as the main programming language, as it has extensive standard libraries and frameworks for data analysis, DL, CV, and visualization methods. The following are the libraries that were used:

- **Pytorch:** Pytorch[1] is a popular open-source Machine Learning (ML) framework based on the Torch library and developed by Facebooks AI Research Lab (FAIR). Pytorch has two key features: GPU-accelerated tensor computation and neural networks based on an algorithmic differentiation framework. Therefore, it helps perform GPU or TPU training tasks.
- **segmentation_models:** segmentations_models[2] is a popular open-source Python library built on PyTorch for implementing state-of-the-art semantic segmentation models like U-Net, DeepLabV3, and

[1] `https://huggingface.co/`
[2] `https://github.com/qubvel/segmentation_models`

FPN. It offers a lightweight API supporting various encoders, including ResNet and EfficientNet, which are pretrained on ImageNet dataset. The library is modular, allowing easy customization of input channels, class numbers, and loss functions while being scalable across GPUs and TPUs. Its intuitive design and extensive documentation make it ideal for both research and production. Additionally, it integrates with frameworks like Keras and TensorFlow for enhanced flexibility.

- **Pandas:** Pandas [3] is a powerful open-source Python library built on NumPy for data manipulation and analysis. It provides data structures like Dataframes and Series that can be used for tasks such as data cleaning, data pre-processing, transformation, and analysis.

- **NumPy:** Numerical Python, or NumPy[4], is a Python library that is used for numerical and scientific computing. It offers a special ndarray (n-dimensional array) data structure, which enables efficient array operations and mathematical computations. NumPy is the foundation for many AI and ML libraries, providing the backbone for handling numerical data.

- **Plotly:** Plotly[5] is an open-source Python library for creating interactive and high-quality visualizations. It supports a wide range of charts, including scatter plots, line charts, bar graphs, and 3D plots, all with built-in interactivity. Plotly integrates seamlessly with Pandas and NumPy, making it ideal for data exploration and analysis. It is also the backbone of Dash, a framework for building interactive web-based dashboards. Plotlys versatility allows its use in Jupyter Notebooks, web applications, and standalone HTML files.

- **Hugging Face:** Hugging Face[6] is a popular library and an open-source community in the field of CV. It provides access to a wide range of datasets, transformer-based pre-trained models (ViT, DeTR, Mit_B3, MAE etc.), and resources for tasks such as image classification and image segmentation.

[3] `https://pandas.pydata.org/`
[4] `https://numpy.org/`
[5] `https://plotly.com/`
[6] `https://github.com/huggingface`

- **WandB:** Weights & Biases (WandB)[7] is a powerful tool for machine learning experimentation and collaboration. It enables tracking and visualizing metrics, managing datasets, and saving model checkpoints. WandB supports seamless integration with popular frameworks and provides tools for hyperparameter optimization and model versioning. Its collaborative features make sharing and comparing experiments easy across teams.

- **TensorBoard:** TensorBoard[8] offers essential tools for machine learning experimentation, enabling tracking and visualization of metrics like loss and accuracy. It provides a clear view of model graphs, including operations and layers, and tracks changes in weights, biases, and other tensors with histograms. TensorBoard also supports displaying images, text, and audio data.

- **Pytorch Lightning:** Pytorch Lightning[9] is a well-known package that provides a lightweight Pytorch wrapper developed by researchers at NYU CILVR and Facebook AI Research to make DL techniques easy to read and reproduce. It is mainly intended to develop scalable DL models that can run on distributed hardware, such as a GPU or TPU, while being hardware-agnostic.

[7] `https://wandb.ai/`
[8] `https://www.tensorflow.org/tensorboard`
[9] `https://lightning.ai/`

3

Related Work

This chapter reviews significant related work on uncertainty quantification in deep learning models. It highlights the techniques used in this thesis to estimate uncertainty. Additionally, we explore various methods for assessing model confidence calibration and detecting out-of-distribution instances in semantic segmentation systems.

Deep Neural Networks (DNNs) have gained significant recognition in various fields due to their ability to manage millions of parameters and complex non-linear relationships. This has enabled them to excel in areas like Natural Language Processing (NLP) and Computer Vision (CV), particularly in tasks such as semantic segmentation, as highlighted in the work by (BADRINARAYANAN et al., 2016). However, despite their remarkable capabilities, DNNs are unreliable(DE JORGE et al., 2023). They can sometimes produce outputs that are misleading or entirely incorrect. One of the challenges with DNNs arises from their intricate architectures, making it nearly impossible to determine their output's exact posterior probability distribution. This limitation prompts an essential question: how can users know when the output produced by the model is reliable? While one might expect that model confidence or output probability could serve as indicators of correctness where incorrect predictions are associated with low confidence and correct predictions are expected to carry high confidencethis is not always the case (GUO et al., 2017).

To address this challenge, uncertainty estimation has become a valuable tool for evaluating the trustworthiness of machine learning models. Research has shown that quantifying the uncertainty present in a model's predictions is an effective way to enhance the safety and reliability of these applications [(LEE et al., 2018), (DE JORGE et al., 2023), (LANDGRAF

et al., 2023b), (LANDGRAF et al., 2023a)]. This approach provides a practical means to navigate the complexities of calculating the exact posterior distribution, allowing practitioners to gauge the confidence they can place in a model's predictions. By understanding these uncertainties, users gain insights that extend beyond the typical softmax probabilities, highlighting areas where the model is uncertain and more likely to make errors.

Uncertainty quantification for semantic segmentation using deep learning is a well-studied topic (DECHESNE et al., 2021), with various approaches proposed to measure uncertainty in model results. The most prominent methods include Bayesian Neural Networks (MACKAY, 1992), Monte Carlo Dropout (GAL und GHAHRAMANI, 2016), and Deep Ensembles (LAKSHMINARAYANAN et al., 2017).

Bayesian Neural Networks (BNNs) offer a mathematically grounded, though computationally complex, approach to uncertainty quantification in deep learning. They return a probability distribution over outputs as an alternative way to represent model uncertainty. By transforming a deterministic network into a stochastic one, BNNs incorporate probabilistic distributions over the network's weights or activations, capturing the inherent uncertainty in model parameters(JOSPIN et al., 2022). In contrast to traditional neural networks, where weights are fixed, BNNs treat weights as random variables, allowing them to encode the uncertainty associated with model predictions. One popular method for training BNNs is Bayes by Backprop by (BLUNDELL et al., 2015), which utilizes variational inference to approximate the posterior distribution of the weights by optimizing a simpler distribution (e.g., Gaussian) that closely resembles the true posterior. This approach allows the model to sample from multiple weight configurations, creating an ensemble of models with different weights. These models can then be used to approximate the posterior distribution of predictions, providing not only point estimates but also uncertainty estimates, which are valuable in domains where understanding model confidence is crucial. However, the computational cost of BNNs remains challenging due to the need for sampling and approximation techniques, making them less practical for large-scale applications.

Deep Ensembles (LAKSHMINARAYANAN et al., 2017) are a state-of-the-art method for uncertainty quantification, assessing uncertainty by comparing the predictions of multiple, slightly different models trained on the

same task (LEE et al., 2020). They generate diverse predictions at test time due to randomness introduced through random weight initialization or varied data augmentations across ensemble members. This inherent diversity ensures well-calibrated probability estimates (NG et al., 2023) and robust performance. Multiple studies have demonstrated that Deep Ensembles outperform other uncertainty quantification techniques across domains [(GUSTAFSSON et al., 2020), (OVADIA et al., 2019)]. However, this superior performance comes with a trade-off in the form of high computational cost, as each model in the ensemble requires separate training and additional resources, making them resource-intensive for large-scale or real-time applications.

Recent studies have demonstrated that using student-teacher distillation techniques, even single models can effectively predict pixel-wise uncertainty in segmentation tasks [(LANDGRAF et al., 2024), (HOLDER und SHAFIQUE, 2021)]. These methods transfer the knowledge and uncertainty estimates from a larger, more complex ensemble of models to a single, lightweight model, significantly reducing the computational demands at inference time. By approximating the predictive distribution of the ensemble, the distilled model retains the ability to provide reliable uncertainty estimates while maintaining high accuracy.

Despite the potential benefits, the role of uncertainties in training segmentation models remains relatively underexplored. This thesis aims to fill this gap by integrating dynamic uncertainty estimates directly into the training process of transformer-based semantic segmentation models, as illustrated in section 4.4. Specifically, we employ three accessible and effective uncertainty quantification techniques: section 3.1, section 3.2, and section 3.3. A detailed discussion of each technique follows in the sections below.

3.1 Monte Carlo Dropout

(GAL und GHAHRAMANI, 2016) simplified the approximation of posterior distributions by establishing a pivotal connection between model uncertainty and Dropout((SRIVASTAVA et al., 2014)). The authors showed that sampling models with dropped nodes at inference time can approximate the probability distribution over possible model predictions for a

given input. This approach called Monte Carlo Dropout (MCD), reframes dropout, traditionally a regularization technique (SRIVASTAVA et al., 2014), as a practical tool for uncertainty quantification in deep learning. MCD operates within a Bayesian framework, allowing the network to approximate posterior distributions without explicitly storing or sampling a large set of weights. By applying dropout at test time and performing multiple stochastic forward passes through the network, MCD generates a distribution of predictions, enabling the estimation of epistemic uncertainty, which reflects the uncertainty in the model's parameters due to limited training data. In its original form, MCD only captures the epistemic uncertainty inherent to the model.

(KENDALL und GAL, 2017) further extended this idea by combining MCD with a model designed to output both a predictive mean and variance for each data point, effectively capturing aleatoric uncertainty in addition to epistemic uncertainty. They combined learned uncertainty predictions and assumed density filtering with MCD to achieve this, making the overall uncertainty estimation more comprehensive. More closely to our work, (LANDGRAF et al., 2023a) utilized uncertainty aware cross-entropy loss to facilitate semantic segmentation using conventional CNNs. The authors employed MCD with convolutional architectures, such as ResNet-18 and ResNet-108, as backbone networks to train the model. (WANG et al., 2019) combined MCD with test-time image augmentation to encompass data uncertainties. (NAIR et al., 2018) compared different uncertainty measures in brain lesion segmentation and showed that uncertainty measures could be used to improve lesion detection accuracy. (SANDER et al., 2019) applied MCD for cardiac MRI segmentation. They showed that training CNNs using a Brier loss or cross-entropy loss produced well-calibrated pixel-wise segmentation uncertainty, and correcting uncertain pixels can improve segmentation results consistently.

MCD has proven effective in addressing uncertainties arising from input data variations. It offers a scalable and computationally efficient method for uncertainty estimation, particularly compared to more complex approaches like Deep Ensembles. This thesis also employs MCD to train a transformer-based semantic segmentation model. We can compute each pixel's predictive uncertainty by applying MCD sampling during training, enabling us to use pixel-wise weighting with the established cross-entropy

loss. Predictive uncertainty refers to the standard deviation of the softmax probabilities for the predicted classes of every pixel obtained through Monte Carlo Dropout (MCD) sampling. subsection 4.4.1 will offer a comprehensive architecture description.

3.2 Online Hard Example Mining

Online Hard Example Mining (OHEM) is a dynamic training strategy aimed at improving the performance of deep learning models by focusing on challenging samples during the training process. Traditional training methods often prioritize easy examples or allow them to dominate the loss calculation, hindering the model's ability to learn from complex cases . OHEM addresses this by dynamically selecting the most challenging examples with the largest prediction errors or, equivalently, the highest uncertainty during training. This strategy is particularly effective for pixel-level tasks like semantic segmentation, where class imbalance, such as large background regions, often overshadows smaller, harder-to-classify objects.

The workflow of the OHEM strategy, as outlined in 1, primarily involves two key parameter settings: 'thresh' and 'mining_percent'. The 'thresh' parameter represents the threshold score; only pixels with loss scores greater than this threshold will be included in the training process. We call such pixels hard pixels. The 'mining_percent' parameter indicates the minimum percent of pixels that must be retained in the training batch. This requirement helps to consolidate the training results and ensures that forward propagation is not wasted.

Proposed initially by (SHRIVASTAVA et al., 2016) for object detection, OHEM has been adapted for semantic segmentation, specifically targeting hard examples that correspond to regions of high uncertainty. The authors computed feature maps for an image only once and employed a Region Proposal Network (RPN) to generate candidate regions of interest (RoIs). Each RoI was assigned a loss value, and the hardest RoIs with the highest loss were selected for training to facilitate backpropagation. By prioritizing these challenging regions, the model could concentrate on improving predictions in uncertain areas, thereby enhancing segmentation performance while minimizing computational costs.

Algorithm 1 Online Hard Example Mining (OHEM)

Input:

Training data: $X = \{(x_i, y_i)\}_{i=1}^{N}$ (input images x_i and ground truth y_i)
Model predictions: $P = \{p_i\}_{i=1}^{N}$ (predicted probabilities for each class)
Parameters:

- thresh: Minimum loss threshold for hard examples.
- mining_percent: Percentage of pixels to retain for training.

Step 1: Forward Pass

Compute the predicted probabilities p_i and cross-entropy loss for each pixel:

$$L_i = -\sum_{c \in C} y_{i,c} \log(p_{i,c})$$

Step 2: Determine Number of Hard Pixels

Calculate the number of pixels to retain:

$$\text{num_mining} = \text{mining_percent} \times \text{num_pixels}$$

Step 3: Rank and Select Hard Pixels

Sort all pixels by their loss values in descending order.
Compute a dynamic loss threshold:

$$\text{threshold} = \max(\text{thresh}, \text{sorted_loss}[\text{num_mining}])$$

Identify hard pixels where $L_i \geq$ threshold.

Step 4: Handle Edge Cases

If fewer hard pixels exist than num_mining, select the top num_mining pixels regardless of the threshold.

If more hard pixels exist than num_mining, select only the top num_mining pixels based on the threshold.

Step 5: Loss Calculation

Compute the batch loss using only the selected hard pixels:

$$L_{\text{batch}} = \frac{1}{|S|} \sum_{i \in S} L_i$$

where S is the set of selected hard pixels, and $|S|$ is its cardinality.

Step 6: Backward Pass

Perform backpropagation using L_{batch} to update the model parameters.

Step 7: Iterate

Repeat steps 16 for each batch during training.

Output: Optimized model parameters with enhanced focus on hard-to-classify pixels.

In semantic segmentation, OHEM is commonly used to improve performance on imbalanced datasets by concentrating on regions with high uncertainty. A notable application of OHEM is in FCNs, which prioritizes misclassified pixels or areas with high loss values. This focus enables the model to improve its understanding of complex structures. For instance, in the DeepLabV3 by (CHEN et al., 2017b), the authors incorporated OHEM to handle hard images presented by datasets like PASCAL VOC, which exhibit highly imbalanced pixel distributions and varying object scales. (LOSHCHILOV und HUTTER, 2016) studied mini-batch selection in terms of image classification. They picked hard training images from the whole training set according to their current losses, which were lazily updated once an image had been forwarded through the network being trained. (WU et al., 2016) proposed online bootstrapping of hard pixel samples for the problems of instance and semantic image segmentation. They search for hard pixels based on the current loss within the current mini-batch rather than the whole training set. (POHLEN et al., 2016) utilized hard pixel mining with a bootstrapped cross-entropy loss to train the segmentation model. The authors weighted the cross-entropy loss based on a threshold determined by ranking the predicted log probabilities for hard pixels.

By concentrating on difficult-to-classify pixels, OHEM enhances the models grasp of complex structures, such as object boundaries or small-scale objects, which are often areas of high epistemic uncertainty. This implicit targeting of uncertainty-rich regions makes OHEM a practical uncertainty-aware training approach.

OHEM has also been used in region-based semantic segmentation models to identify and focus on difficult regions during training (LI et al., 2017). By applying OHEM to region proposals of highest losses and back-propagating only their error gradients, these models effectively improve their segmentation accuracy on cluttered or occluded objects. In (LV et al., 2024), an importance-aware loss based on OHEM is invoked to focus on segmenting small and hard objects in autonomous driving scenarios such as riders, bicycles, motorcycles, etc., which constitutes a small proportion of the dataset.

(BIAN et al., 2018) utilizes a 2D network with a pyramid module for multi-scale feature extraction and employs OHEM to address low-certainty vox-

els for segmenting the left atrium in gadolinium-enhanced MR volumes. In (SCHMIDT-MENGIN et al., 2022), authors perform segmentation of Multiple Sclerosis lesions by modifying OHEM. They utilized an exponential moving average of the models weights to smooth predictions and provide consistent feedback for hard example selection. This innovative approach enhances training efficiency and model performance without requiring handcrafted priors or multi-stage methods.

All the works demonstrated the potential of OHEM to enhance various segmentation pipelines by implicitly addressing uncertainty and emphasizing hard-to-classify, uncertain pixels, particularly for smaller or intricately shaped objects. However, OHEM introduces computational overhead in selecting hard examples and calculating their loss. In this thesis, we examine the integration of OHEM as a strategy guided by uncertainty during the training process. Our focus is on addressing challenging and ambiguous regions encountered in training. By prioritizing these uncertain areas, OHEM has the potential to improve the model's ability to handle uncertainty and enhance its performance in critical semantic segmentation tasks.

3.3 Evidential Deep Learning

Evidential Deep Learning (EDL) (SENSOY et al., 2018) is an innovative approach in machine learning that addresses uncertainties in decision-making. Traditional deep learning models often produce deterministic predictions, which can lead to overconfidence, especially when encountering ambiguous or novel data. EDL tackles this limitation by explicitly quantifying uncertainty in its predictions, drawing from concepts like Dempster-Shafer theory of evidence (YAGER und LIU, 2010) and Bayesian inference.

In conventional deep learning, the standard approach involves training the network to minimize a prediction loss. Given an input $\mathbf{x}$, the network predicts a categorical probability distribution over K classes as represented in Equation 3.1.

$$\textbf{Output} = (p_1, \ldots, p_K \mid \sum_{k=1}^{K} p_k = 1) \quad \text{over } K \text{ classes.} \tag{3.1}$$

where the predicted probabilities sum to one. Formally, for a K-class classification problem, the likelihood function for an observed tuple $(\mathbf{x}, y)$ is given by:

$$Pr(y \mid \mathbf{x}, \boldsymbol{\theta}) = \text{Mult}\big(y \mid \sigma(f_1(\mathbf{x}, \boldsymbol{\theta})), \ldots, \sigma(f_K(\mathbf{x}, \boldsymbol{\theta}))\big), \tag{3.2}$$

where Mult$(\cdot)$ is the multinomial mass function, $f_k(\mathbf{x}, \boldsymbol{\theta})$ is the kth output channel of an arbitrary neural net $f(\cdot)$ parameterized by $\boldsymbol{\theta}$, and $\sigma(u_k)$ is the softmax function (Equation 3.3) that converts the continuous activations of the output layer to class probabilities.

$$\sigma(u_k) = \frac{e^{u_k}}{\sum_{k=1}^{K} e^{u_k}}. \tag{3.3}$$

The continuous neural network adjusts the ratio of class probabilities through its outputs, $f_k(\mathbf{x}, \boldsymbol{\theta})$, while the softmax function squashes these ratios into a probability simplex. The resulting softmax-squashed multinomial likelihood is then maximized with respect to the neural network parameters $\boldsymbol{\theta}$. For computational convenience, the equivalent problem of minimizing the negative log-likelihood is preferred. This is expressed as:

$$-\log Pr(y \mid \mathbf{x}, \boldsymbol{\theta}) = -\log \sigma(f_y(\mathbf{x}, \boldsymbol{\theta})). \tag{3.4}$$

which is widely known as the cross-entropy loss. Notably, the probabilistic interpretation of the cross-entropy loss is mere Maximum Likelihood Estimation (MLE). As being a frequentist technique, MLE is not capable of inferring the predictive distribution variance. Softmax is also notorious for inflating the probability of the predicted class due to the exponent employed on the neural net outputs. The consequence is unreliable uncertainty estimations, as the distance of the predicted label of a newly seen observation is not helpful for the conclusion besides its comparative value against other classes.

EDL is built upon the DempsterShafer Theory of Evidence (DST) (YAGER und LIU, 2010), which is a generalization of the Bayesian theory to subjective probabilities. Orthogonally to Bayesian neural nets that indirectly infer predictive uncertainty through weight uncertainties, EDL employs explicit modeling of the same using the subjective logic theory. By placing

a Dirichlet distribution on the class probabilities, EDL treats predictions of a neural net as subjective opinions and learns the function that collects the evidence leading to these opinions by a deterministic neural net from data.

To address this, EDL replaces the traditional softmax layer with a Dirichlet distribution, which models the parameters of the probability mass function for the possible classes. SENSOY et al. (2018) argued that a neural network can form opinions for classification tasks as Dirichlet distributions. If $\boldsymbol{\alpha}_i = (\alpha_{i1}, \ldots, \alpha_{iK})$ represents the parameters of a Dirichlet distribution for the classification of a sample i, then $e_{ik} = (\alpha_{ik} - 1)$ corresponds to the total evidence estimated by the network for assigning sample i to the k^{th} class.

Instead of directly predicting class probabilities, EDL outputs a distribution of evidence, quantifying the models belief in each class based on the input data. The model trains by accumulating pseudo-counts of evidence, which measure the support collected from the data for classifying a sample into a particular class (DAVIES et al., 2023). The Dirichlet distribution is commonly used to represent this evidence in classification tasks, allowing the model to capture both the strength of its evidence and the level of uncertainty when evidence is low or conflicting.

This method modifies the network's output layer to predict the parameters of the Dirichlet distribution and adjusts the loss function to achieve two objectives: minimizing the discrepancy between predicted evidence and true labels while penalizing overconfident predictions, particularly in uncertain or incorrect scenarios. This dual-objective training ensures that the model remains cautious when evidence is insufficient. Furthermore, given the Dirichlet parameters, the epistemic uncertainty of the classification can be easily computed using Equation 3.6.

From the evidence, the belief masses (b_k) and the uncertainty (u) for each class can be calculated as follows. Let $e_k \geq 0$ be the evidence derived for the k-th class:

$$b_k = \frac{e_k}{S} \tag{3.5}$$

A belief mass assignment, i.e., subjective opinion, corresponds to a Dirichlet distribution with parameters $\alpha_k = e_k + 1$. That is, a subjective opinion

can be derived easily from the parameters of the corresponding Dirichlet distribution using $b_k = \frac{\alpha_k - 1}{S}$.

$$u = \frac{K}{S} \tag{3.6}$$

where K is the number of classes and $S = \sum_{k=1}^{K}(e_k + 1)$, which is the sum of evidence over all classes, is referred to as the Dirichlet strength.

The usual softmax function provides a point estimate for the class probabilities of a sample and does not provide the associated uncertainty. On the other hand in EDL, multinomial opinions or equivalently Dirichlet distributions can be used to model a probability distribution for the class probabilities. The neural networks were trained to form their multinomial opinions for the classification of a given sample i as a Dirichlet distribution $D(p_i \mid \alpha_i)$, where p_i represents class assignment probabilities. EDL based neural networks for classification are very similar to classical neural networks. The only difference is that the softmax layer is replaced with an activation layer, e.g., ReLU or softplus, to ascertain non-negative output, which is taken as the evidence vector for the predicted Dirichlet distribution.

For a sample i, $f(\mathbf{x}_i \mid \boldsymbol{\Theta})$ represent the evidence vector predicted by the network for the classification, where $\boldsymbol{\Theta}$ is network parameters. Then, the corresponding Dirichlet distribution has parameters $\boldsymbol{\alpha}_i = e_i + 1 = f(\mathbf{x}_i \mid \boldsymbol{\Theta}) + 1$. Once the parameters of this distribution are calculated, its mean, i.e., $P_{ik} = \frac{\alpha_{ik}}{S_i}$ can be taken as an estimate of the class probabilities.

During training, the model can discover patterns in the data and generate evidence for specific class labels such that the overall loss is minimized. However, these features may also be present in counter-examples, giving rise to misleading evidence. Reducing the magnitude of generated evidence may increase the overall loss, despite reducing the loss contributed by these counter-examples. To combat this, a regularization term is included, which incorporates a Kullback-Leibler (KL) divergence term between a uniform Dirichlet distribution and $\tilde{\boldsymbol{\alpha}}$, where $\tilde{\boldsymbol{\alpha}}$ is the parameters of the output Dirichlet distribution $\boldsymbol{\alpha}$ after removing the non-misleading evidence from $f(\mathbf{x}_i \mid \boldsymbol{\Theta})$, such that a correctly classified sample with no evidence for other classes will generate $\tilde{\boldsymbol{\alpha}}$ as a uniform Dirichlet distribution.

To learn the parameters $\boldsymbol{\Theta}$ of a neural network, EDL defines the loss function as

$$L(\boldsymbol{\Theta}) = \sum_{i=1}^{N} L_i(\boldsymbol{\Theta}) + \lambda_t \sum_{i=1}^{N} \mathrm{KL}\left[D(\boldsymbol{p}_i \mid \tilde{\boldsymbol{\alpha}}_i) \parallel D(\boldsymbol{p}_i \mid \langle 1, \ldots, 1 \rangle)\right], \tag{3.7}$$

where $\lambda_t = \min\left(1.0, \frac{t}{\text{annealing_step}}\right) \in [0,1]$ is the annealing coefficient, t is the index of the current training epoch, annealing_step is the epoch index at which $\lambda_t = 1$, $D(\boldsymbol{p}_i \mid \langle 1, \ldots, 1 \rangle)$ is the uniform Dirichlet distribution and N is the number of samples in the dataset.

Several options for $L_i(\boldsymbol{\Theta})$ have been considered from (SENSOY et al., 2018), while most of the analysis in the original paper is performed using

$$L_i(\boldsymbol{\Theta}) = \sum_{j=1}^{K} (y_{ij} - \hat{p}_{ij})^2 + \frac{\hat{p}_{ij}(1 - \hat{p}_{ij})}{S_i + 1} \tag{3.8}$$

where y_i represents the one-hot vector encoding of the ground truth label for sample x_i with $y_{ij} = 1$ and $y_{ik} = 0$ for all $k \neq j$. This loss is called the Sum of Squares Bayes Risk Loss and aims to achieve the joint goals of minimizing the prediction error and the variance of the Dirichlet experiment generated by the neural net specifically for each sample in the training set.

The key strength of EDL is its ability to explicitly handle two types of uncertainty that are crucial for robust decision-making. Aleatoric uncertainty arises from inherent noise in the data, such as measurement errors or random fluctuations, which is irreducible even with perfect knowledge of the system. Epistemic uncertainty, on the other hand, stems from the model's lack of knowledge, often observed in situations with limited data or in out-of-distribution scenarios.

EDL directly models both forms of uncertainty by treating predictions as belief distributions, typically using a Dirichlet distribution. This allows the model to quantify not only the predictions themselves but also the confidence associated with them. In this thesis, we leverage EDL's capability to handle uncertainty to improve the robustness and reliability of predictions. This approach is particularly valuable when uncertainty plays a significant role, such as detecting rare events, handling noisy data, or managing out-of-distribution examples.

3.4 Expected Calibration Error

Semantic segmentation can be formulated as a multiclass classification problem with a deep neural network like image classification. Calibration refers to the alignment between a model's predicted probabilities and actual outcomes, meaning that if a model predicts a class with 70% confidence, that class should be correct approximately 70% of the time. We expect a well-calibrated model to provide an accurate prediction when its confidence is high.

Several works have been proposed to address miscalibration in probabilistic classification methods. Histogram binning, introduced by (ZADROZNY und ELKAN, 2001), is a non-parametric approach to calibration. It divides the range of uncalibrated predicted probabilities into mutually exclusive discrete bins and adjusts the confidence within each bin to match the observed frequency of correct predictions. While simple and effective, its performance is sensitive to the choice of bin size, potentially leading to overfitting in small datasets.

Isotonic regression, also proposed by (ZADROZNY und ELKAN, 2002), is another non-parametric method that learns a piecewise constant function to map predicted probabilities to calibrated outputs. It offers flexibility and performs well with sufficient data but may overfit when data is limited, especially for higher-dimensional problems.

Platt scaling, introduced by (PLATT, 1999), is a parametric calibration method that transforms non-probabilistic predictions into calibrated probabilities using logistic regression. In the context of neural networks, it learns two scalar parameters, $a, b \in \mathbb{R}$, to produce calibrated probabilities as $\hat{q}_i = \sigma(az_i + b)$, where z_i are the model's logits, and $\sigma(x)$ is the sigmoid function. These parameters are optimized on a validation set using the negative log-likelihood (NLL) loss, keeping the neural network weights fixed.

Temperature scaling, popularized by (GUO et al., 2017), is a parametric technique that adjusts the models logits using a single scalar parameter (temperature). They divide the output logits before the softmax layer of a neural network by temperature parameter in a way that softens them. It is applied post-hoc to rescale the predicted probabilities, minimizing

calibration error while preserving classification accuracy. Despite its simplicity, temperature scaling assumes the model is well-trained and only addresses overconfidence, limiting its applicability in cases of underconfidence.

Expected Calibration Error (ECE) (PAKDAMAN NAEINI et al., 2015) has become the de facto standard to evaluate the calibration of probabilistic models, particularly in classification tasks. ECE quantifies the alignment of predictive confidence and accuracy by partitioning predictions into confidence intervals (bins) and measuring the weighted average of the differences between the model's accuracy and confidence in each bin.

The formal definition of miscalibration is the difference in expectation between confidence and accuracy.

$$\text{ECE} = \mathbb{E}_{\hat{p}}\left[\left|\mathbb{P}(\hat{y} = y \mid \hat{p} = p) - p\right|\right] \tag{3.9}$$

Where $\hat{y}$ is the predicted label, y is the true label, p is expected confidence, and $\mathbb{P}(\hat{y} = y \mid \hat{p} = p)$ is model predictive accuracy. The expectation is about the discrepancy between accuracy and confidence.

(PAKDAMAN NAEINI et al., 2015) approximates Equation 3.9 by partitioning predictions into M equally-spaced bins (similar to the reliability diagrams section 3.5) and taking a weighted average of the difference between accuracy and confidence across these bins. This difference between accuracy and confidence for a given bin represents the calibration gap (red bars in reliability diagrams, e.g., Figure 3.1). Formally,

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|, \tag{3.10}$$

Where:

- M: Total number of confidence bins, typically evenly spaced.
- B_m: The set of samples whose predicted confidence scores fall into the m-th bin.
- $|B_m|$: The number of samples in the m-th bin.
- N: The total number of samples.

- acc(B_m): The accuracy of the m-th bin, defined as the fraction of correctly classified samples in the bin.
- conf(B_m): The average predicted confidence of the samples in the m-th bin.

A lower ECE indicates better calibration, reflecting a model that is neither overconfident (predicting higher probabilities than their true likelihood of correctness) nor underconfident (predicting lower probabilities than their true likelihood of correctness). A perfectly calibrated model has an ECE of zero. ECE provides a single scalar value that represents the overall calibration error of the model. Our thesis uses ECE as the primary empirical metric to measure calibration.

3.5 Reliability Diagrams

Reliability diagrams, introduced by (DEGROOT und FIENBERG, 1983) and later explored by (NICULESCU-MIZIL und CARUANA, 2005), visually represent a models calibration. They plot the expected accuracy as a function of the models predicted confidence scores. A perfectly calibrated model is one where the predicted confidence corresponds exactly to the observed accuracy. For example, for predictions with a confidence score of 0.8, the accuracy over those predictions should be 80%.

Formally, the perfect calibration condition can be expressed as:

$$\text{acc}(B_m) = \text{conf}(B_m)$$

where acc(B_m) is the accuracy of the samples in bin m, and conf(B_m) is the average confidence for those samples in that bin.

To estimate the accuracy from finite samples, predictions are grouped into M bins, each with an interval size of $\frac{1}{M}$. The accuracy of bin B_m is defined as:

$$\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}(\hat{y}_i = y_i) \tag{3.11}$$

where $\hat{y}_i$ and y_i are the predicted and true labels for sample i, respectively, and $\mathbb{1}$ is the indicator function. The average confidence within bin B_m is:

$$\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i \tag{3.12}$$

where $\hat{p}_i$ is the predicted confidence for sample i. Thus, for a perfectly calibrated model, $\text{acc}(B_m) = \text{conf}(B_m)$ for all bins $m \in \{1, 2, \ldots, M\}$.

Figure 3.1 illustrates the reliability plot. The top portion of the figure displays the reliability diagram, while the bottom part presents the confidence histogram.

In the reliability diagram, predictions are divided into bins based on the confidence score of the winning class. For each bin, we compute both the average confidence and the accuracy. The confidence histogram at the bottom shows the distribution of the test samples across the bins, with the plot utilizing 10 bins. The diagram also demonstrates how each bin's average confidence and accuracy are related. Ideally, the average confidence for each bin lies along the diagonal. For instance, if there are 20 bins, each bin is 0.05 apart, the average confidence for the bin (0.9, 0.95] would be approximately 0.925.

For each bin, the difference between the accuracy and the confidence is represented by the red bars. If the accuracy and confidence match in all the bins, the model is calibrated, and the confidence score can be interpreted as true probability. If a gap exists between accuracy and confidence, the model is not properly calibrated. Larger gaps signify worse calibration.

Model is over-confident when $\text{conf}(B_m) > \text{acc}(B_m)$ resulting in more false positives. Model is under-confident when $\text{acc}(B_m) > \text{conf}(B_m)$ leading to more false negatives. The ideal scenario is for the confidence to align with the accuracy, meaning a prediction of confidence 0.8 corresponds to 80% accuracy.

The ECE, depicted in the reliability diagram, summarizes the discrepancy between confidence and accuracy across all bins. It measures the weighted difference between confidence and accuracy, with each bin's weight proportional to the number of examples it contains. A lower ECE indicates better calibration, where the confidence scores more accurately reflect the true probabilities of the model's predictions.

Through calibration, the goal is to reduce the gap between confidence and accuracy, enabling the model to produce more reliable probability esti-

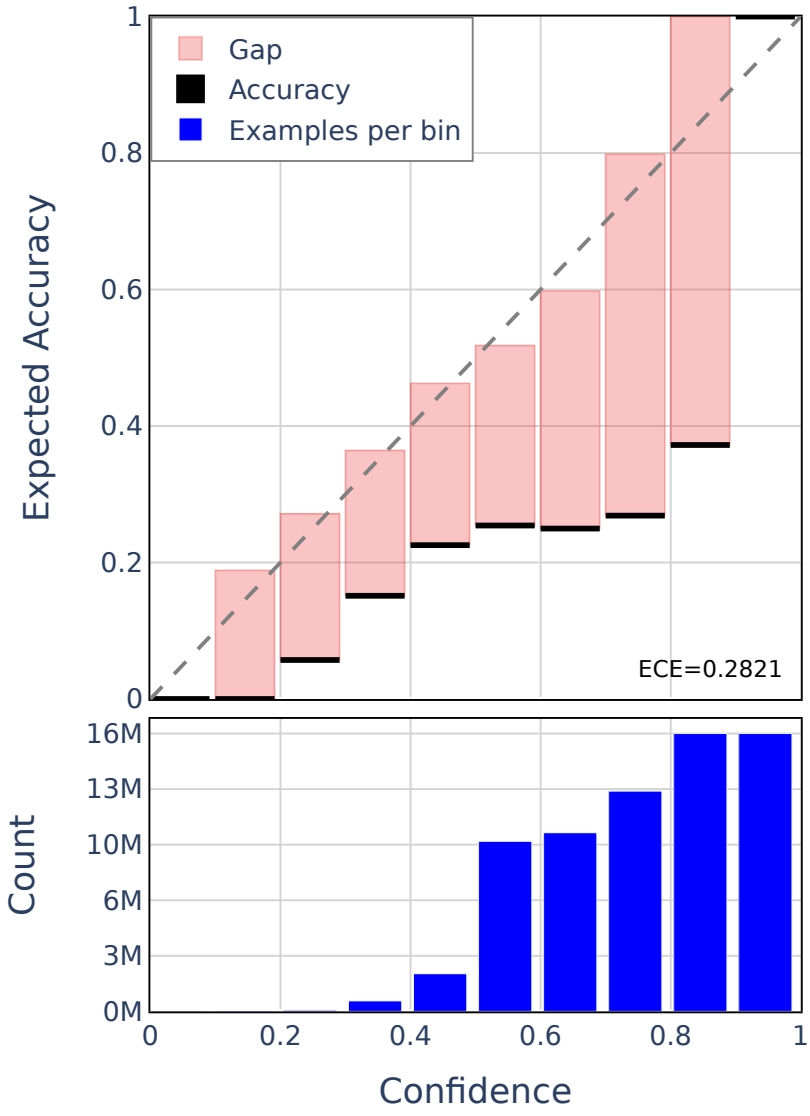


Figure 3.1: Reliability Diagram

mates. While calibration does not alter the models accuracy, it improves the interpretation of the models confidence scores, ensuring they are better aligned with the actual probabilities of correct predictions.

4
Methodology

This chapter presents the datasets used for training and evaluations in section 4.1. The section 4.2 focuses on the baseline architecture used for training the model in our thesis. We also provide a detailed description of its encoder and the decoder. In section 4.3, we outline the preprocessing steps. In Section section 4.4, we present the proposed architectures, including comprehensive information about each component within it. Lastly, we discuss the implementation details, such as model training and optimization techniques in section 4.5.

4.1 Datasets

This thesis employed three primary datasets for model training and evaluation. We used the Cityscapes dataset for model training, as described in subsection 4.1.1. For Out of Distribution (OOD) evaluation, we utilized two popular datasets, Fishyscapes and LostAndFound, that were developed based on the Cityscapes dataset and are detailed in subsection 4.1.2 and subsection 4.1.3, respectively.

4.1.1 Cityscapes

The Cityscapes Dataset for Semantic Urban Scene Understanding was introduced by (CORDTS et al., 2016). The dataset is a large-scale benchmark extensively used in CV research, particularly for tasks such as semantic segmentation, instance segmentation, and panoptic segmentation. Designed to facilitate advancements in autonomous driving and urban scene understanding, it provides high-resolution images captured across 50 cities in Germany and neighboring European countries. These images

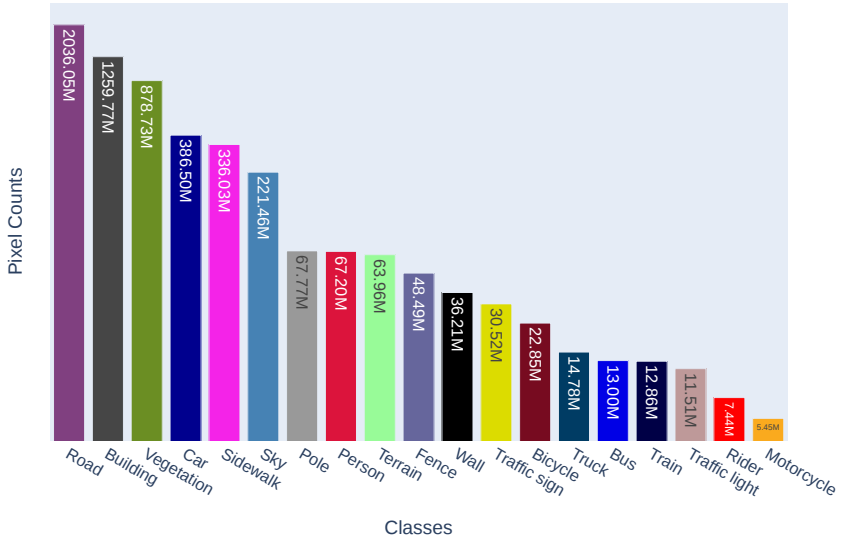


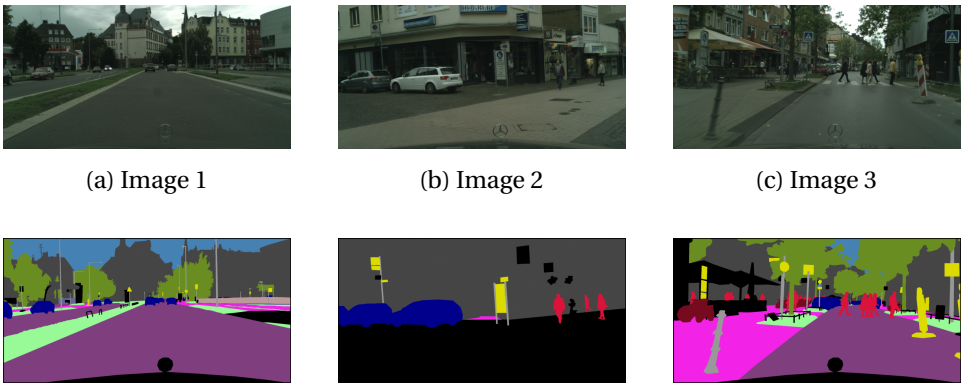
Figure 4.1: Distribution of Cityscapes classes

reflect diverse urban environments under varying weather conditions, seasons, and times of day. The dataset comprises 5,000 finely annotated images and an additional 20,000 coarsely annotated images, enabling robust model training and evaluation. In our thesis, we use fine annotations, which are further split into 2,975 training, 500 validation, and 1,525 test images, each with a resolution of 2048x1024 pixels. However, the test set was withheld for benchmarking purposes. Therefore, we report the evaluation results on the validation set in this thesis. The dataset consists of 30 semantic classes organized into eight categories, including flat surfaces, buildings, vegetation, vehicles, and pedestrians. Out of these, 19 classes (Road, Sidewalk, Building, Wall, Fence, Pole, Traffic light, Traffic sign, Vegetation, Terrain, Sky, Person, Rider, Car, Truck, Bus, Train, Motorcycle, Bicycle) are relevant to urban scenes and are utilized, excluding the void label, in accordance with the evaluation protocol outlined in (CORDTS et al., 2016). Cityscapes pose unique challenges due to their imbalanced class distribution, where common objects like roads and buildings dominate while smaller objects like people and riders appear less frequently as illustrated in Figure 4.1. This imbalance and diverse environmental

conditions require models to balance precision across frequent and rare classes. The Table 4.1 shows the relative distribution of pixels for each class in the dataset. The percentages in the table represent the proportion of each class relative to the total number of pixels, excluding the void class. Figure 4.2 displays three sample RGB input images with three channels (Red, Green, and Blue) alongside their corresponding masks.

Category	**Class**	**Pixel Percentage (%)**	**Description**
Flat Surfaces	Road	36.88	Large areas of streets and highways.
	Sidewalk	6.09	Pedestrian walkways alongside roads.
Construction	Building	22.82	Structures like houses, shops, and offices.
	Wall	0.66	Walls bordering streets or buildings.
	Fence	0.88	Fence dividing the road from other constructions.
Objects	Pole	1.23	Street poles and sign holders.
	Traffic Light	0.21	Traffic control signals.
	Traffic Sign	0.55	Road signs and directions.
Nature	Vegetation	15.92	Trees, bushes, and other greenery.
	Terrain	1.16	Unpaved natural ground or soil.
Sky	Sky	4.01	Sky area visible in the scene.
Human	Person	1.22	People walking or standing in the scene.
	Rider	0.13	People riding bicycles, motorcycles, etc.
Vehicles	Car	7.00	Passenger vehicles, taxis, etc.
	Truck	0.27	Larger cargo vehicles.
	Bus	0.24	Public transportation buses.
	Train	0.23	Urban trains visible on tracks.
	Motorcycle	0.10	Two-wheeled motorized vehicles.
	Bicycle	0.41	Non-motorized bicycles.

Table 4.1: Description of Cityscapes Dataset categories



(a) Image 1 (b) Image 2 (c) Image 3

(d) Mask 1 (e) Mask 2 (f) Mask 3

Figure 4.2: Sample RGB images and their corresponding segmentation masks from the cityscapes dataset.

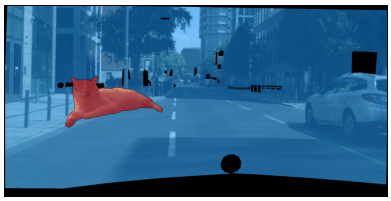
4.1.2 Fishyscapes

Fishyscapes (BLUM et al., 2019) is a benchmark dataset designed for high-resolution anomaly detection and OOD evaluation, particularly in urban driving scenarios. Fishyscapes is derived from the Cityscapes validation dataset. It comprises 30 images augmented with synthetic anomalous objects extracted from the Pascal VOC dataset (EVERINGHAM et al., 2007). These objects are carefully selected from classes absent in the Cityscapes dataset (e.g., airplanes, birds, and mammals) and are overlaid onto the images. These objects are randomly resized and positioned within the scenes to maintain realistic spatial relationships, ensuring a challenging environment for anomaly detection models. To further increase complexity, Fishyscapes employs synthetic fog effects and excludes void pixels from evaluation, creating a challenging setup that prevents trivial solutions such as direct comparisons with the original Cityscapes dataset. By combining synthetic and real-world anomalies, Fishyscapes serves as a robust benchmark for assessing a model's ability to detect OOD regions in complex urban scenarios. The dataset contains approximately 4.5 million Out of Distribution (OOD) pixels and 1.8 billion In-distribution (ID) pixels, providing a substantial testbed for assessing model performance.

(a) Image 1

(b) Image 2

(c) Mask 1



(d) Mask 2

Figure 4.3: Sample RGB images and their associated segmentation masks from the Fishyscapes dataset. In the mask, blue indicates In-distribution (ID) pixels the model was trained on, red signifies Out of Distribution (OOD) pixels, and black represents the background areas.

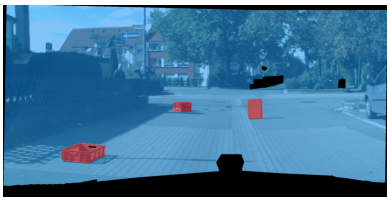
In Figure 4.3, we present sample Fishyscapes input images and respective ground truth segmentation masks. The first image features a roadway with a synthetic dog walking, evaluating the model's proficiency in detecting small, fast-moving anomalies in urban settings. The second image shows a city road with a cat in the traffic, testing the model's capacity to identify unexpected objects.

4.1.3 LostAndFound

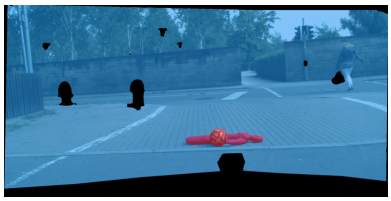
The LostAndFound dataset (PINGGERA et al., 2016) complements Fishyscapes, tailored explicitly for real-world anomaly detection within urban driving environments. Unlike Fishyscapes, which focuses on broader object-level anomaly detection, the LostAndFound dataset concentrates on anomalies within typical urban landscapes, such as residential streets, industrial zones, and parking lots. It consists of 375 high-resolution images divided into 100 validation images and 275 withheld test images, providing a robust foundation for evaluating model performance across varied real-world conditions. This thesis uses the 100 validation images of the LostAndFound dataset for Out of Distribution (OOD) evaluations.

(a) Image 1

(b) Image 2



(c) Mask 1



(d) Mask 2

Figure 4.4: Sample RGB images and their associated segmentation masks from the LostAndFound dataset. In the mask, blue indicates In-distribution (ID) pixels the model was trained on, red signifies Out of Distribution (OOD) pixels, and black represents the background areas.

The dataset includes real-world anomaly scenarios, such as misplaced objects or road hazards, while explicitly excluding common Cityscapes objects like bicycles or pedestrians to ensure the focus remains on OOD detection. The Figure 4.4 shows the sample images and their corresponding segmentation masks. This targeted design makes the dataset particularly valuable for testing models that need to generalize well to anomalies in real-world driving contexts, offering a more rigorous benchmark for the development of practical anomaly detection systems.

Together, Fishyscapes and LostAndFound provide complementary challenges for Out of Distribution (OOD) evaluation in autonomous driving applications. Fishyscapes tests the models adaptability to synthetic and diverse anomalies, while LostAndFound emphasizes generalization to real-world anomalies. By combining synthetic and real-world benchmarks, they form a comprehensive framework for assessing Out of Distribution (OOD) detection performance in safety-critical environments.

4.2 Baseline Model

Based on the existing literature, transformer-based encoder-decoder models are widely used for complex Computer Vision (CV) tasks due to their effective attention mechanisms. This thesis adopts the state-of-the-art (SOTA) transformer-based model, specifically SegFormer's MiT-B3 encoder, (XIE et al., 2021) as our baseline encoder. For the decoder, we utilize Feature Pyramid Network (FPN) by (LIN et al., 2017) to conduct our experiments. The baseline architecture used in this thesis is illustrated in Figure 4.5, and its key blocks are detailed in subsection 4.2.1 and subsection 4.2.2, respectively. The baseline model has a total of 5.078 million trainable parameters. MiT-B3 is built upon the transformer architecture introduced by (VASWANI et al., 2023), which leverages attention mechanisms effectively.

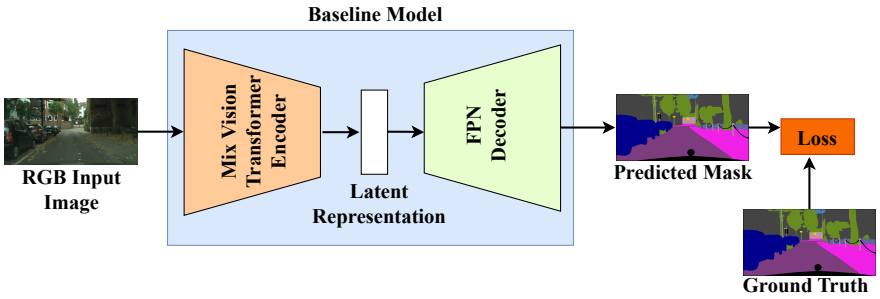


Figure 4.5: Baseline Architecture

4.2.1 Encoder: MiT-B3 Encoder

SegFormer, introduced by XIE et al. (2021), is a cutting-edge Transformer-based framework for semantic segmentation. Central to SegFormer's architecture is its innovative encoder, known as the Mix Transformer encoder (MiT). While inspired by the original Vision Transformer (ViT), the MiT has been meticulously re-engineered and optimized to address the unique challenges of semantic segmentation, setting it apart as a specialized and highly efficient encoder. The MiT encoder comes in a series of scalable variants, ranging from MiT-B0 to MiT-B5, each designed to cater

to varying levels of computational resources and performance requirements:

- MiT-B0: A lightweight model optimized for fast inference, making it ideal for real-time applications and edge devices.
- MiT-B5: The most robust and deeper variant, offering maximum performance for tasks demanding detailed and accurate segmentation.

For our thesis, considering the balance between computational efficiency and segmentation accuracy, we have selected MiT-B3 as the encoder. This middle-tier model strikes an optimal balance between complexity and performance.

MiT-B3 is a hierarchical Transformer encoder. Unlike ViT, which generates single low-resolution feature maps of fixed resolution, the hierarchical design in MiT-B3 allows the encoder to produce multi-resolution feature maps, enabling the encoder to capture both high-resolution fine features and low-resolution global coarse features. Such a design is critical for accurately segmenting objects of varying scales within an image.

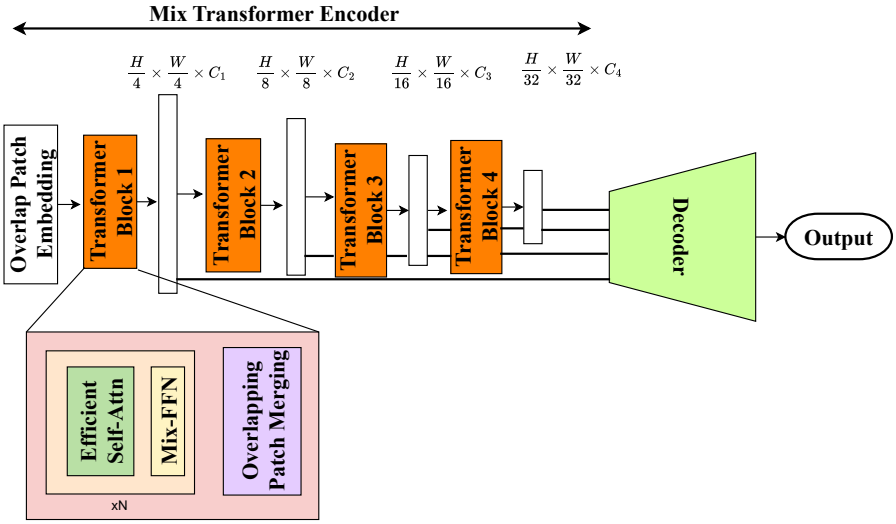


Figure 4.6: Architecture of Mix Vision Transformer

In this work, we use MiT and MiT-B3 interchangeably to refer to the encoder architecture employed in this work. The detailed architecture of

MiT, including its hierarchical structure and key components, is illustrated in Figure 4.6. Given an image of size $H \times W \times 3$, the MiT-B3 encoder begins by partitioning it into overlapping patches of size 4×4, unlike Vision Transformer (ViT), which typically use larger 16×16 non-overlapping patches. The smaller patch size in MiT is particularly advantageous for dense prediction tasks such as semantic segmentation, as it helps preserve fine-grained spatial details. The encoder processes these patches to generate multilevel feature maps at resolutions $\{\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}\}$ of the original image dimensions, capturing features at progressively coarser scales.

- **Overlap Patch Merging**:

  The overlapping patch embedding stage in the MiT encoder is the first component designed to convert raw input into overlapping patches. This stage begins with partitioning an input image of dimensions $H \times W \times C$, where H and W represent the height and width of the image, and C denotes the number of channels (typically 3 for RGB images) into overlapping patches. The Overlap Patch Merging block can be implemented with a convolution layer with a stride less than the kernel size, so it overlaps different patches. For example, consider a kernel size (K), stride (S), and padding (P). To obtain overlapping patches, using $K = 7, S = 4$, and $P = 3$ results in each patch covering a 7×7 area, with 3 pixels overlapping between neighboring patches. Each patch is then embedded into a feature vector through the Convolution layer. By using overlapping patches, this method preserves local continuity between patches, captures richer contextual information, and reduces information loss at patch boundaries, setting the foundation for the hierarchical representation learning in the subsequent Transformer blocks.

- **Hierarchical Structure with Multiscale Features**: The MiT encoder then processes the overlapping patch embeddings through a sequence of hierarchical Transformer blocks, each designed to generate feature maps at progressively coarser resolutions. Each Transformer block incorporates an optimized self-attention mechanism, which efficiently models long-range dependencies. The output of each block is a feature map with a different resolution, analogous to the multiscale feature representations in convolutional neural

networks (CNNs). Specifically, the feature maps are represented as $F_i = \frac{H}{2^{i+1}} \times \frac{W}{2^{i+1}} \times C_i$ where $i \in \{1,2,3,4\}$, C_i is the number of output channels at stage i, and $C_{i+1} > C_i$. For instance, For example, at stage 1, the feature map is $F_1 : \frac{H}{4} \times \frac{W}{4} \times C_1$, for stage 2 is $F_2 : \frac{H}{8} \times \frac{W}{8} \times C_2$, for stage 3 is $F_3 : \frac{H}{16} \times \frac{W}{16} \times C_3$, and so forth, capturing information at increasingly larger receptive fields.

To achieve this hierarchical feature representation, the overlapping patch merging process is applied between successive Transformer blocks. This merging operation reduces the spatial resolution of feature maps while increasing their depth, enabling the encoder to gradually shift its focus from fine-grained local details to broader global context. This design equips the MiT encoder with the ability to capture multiscale information essential for accurately segmenting objects of varying sizes. By seamlessly transitioning between resolutions, the encoder effectively balances the preservation of fine details with the aggregation of global contextual information, making it highly suited for dense prediction tasks like semantic segmentation.

- **Positional Encoding Free Mix-FFN**:

  After the embedding passes through the self-attention layers, it enters the Mix-FFN layer, a critical component of the MiT architecture. Traditional transformers like ViT rely on positional encoding (PE) to incorporate the location information of tokens. Positional encoding typically maps the input sequence to a fixed positional range, marking the start and end points of the input. However, this approach introduces challenges during inference when test images have resolutions different from those in the training dataset. Interpolating positional encodings to accommodate varying resolutions often results in reduced model accuracy (XIE et al., 2021).

  To address this limitation, the authors of MiT replaced positional encoding with the Mix-FFN layer, which integrates positional information implicitly. Mix-FFN leverages a combination of a 3Œ3 convolutional layer and an MLP(Multi-Layer Perceptron) into each Feed-Forward Network (FFN) to embed positional information directly into the feature representation. The 3×3 convolution in a feed-forward network is particularly effective because it considers the effects of zero-padding, which inherently leaks spatial information, as

demonstrated in (ISLAM et al., 2020). This approach eliminates the need for explicit positional encoding, avoiding interpolation-related inaccuracies. The operation of Mix-FFN can be mathematically expressed as:

$$x_{out} = MLP(GELU(Conv_{3\times3}(MLP(x_{in})))) + x_{in} \tag{4.1}$$

where x_{in} is the input feature from the self-attention module, and x_{out} is the output feature. Furthermore, depth-wise convolutions are employed within the Mix-FFN to reduce the number of parameters and enhance computational efficiency. This design ensures that the Transformer retains strong positional awareness while remaining adaptable to inputs of varying resolutions, making it highly effective for tasks like semantic segmentation.

This combination of overlapping patches, hierarchical feature representation, and positional encoding-free design ensures that the MiT-B3 encoder captures rich multiscale features essential for high-quality semantic segmentation. These multi-level features from the encoder are then passed to the FPN decoder to predict the segmentation mask at a $\frac{H}{4} \times \frac{W}{4} \times N_{cls}$ resolution, which is then interpolated in the post-processing stage. Here N_{cls} is the number of categories.

4.2.2 Decoder: Feature Pyramid Networks

Detecting objects at different scales is challenging, particularly for small objects. Earlier approaches used a pyramid of the same image at various scales to detect objects. However, processing multiple-scale images is time-consuming, and the memory demand is too high to be trained end-to-end simultaneously. Some studies only used the multi-scale pyramid approach for inference to push accuracy as high as possible when speed is not a concern.

The Feature Pyramid Network (FPN) is a deep learning architecture designed to tackle multi-scale challenges in tasks such as object detection, semantic segmentation, and instance segmentation. It achieves this by leveraging multi-scale feature maps generated by a backbone encoder (e.g., CNN or Transformer-based models). FPN combines features from

different stages of the encoder to produce outputs that are both high-resolution and semantically rich, enabling accurate pixel-level predictions. The core concept behind FPN is to construct a top-down feature pyramid with the goal of recognizing objects of varying sizes. This pyramid combines low-resolution, high-semantic information from deeper encoder layers with high-resolution, low-semantic information from shallower layers. The result is a richer feature representation that integrates both global context and fine details. The workflow of FPN is illustrated in Figure 4.7

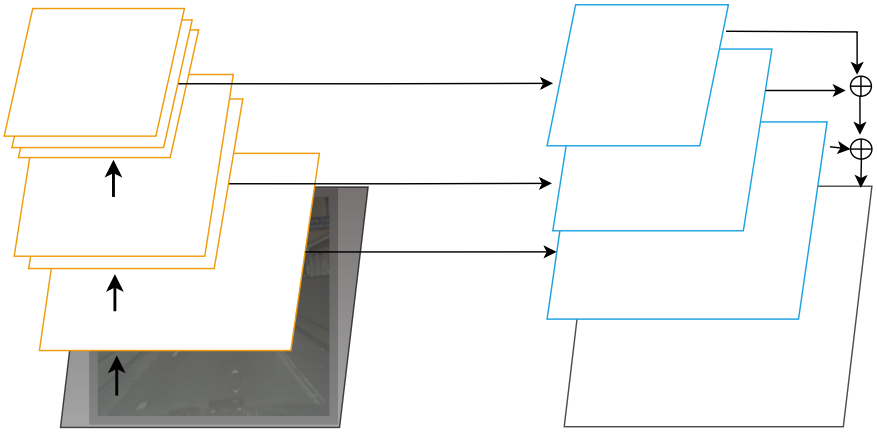


Figure 4.7: Workflow of Feature Pyramid Network Decoder

The FPN architecture typically consists of the key components:

- **Multiscale Feature Fusion**: The backbone encoder generates feature maps at multiple scales, corresponding to different resolutions (e.g., (e.g., $\{\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, and \frac{1}{32}\}$ of the input size). These maps contain detailed spatial information at higher resolutions and semantic richness at lower resolutions. Shallow layers capture fine-grained details (edges, textures), while the deep layers capture abstract, high-level information (object context). The FPN decoder combines these multiscaled feature maps of the encoder to generate precise outputs, such as segmentation masks or bounding boxes.

- **Top-Down Pathway and Upsampling**: The top-down pathway of the FPN decoder begins by taking the encoder's deepest feature map (smallest resolution from deeper layers) as input and progressively

upsampling it to match the size of shallower feature maps. This pathway propagates global semantic context upward from deeper layers.

- **Lateral Connections**: To preserve spatial details, lateral connections merge the upsampled feature maps of the FPN decoder with corresponding encoder features from the same scale. A 1×1 convolution aligns the channel dimensions of the encoder features with those of the upsampled decoder features before merging.
- **Merging**: Upsampled features are combined with processed encoder features using operations like addition or concatenation. This step is performed iteratively across all scales to ensure that the resulting feature maps are semantically meaningful and spatially precise.
- **Final Prediction**: After the multiscale fusion, a 1×1 convolution layer generates the final high-resolution output, such as a segmentation mask or a dense prediction map.

The baseline model is minimized using a categorical cross-entropy loss function as defined below:

$$L_{\text{CE(y,p)}} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} y_{n,c} \cdot \log(p_{n,c}) \tag{4.2}$$

Here, N is the number of pixels in the image, C is the number of classes, $y_{n,c}$ is the respective ground truth label, $p_{n,c}$ is the respective predicted softmax probability for each class for a pixel.

4.3 Regularization Techniques

A deep learning models ability to generalize refers to how well it performs on unseen data. Data augmentation and transfer learning techniques have been used to prevent overfitting and help the model generalize better on unseen data.

4.3.1 Data Augmentation Preprocessing

Data augmentation techniques are instrumental in expanding the available dataset. They achieve this by transforming existing data while pre-

serving their labels or generating synthetic data from the available data instances. In our thesis, we used the Cityscapes dataset subsection 4.1.1 to train all our models. To preprocess and augment the images effectively, we implemented a variety of transformations tailored for the training, validation, and test splits. The Python library Albumentations (BUSLAEV et al., 2020) was employed to facilitate these augmentations.

For the training stage, we began with random resizing, which crops images to a size that is randomly determined within a range of 50% to 150% of the designated crop size (1024x1024). This variability introduces robustness to the model. We further enhanced the dataset through random horizontal flipping with a 50% probability, which helps the model become invariant to orientation. Additionally, we applied a color jitter transformation that adjusts brightness, contrast, saturation, and hue, simulating various lighting conditions to enrich the dataset. Post-augmentation, images are normalized to standardize pixel values according to predefined ImageNet dataset (RUSSAKOVSKY et al., 2015) mean and standard deviation values. In the validation phase, we focused on ensuring consistent image sizes by resizing images to the specified crop size (1024x1024) while maintaining the aspect ratio. A center crop is applied to highlight the central part of each image, ensuring uniformity across all validation images. The normalization process mirrors that of training but excludes random augmentations to provide consistent inputs for accurate evaluation. During testing, we concentrate on applying padding and normalization while avoiding any resizing or cropping; this approach allows us to leverage the full-size images for inference.

4.3.2 Transfer Learning

Transfer learning is a powerful technique in deep learning that involves using a model trained on a large dataset and fine-tuning it for a specific task using a smaller dataset. This approach effectively addresses issues related to data scarcity, helps prevent overfitting, and enhances the model's ability to generalize. Transfer learning is particularly effective for tasks like image classification and segmentation, as many image datasets share common low-level features. During the transfer learning process, the knowledge acquired by the pre-trained model is captured in its weights,

which are then leveraged during the fine-tuning stage to address the target problem more effectively. Typically, one can either fine-tune the entire model or retrain only specific layers while keeping the other parameters frozen with the pre-trained weights. All the models used in this thesis have been pre-trained on the ImageNet dataset (RUSSAKOVSKY et al., 2015). The ImageNet dataset, known for its extensive collection of labeled images, is commonly used for pretraining due to its diversity and scale. This allows the model to learn generic and transferable feature representations. By reusing features such as edge and texture detectors learned from ImageNet, transfer learning significantly reduces the computational resources and training time required while improving performance, especially for tasks with limited data.

4.4 Proposed Methodology

As mentioned earlier, the primary objective of this thesis is to utilize existing uncertainty estimation techniques as guidance signals for training the segmentation model, ultimately aiming to enhance segmentation performance, OOD detection and calibration. To achieve this goal, we implemented architectural improvements to the baseline model in three distinct ways.

First, we incorporated the concept of Monte Carlo Dropout (MCD) as a guidance signal into the transformer-based model training process. Second, we applied Online Hard Example Mining (OHEM) as a guidance signal, as detailed in Algorithm 1. This technique enables the model to better account for areas of high uncertainty, ensuring that the training process emphasizes these regions for improved learning. Finally, we employed Bayesian-based Evidential Deep Learning (EDL) to implicitly model uncertainties during training as explained in section 3.3.

4.4.1 MCD based Uncertainty signal training

The first method utilizes Monte Carlo Dropout (MCD) during training to incorporate predictive uncertainties into the segmentation process, thereby enhancing performance by adapting the loss function based on these uncertainties. The study by (LANDGRAF et al., 2023a) was the first

to integrate MCD into the conventional CNN-based training process. In our thesis, we will employ a similar approach using a more advanced transformer-based segmentation model. This method introduces two main enhancements to the traditional training framework. First, MCD is used to sample from the posterior distribution during training, generating predictive uncertainties alongside the regular segmentation outputs. Second, these uncertainties are leveraged to apply pixel-wise weighting to the cross-entropy loss, effectively modifying the loss calculation to give more importance to uncertain regions.

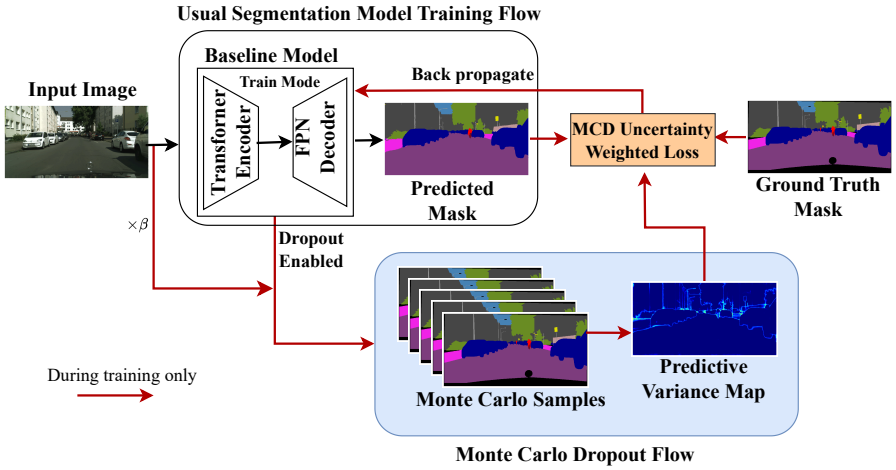


Figure 4.8: Architecture of Monte Carlo Dropout Based Uncertainty Signal Training

As illustrated in Figure 4.8, during each training step, along with segmentation prediction, β segmentation samples are generated by performing multiple forward passes with dropout enabled, with $\beta = 5$ in our experiments. These samples are used to estimate pixel-level uncertainties by computing the standard deviation of the softmax probabilities across the samples. To minimize computational overhead, gradient computation is disabled during the sampling process, as backpropagation relies solely on regular segmentation predictions. This optimization ensures that the additional cost introduced by integrating MCD is manageable in terms of both training time and GPU memory usage.

In classification tasks such as segmentation, the cross-entropy loss is commonly used to evaluate the model's performance. This loss function measures the difference between the predicted probabilities and the true class labels. It is typically employed for multi-class classification problems. The conventional cross-entropy loss is modified with pixel-wise weights to incorporate these uncertainties into the learning process. For each pixel, the weight is calculated as $w_n = (1+\sigma_n)^{\alpha}$, where σ_n represents the predictive uncertainty derived from the segmentation samples and α is a hyperparameter controlling the influence of these uncertainties, we set $\alpha = 10$ empirically. Hence, the final loss is calculated as:

$$L_{\text{MCD}} = -\frac{1}{N}\sum_{n=1}^{N} w_n \sum_{c=1}^{C} y_{n,c} \cdot \log(p_{n,c}) \tag{4.3}$$

Here N is the number of pixels in the image, C is the number of classes, $y_{n,c}$ is the respective ground truth label, $p_{n,c}$ is the respective predicted softmax probability, and w_n represents the pixel-wise uncertainty weight.

By scaling the loss based on the uncertainty, the model prioritizes learning in regions with higher variability, resulting in improved segmentation accuracy and robustness. When w_n is set to one for all pixels, the loss function simplifies to the conventional cross-entropy loss, indicating the flexibility of this approach. This methodology effectively integrates the predictive uncertainties from MCD into both the training process and the loss function. While MCD is utilized here for its practical benefits, the framework can be adapted to use alternative uncertainty quantification methods, providing a versatile strategy for enhancing segmentation tasks.

4.4.2 OHEM based Uncertainty signal training

Online Hard Example Mining (OHEM) is integrated into the training process to improve model performance by dynamically focusing on challenging examples. Unlike conventional training, which often prioritizes easy examples, OHEM emphasizes samples with the highest prediction errors or uncertainty as detailed in 1

In the OHEM framework, the model processes a batch of training images and computes the cross-entropy loss for each pixel. These losses are ranked, and pixels are selected based on two criteria: (1) their loss scores

fall below a predefined threshold ($thresh$), and (2) they are among the top $mining_percent$ pixels with the highest loss. This selection ensures the model learns more effectively from under-represented or difficult-to-classify regions, such as object boundaries or smaller objects within the image.

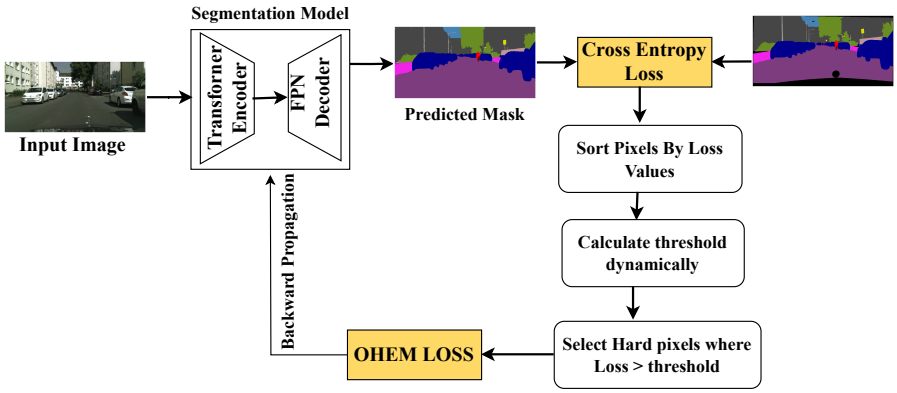


Figure 4.9: Architecture of OHEM Based Uncertainty Signal Training

$$L_{\text{OHEM}} = -\frac{1}{|S|}\sum_{i\in S}\left(\sum_{c=1}^{C} y_{i,c}\cdot\log(p_{i,c})\right) \tag{4.4}$$

Here, S is the set of selected hard pixels identified based on the loss ranking and threshold, $|S|$ is the total number of selected hard pixels, $y_{i,c}$ is the ground truth label for pixel i and class c. $p_{i,c}$ is the predicted probability for pixel, and C is the total number of classes.

Once the hard pixels are identified, the batch loss in Equation 4.4 is calculated using only these difficult pixels, enabling the model to focus its learning on more challenging areas. By dynamically adjusting the selection of hard examples during each training step, OHEM mitigates the dominance of easy pixels, such as large background regions, and improves segmentation accuracy for difficult classes. The process is repeated iteratively across batches, with backpropagation updating the model parameters using the hard example-driven loss as illustrated in Figure 4.9. This methodology ensures the model learns from hard-to-classify pixels dynamically, improving its ability to segment complex scenes with significant class imbalance or challenging object boundaries.

4.4.3 EDL based Uncertainty signal training

The training pipeline for Evidential Deep Learning (EDL) focuses on enhancing the model's ability to quantify and manage uncertainty in its predictions. In contrast to traditional deep learning models that produce deterministic outputs, EDL incorporates explicit uncertainty estimation by modeling predictions as belief distributions. As detailed in section 3.3, the network achieves this by replacing the typical softmax output layer with a Dirichlet distribution, representing the model's belief about the class membership. This Dirichlet distribution outputs parameters that reflect the evidence gathered for each class, allowing the model to capture not only its confidence in the predicted class but also its uncertainty in the decision.

During training, the network learns to minimize two objectives as detailed by the loss function in Equation 3.7. The first component is the classification loss that calculates the difference between the predicted evidence parameters and the actual ground truth as in Equation 3.8, which ensures that the model generates predictions close to the true class labels. The second component is a regularization term, specifically the Kullback-Leibler (KL) divergence, which penalizes overconfident predictions. The KL divergence measures the difference between the predicted Dirichlet distribution and a uniform Dirichlet distribution, with the regularization ensuring that the model penalizes overconfident predictions and making sure to remain cautious when uncertainty is high. In other words, this term encourages the model to remain cautious when the evidence for a given class is weak or ambiguous.

The training process involves updating the network's parameters through backpropagation. Gradients are computed by optimizing the combined objective function of the sum of squared Bayesian classification loss and the KL divergence. To gradually shift the focus towards uncertainty as training progresses, the influence of the KL divergence term is annealed. This means that during the initial stages of training, the model is more focused on reducing classification errors, and as training advances, the model increasingly prioritizes handling uncertainty. Therefore, the loss function in Equation 3.7 includes a regularization strength that is adjusted using an annealing coefficient λ.

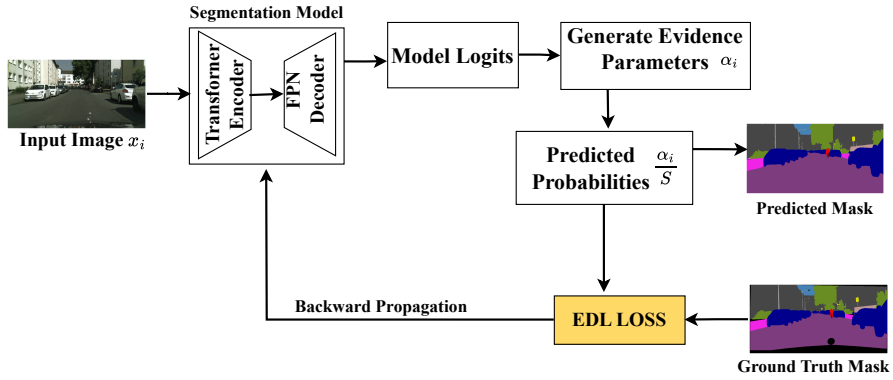


Figure 4.10: Architecture of EDL Based Uncertainty Signal Training

Throughout this pipeline, the model is trained iteratively, with each update refining both its predictions and its ability to manage uncertainty. By explicitly incorporating uncertainty in its predictions, EDL enables the model to make more robust decisions, mainly when dealing with noisy data or OOD inputs. This dual-objective approach of minimizing prediction error while controlling for overconfidence is what distinguishes EDL from traditional methods and makes it a powerful tool in deep learning tasks. The training pipeline ensures that the model learns not just to classify the input but also to express confidence in its predictions, offering a more robust approach to handling uncertainty in deep learning applications.

4.5 Model Training

4.5.1 Training Process and Optimization

For all training processes, we employed the **AdamW** optimizer (LOSHCHILOV und HUTTER, 2019), an advanced variant of the Adam optimizer that integrates decoupled weight decay. Adam combines the principles of gradient descent and momentum to efficiently minimize the loss function. It calculates gradients representing the steepest increase in the loss function and iteratively updates the model weights in the opposite direction to reduce the loss. Momentum further accelerates convergence by smoothing

out gradient updates, leveraging accumulated past gradients to maintain consistent progress.

A key feature of AdamW is applying a "decoupled weight decay" factor, which serves as a regularization mechanism. Instead of adding a penalty proportional to the magnitude of the weights directly to the loss function, weight decay in AdamW modifies the parameter update equation itself. This approach encourages smaller weights, which improves the model's generalization capability. In our implementation, the AdamW optimizer was configured with standard default settings: momentum terms of 0.9 and 0.999, a weight decay factor of 0.0001, and a base learning rate of 0.0001.

To accelerate training, the learning rate for the decoder and segmentation head was multiplied by a factor of 10. Additionally, **ReduceLROnPlateau** scheduling was applied to dynamically adjust the learning rate during training. This technique monitors a specific performance metric, such as validation loss, and reduces the learning rate by a predefined factor when the metric plateaus (i.e., shows no improvement) for a set number of epochs. The learning rate was adjusted using the formula:

$$lr_{\text{new}} = lr_{\text{current}} \times \text{factor}$$

In this work, we set the reduction factor to 0.5, patience to 5 epochs, and a minimum learning rate of 1×10^{-6}, below which the learning rate would not decrease further.

4.5.2 Computational Resources and Configuration

All the training processes used a crop size of 1024×1024 pixels and a batch size of 4. Small batch sizes are selected due to memory limitations and the desire for faster convergence. The significant memory usage during training is likely caused by the large model architecture and the high-resolution input data. Moreover, smaller batch sizes introduce some noise, which acts as a regularization factor and helps the model escape local minima. The images were preprocessed as described in subsection 4.3.1. All the models were trained for 50 epochs on four NVIDIA Tesla V100 GPUs, each with 32GB of memory. The training was distributed across these 4 GPUs

using PyTorch's Distributed Data-Parallel (DDP) implementation, which minimizes inter-GPU communication overhead. Each training methodology utilized the respective loss functions as detailed in section 4.4, and the performance evaluation on the test set determined the best-fit model. The Mean Intersection Over Union (mIoU) metric was used for validation, and the model with the best validation mIoU was saved for further evaluation. We processed the cityscapes data primarily using the cityscapes scripts [1], Google Colab [2] and Hugging Face datasets library[3]. All the models used in this thesis have been pre-trained on the ImageNet dataset (RUSSAKOVSKY et al., 2015). To monitor the training, we utilized the TensorBoard platform [4].

4.5.3 Mixed-Precision Training with Gradient Scaling

To enhance computational efficiency and reduce memory consumption during training, **mixed-precision training** (MICIKEVICIUS et al., 2018) was integrated into the pipeline. This approach leverages both 16-bit and 32-bit floating-point precision to accelerate computations while maintaining numerical accuracy. To manage potential numerical instabilities associated with lower-precision formats, PyTorch's GradScaler mechanism was employed in this thesis. GradScaler applies **gradient scaling**, a technique that prevents gradients from becoming too small (underflowing) in 16-bit precision. The gradients are multiplied by a scaling factor during each training iteration, effectively shifting their range to higher magnitudes. This adjustment ensures that even small gradients are adequately represented. After the backward pass, the gradients are unscaled before being applied to update the model parameters. An added advantage of GradScaler is its dynamic adjustment mechanism, which adapts the scaling factor during training. If an overflow occurs, the scaling factor is reduced; if no overflow occurs for a set number of iterations, the factor increases. This dynamic adjustment enhances the robustness and stability of mixed-precision training, allowing the model to benefit from faster computations and reduced resource usage without compromising accuracy.

[1] https://github.com/mcordts/cityscapesScripts
[2] https://colab.google/
[3] https://github.com/huggingface/datasets
[4] https://www.tensorflow.org/tensorboard

5 Evaluation and Results

In this chapter, we perform a series of experiments using the datasets and methods mentioned in chapter 4. We present the results of the experiments and evaluate these results using the appropriate metrics to answer the research questions introduced in chapter 1. Firstly, section 5.1 discusses the evaluation metrics reported in our thesis. In the subsequent section, we provide the details of the experiments that were carried out and present our findings in section 5.2. We finally conclude with a discussion section in chapter 6, which analyzes the findings.

5.1 Evaluation Metrics

For our quantitative evaluations, we primarily report the Mean Intersection Over Union (mIoU), also known as the Jaccard Index, to measure segmentation performance. In addition to the mIoU, we also provide accuracy metrics. Beyond these segmentation metrics, we use the Expected Calibration Error (ECE) and reliability diagrams to assess confidence calibration. Alongside the quantitative evaluation, we present qualitative results to gain a better understanding of the segmentation and OOD performance of our models. Detailed information about these metrics is provided in the following sections.

5.1.1 Segmentation Metrics

5.1.1.1 Mean Per Category Accuracy

Accuracy is a fundamental evaluation metric to evaluate the performance of classification models, particularly in tasks like semantic segmentation.

It measures the proportion of correctly classified pixels from the total number of pixels in an image. Specifically, the accuracy for each class c is computed as:

$$\textbf{Accuracy}_c = \frac{TP_c + TN_c}{TP_c + TN_c + FP_c + FN_c} = \sum_{i=1}^{N_c} 1(y_i = \hat{y}_i) \tag{5.1}$$

Here c represents the class, y is the true target value, and $\hat{y}$ is the predicted value and N_c represents total number of pixels in class c.

Mean Accuracy is a metric that provides a summary of the model's overall performance by averaging the individual per-category accuracies. It is measured by taking the arithmetic mean of the individual per-category accuracy scores. This method ensures that each class's performance is considered independently and equally.

$$\textbf{Mean Accuracy} = \frac{1}{C} \sum_{c=1}^{C} \text{Accuracy}_c \tag{5.2}$$

Where C is the number of classes c in the task. The mean accuracy offers a single scalar value that summarizes how well the model performs across all classes. In contrast to the per-category accuracy Equation 5.1, which can highlight the performance on individual classes, the mean accuracy is a more global measure of performance across all classes, treating each class equally. It is important to note that in cases of class imbalance, the mean accuracy might not fully reflect the performance on underrepresented classes, as each class contributes equally regardless of its prevalence in the dataset.

5.1.1.2 Overall Accuracy

This metric calculates overall accuracy while incorporating a weighted average based on class support. Support refers to the total number of true instances for each class, defined as the sum of true positives (TP) and false negatives (FN). By considering support, the metric accounts for class distribution, ensuring that classes with more examples contribute proportionally to the final result. This approach prevents skewed evaluations when dealing with imbalanced datasets, where certain classes appear

more frequently than others. This metric provides a more representative assessment of model performance across all classes.

In semantic segmentation, the overall weighted accuracy is calculated as:

$$\textbf{Overall Accuracy} = \sum_{c=1}^{C} \text{Accurcay}_c \times \text{support}_c \tag{5.3}$$

This provides a single scalar value that gives a global measure of accuracy but balances the contributions of each class according to how many instances belong to that class. This is useful in cases where certain classes are underrepresented, ensuring they are not overshadowed by dominant classes in the overall accuracy.

While these metrics offer a general understanding of a model's performance, they have limitations in capturing finer segmentation details, particularly when dealing with imbalanced classes or small, difficult-to-detect objects. For instance, if most of the image consists of background pixels, a model that labels the majority of pixels as background could still achieve a high overall accuracy. However, this would indicate a failure to detect smaller or more critical objects in the foreground. Therefore, additional metrics such as Mean Intersection Over Union (mIoU) are often used in conjunction with accuracy to offer a more comprehensive evaluation of segmentation performance.

5.1.1.3 Mean Intersection Over Union

The Intersection Over Union (IoU) (MÜLLER et al., 2022), also known as the Jaccard index, is one of the most common and widely used metric to evaluate semantic segmentation performance. It is a very straightforward metric that has proven to be extremely effective and is used by all the benchmarks for semantic segmentation. IoU quantifies the similarity between predicted and ground truth segmentation by calculating the ratio of their intersection to their union. Formally, IoU is expressed as Equation 5.5. The metric ranges from 0 to 1 (or 0-100%), with 0 indicating no overlap and 1 as a perfect overlap. Figure 5.1 visualizes the IoU calculation.

$$\textbf{IOU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{\text{TP}}{\text{TP+FP+FN}} \tag{5.4}$$

Here True Positives (TP) are the pixels correctly predicted as belonging to the positive class, False Positives (FP) are the pixels incorrectly predicted as belonging to the positive class (should belong to another class or background), and False Negatives (FN) are the pixels belonging to the positive class but incorrectly predicted as belonging to another class or background.

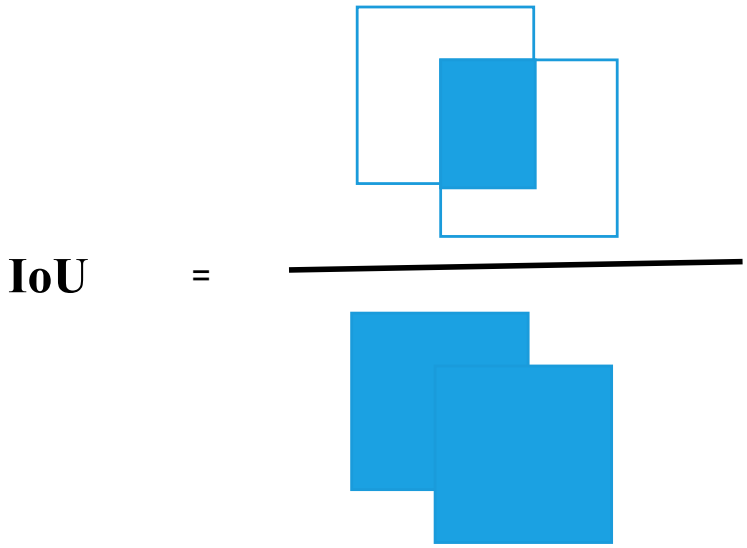


Figure 5.1: IoU Calculation

In multi-class classification tasks, Mean Intersection Over Union (mIoU) extends IoU to multiple classes by calculating the IoU for each class separately and then averaging these values. This ensures that the metric accounts for all classes equally, regardless of class size or imbalance. mIoU is expressed formally as:

$$\textbf{mIOU} = \frac{1}{C}\sum_{c=1}^{C}\frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c} \tag{5.5}$$

where C is the total number of classes, and TP_c, FP_c, FN_c are the True Positives, False Positives, and False Negatives for class c.

mIoU provides an averaged performance measure across all classes, ensuring that every class is evaluated and this makes it a crucial metric in multi-class segmentation tasks.

5.1.2 Calibration Metrics

In this thesis, we evaluate the calibration performance of the model using two widely recognized and extensively utilized metrics: the Expected Calibration Error (ECE) and Reliability Diagrams, as detailed in section 3.4 and section 3.5. These metrics are fundamental tools for assessing the alignment between a model's predicted confidence and its actual accuracy. ECE provides a concise numerical summary of calibration performance, while reliability diagrams offer a visual representation of the model's calibration behavior. Together, these approaches provide a comprehensive evaluation of how well the model's probabilistic predictions reflect the true likelihood of outcomes, making them indispensable in the study of model reliability.

5.1.3 OOD Detection Metrics

Area under the Receiver Operating Characteristic Curve (AUROC) and False Positive Rate at 95 % True Positive Rate (FPR95) are two widely used metrics in Out of Distribution (OOD) detection benchmarks, as they offer a straightforward and interpretable evaluation of a model's effectiveness in distinguishing between ID and OOD samples under consistent conditions. OOD detection is typically approached as a binary classification problem, where In-distribution (ID) pixels represent the negative class while OOD pixels are classified as the positive class.

5.1.3.1 AUROC

Area under the Receiver Operating Characteristic Curve (AUROC) is a threshold-independent evaluation metric (DAVIS und GOADRICH, 2006) that mainly assesses the performance of a binary classification model, particularly in distinguishing between negative (In-distribution (ID)) and positive (Out of Distribution (OOD)) samples. The ROC curve is a graph that visualizes the tradeoff between the True Positive Rate (TPR = TP /

TP + FN) and the False Positive Rate (FPR = FP / FP + TN) across various classification thresholds. AUROC represents the area under this curve and provides a single scalar value that summarizes the models ability to discriminate between positive and negative samples, regardless of the threshold chosen (FAWCETT, 2006).

The AUROC score can be interpreted as the probability that a randomly chosen positive sample is assigned a higher detection score than a randomly chosen negative sample by the model's detection score (e.g., confidence probability, entropy, energy, or uncertainty) (FAWCETT, 2006). More specifically, it is the measure that an OOD sample has a greater detector score/value than an ID sample. The AUROC score ranges from 0 to 1 or equivalently 0% - 100%, where a higher score reflects better separability between the two categories In-distribution (ID) and Out of Distribution (OOD), making it a crucial metric in tasks such as anomaly detection and OOD detection. A perfect model would have an AUROC score of 1, indicating flawless separation between In-distribution (ID) and Out of Distribution (OOD) samples, while a score of 0.5 indicates random performance with no discrimination ability.

5.1.3.2 FPR95

FPR95, short for the False Positive Rate at 95% True Positive Rate, is a widely used metric for evaluating the performance of binary classifiers in tasks such as Out of Distribution (OOD) detection. It provides a balance between the classifier's ability to correctly identify OOD samples and its ability to correctly classify ID samples. Formally, the False Positive Rate (FPR) and True Positive Rate (TPR) are calculated as:

$$\textbf{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{5.6}$$

$$\textbf{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{5.7}$$

where **FP** is the number of false positives (ID samples misclassified asOOD), **TN** is the number of true negatives (correctly classified ID samples), **TP** is the number of true positives (OOD samples correctly classified as OOD),

and **FN** is the number of false negatives (OOD samples misclassified as ID).

The **FPR95** metric specifically measures the False Positive Rate when the True Positive Rate is fixed at 95%. This means it evaluates the proportion of ID samples incorrectly classified as OOD at a threshold where 95% of OOD samples are correctly identified. A lower FPR95 indicates better performance, as it reflects a model that not only excels at identifying OOD samples (achieving a high TPR) but also minimizes misclassifications of ID samples as OOD (low FPR) (CHAN et al., 2021). This makes FPR95 a particularly vital metric for assessing the trade-off between sensitivity (high TPR) and specificity (low FPR), which is crucial in applications where misclassifying ID samples as OOD could have significant consequences.

5.2 Experiments and Results

This section presents an in-depth evaluation of the segmentation model under various training regimes. The experiments were conducted on the Cityscapes dataset and are evaluated on the test set, aiming to assess the effectiveness of proposed training regimes such as Monte Carlo Dropout (MCD), Online Hard Example Mining (OHEM), and Evidential Deep Learning (EDL) when applied to the segmentation model during training. The results are summarized in the following sections.

5.2.1 Vanilla Models Segmentation Performance

The initial set of experiments focuses on evaluating the segmentation performance of the baseline model and its variants trained with MCD, OHEM, and EDL individually. We refer to these models as vanilla models in this thesis. As detailed in the previous sections, the segmentation performance is evaluated using three key metrics: Mean Intersection Over Union (mIoU), overall accuracy, and mean accuracy. Each training method brings its own distinct dynamics to the learning process, which is clearly illustrated in Table 5.1.

The baseline model demonstrates high performance, establishing a strong reference point for comparison. It achieves an mIoU of 92.7%, an overall accuracy of 96.0%, and a mean accuracy of 86.2%. These results con-

firm its robustness and generalizability across the test set, establishing a solid high-performance benchmark for the advanced training techniques to build upon.

On the other hand, integrating MCD as a guidance signal in training yields a mIoU of 92.4% and an overall accuracy of 95.8%. While the overall segmentation performance of MCD training regime is comparable to the baseline, there is a slight reduction in performance. The marginal performance drop is likely due to the stochastic random sampling introduced by MCD during training, which, while useful for uncertainty estimation, can introduce noise during training and slightly reduce segmentation precision.

Vanilla Models	**Segmentation Performance**		
	mIoU ↑	**Overall Accuracy ↑**	**Mean Accuracy ↑**
Baseline	92.7	96.0	86.2
MCD	92.4	95.8	85.9
OHEM	**92.8**	**96.1**	**86.3**
EDL	90.5	94.2	69.5

Table 5.1: Segmentation Performance of Vanilla Models

OHEM as a guidance signal enhances segmentation performance by prioritizing challenging samples during training. Compared to the baseline model, OHEM achieves a marginal improvement of 0.1% in all the segmentation metrics. Although these numerical gains appear modest, they are notable considering the already high baseline performance on the Cityscapes dataset. Such improvements reflect OHEM's ability to refine the models handling of hard-to-classify regions, enhancing robustness in challenging scenarios. This suggests that OHEM is particularly effective for datasets with complex structures, where standard training regimes may struggle to address edge cases and imbalanced samples.

In contrast, Evidential Deep Learning (EDL) introduces a noticeable drop in performance compared to the baseline. The mIoU for EDL drops to 90.5%, with an overall accuracy of 94.2% and a mean accuracy of 69.5%. Although EDL aims to improve the model's ability to quantify uncertainty in its predictions, as will be demonstrated in the subsequent results, it faces challenges in maintaining competitive segmentation accuracy. This is especially evident in its class-level consistency, which is clearly shown

by the significant drop in mean accuracy. This limitation may stem from EDL's sensitivity to class imbalance, as it often overestimates uncertainty for minority classes. While this trade-off limits its utility for segmentation tasks requiring high accuracy, EDL holds promise in scenarios where uncertainty estimation outweighs raw performance, such as in real-time decision-making systems.

5.2.2 Vanilla Model Varaints

Building upon the vanilla models, we explored combinations of training techniques to assess their potential. Table 5.2 details the results of these experiments.

The MCD+OHEM variant integrates MCD with OHEM during training to enhance segmentation performance. MCD introduces stochasticity into the models forward pass by enabling dropout during multiple forward passes, allowing for the calculation of predictive uncertainty based on the variance across these passes at the pixel level. The uncertainty derived from MCD is then incorporated into the loss function, providing a guidance signal that influences the model's learning process. Subsequently, OHEM is employed to prioritize hard, uncertain examples, those with the highest loss, ensuring that the model focuses on challenging, often misclassified pixels during training. This combination aims to harness the complementary strengths of both techniques: MCD's uncertainty modeling to guide the model's attention to less certain regions, while OHEM dynamically selects hard examples to improve the model's generalization performance. Despite these synergies, the variant yields a mIoU of 92.4%, overall accuracy of 95.9%, and mean accuracy of 85.6%, which is comparable to the standalone MCD and slightly lower than OHEM. This suggests that the integration of MCD's uncertainty modeling with OHEM's focus on hard examples does not lead to substantial performance gains as expected. The two methods, addressing uncertainty in fundamentally different ways, may not align as effectively as anticipated. The stochastic nature of MCD could introduce variability in the models behavior, which may conflict with OHEMs more deterministic approach to hard example selection. Thus, while both techniques aim to improve generalization,

their differing approaches to uncertainty may limit the combined effectiveness in this configuration.

Vanilla Models Variants	Segmentation Performance		
	mIoU ↑	Overall Accuracy ↑	Mean Accuracy ↑
MCD + OHEM	92.4	95.9	85.6
EDL + OHEM	89.6	94.2	60.7
EDL + UNC	89.4	94.0	59.6

Table 5.2: Evaluation of Segmentation Performance Under Different Configurations

The EDL+OHEM variant combines EDL with OHEM to address both uncertainty modeling and hard example mining. EDL models epistemic uncertainty by utilizing a distribution over the network parameters and producing a probabilistic output. During training, EDL encourages the model to estimate uncertainty over its predictions, which can help in regions where the model is less confident. However, EDL struggles with class imbalance and challenges in focusing on the most difficult examples. To address this, OHEM is introduced to dynamically focus the model's training on hard examples, i.e., those with the highest loss, ensuring that the model focuses on areas where it is most likely to fail. Despite this combined approach, the EDL+OHEM configuration results in an mIoU of 89.6%, overall accuracy of 94.2%, and mean accuracy of 60.7%, which is notably lower than both MCD+OHEM and standalone OHEM. This suggests that EDL's ability to model uncertainty does not fully address the limitations caused by class imbalance, and even with OHEM targeting difficult examples, the model struggles to achieve significant improvements.

Finally, the EDL+UNC variant uses the uncertainty derived directly from the EDL model. The EDL framework computes epistemic uncertainty from the evidence parameters and the Dirichlet strength, which represent the model's confidence in its predictions. This uncertainty is incorporated into the loss function, weighing the loss of uncertain predictions more heavily to help the model focus on difficult areas. Despite this, the variant achieves a mIoU of 89.4%, an overall accuracy of 94.0%, and a mean accuracy of 59.6%, indicating that while uncertainty weighting is applied, it does not provide substantial performance improvement in this configuration.

5.2.3 Per Category Performance of Vanilla Models

This section presents the per-class accuracy analysis across the four vanilla models. The Table 5.3 reveals notable trends in the performance of the four models across the 19 classes of the Cityscapes dataset, with the classes arranged in descending order of their frequency in the dataset.

For the highly frequent classes in the training dataset, such as Road, Building, and Sky, all models perform well, with minor differences in accuracy. OHEM marginally outperforms Baseline and MCD, while EDL delivers comparable results, indicating that all approaches effectively handle these high-frequency categories.

Classes	**Accuracy**			
	Baseline	**MCD**	**OHEM**	**EDL**
Road	99.1	98.9	99.2	99.0
Building	96.4	96.3	96.7	94.9
Vegetation	96.7	96.3	96.4	95.1
Car	97.6	97.2	97.5	95.9
SideWalk	91.8	92.8	91.7	89.4
Sky	98.4	98.2	98.5	98.0
Pole	73.5	71.8	74.2	70.5
Person	88.0	90.0	90.8	85.4
Terrain	73.3	70.7	76.2	74.2
Fence	64.7	69.2	64.9	60.4
Wall	67.6	61.6	67.2	65.5
Traffic Sign	88.2	86.9	88.8	85.7
Bicycle	88.7	87.9	87.4	87.7
Truck	88.1	90.7	88.3	59.6
Bus	92.6	92.8	92.6	0
Train	88.2	88.4	88.5	81.0
Traffic Light	84.8	83.6	85.1	0.1
Rider	79.9	78.4	79.0	30.7
Motor Cycle	80.1	79.8	77.6	48.4

Table 5.3: Per-Class Accuracy of Vanilla Models: The rows depict the 19 classes from the Cityscapes Dataset, organized by their frequency. Each row lists the corresponding accuracy values for the different models (Baseline, MCD, OHEM, and EDL) in the columns.

However, EDL struggles significantly with certain moderate and low-frequency classes. Notable declines are observed in Person, Motor Cycle, Truck, and Rider, with particularly poor performance in Traffic Light (0.1%) and Bus (0%), where it fails to maintain competitive performance. These results suggest that EDL's uncertainty-based approach may not effectively address rare and challenging classes, which might require a more focused or deterministic learning approach. In contrast, Baseline and MCD demonstrate more stable performance for these categories, though they too experience some performance dips compared to OHEM.

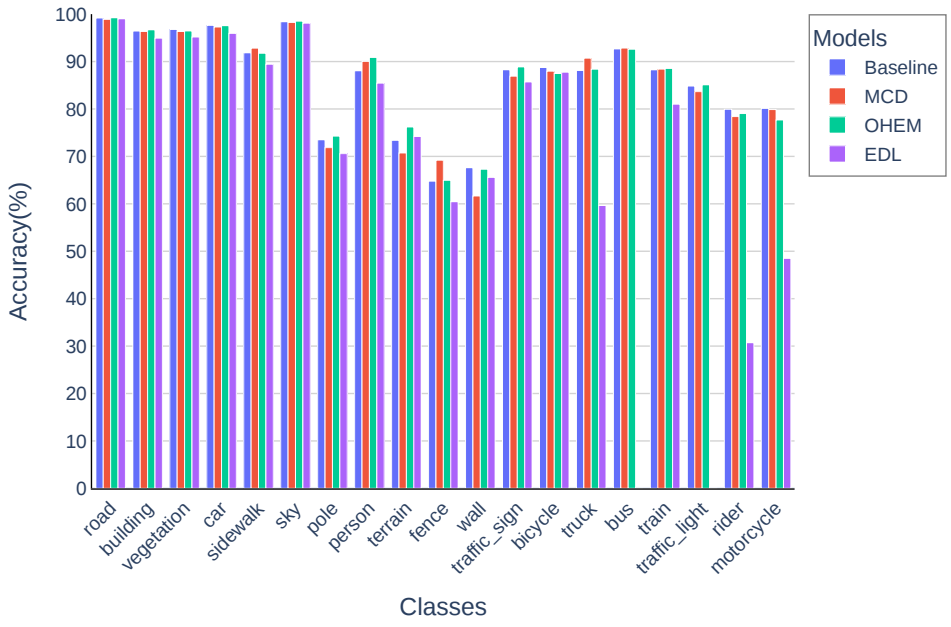


Figure 5.2: Accuracy of vanilla models across classes, ordered by descending class frequency in the dataset.

OHEM, on the other hand, excels in handling moderately frequent classes. The benefits of focusing on difficult samples are particularly noticeable in Pole (74.2%) and Terrain (76.2%), where OHEM consistently outperforms the other models.

In summary, Baseline and MCD offer stable performance across most classes. OHEM excels in addressing difficult and underrepresented classes, making it the most effective approach for handling low-frequency samples. Meanwhile, EDL shows clear limitations in rare and ambiguous classes,

indicating that its uncertainty modeling approach requires refinement to match the robustness of Baseline, MCD, or OHEM in such scenarios.

5.2.4 Finetuned Models Segmentation Performance

To overcome the limitations of standalone training methodologies and enhance the segmentation performance, we adopted a finetuning strategy. This involved initializing all models with the pre-trained parameters from the baseline model and then training them further. This approach leverages the solid foundational knowledge captured in the baseline model, ensuring that the subsequent methods start from a strong, well-generalized state. This process aimed to optimize the model's performance while addressing the challenges posed by complex and low-frequency classes in the dataset.

As presented in Table 5.4, the finetuned models demonstrated notable improvements across various performance metrics, with all models exhibiting similar performance and only marginal differences. Among the finetuned models, MCD+OHEM emerged as the top performer, achieving the highest mIoU of 93.0%, an overall accuracy of 96.2%, and a mean accuracy of 86.7%.

Finetuned Models	**Segmentation Performance**		
	mIoU ↑	**Overall Accuracy ↑**	**Mean Accuracy ↑**
Baseline	92.7	96.0	86.2
MCD	92.9	**96.2**	86.5
OHEM	92.9	**96.2**	86.2
EDL	92.2	95.7	85.5
MCD + OHEM	**93.0**	**96.2**	**86.7**
EDL + OHEM	92.2	95.7	85.4
EDL + UNC	92.3	95.8	85.6

Table 5.4: Segmentation Performance of Finetuned Models

In general, the models performed quite similarly, with some improvements observed across the board. EDL, while showing slightly lower performance than MCD and OHEM, still demonstrated substantial improvements from its vanilla model. With a mIoU of 92.2%, an overall accuracy of 95.7%, and a mean accuracy of 85.5%, EDL exhibited a solid

gain compared to its vanilla model. Although it did not surpass the other models by a small margin, the improvements are noteworthy. This shows that EDL has the potential to be a strong contender when fine-tuned properly. These results are consistent across performance metrics and per-class analysis Table 5.5, where all models showed improvements in handling various categories. Furthermore, the finetuned EDL+OHEM and EDL+UNC models showed performance that remained comparable to EDL on most metrics, reinforcing the observation that EDL can still achieve competitive results, with slight enhancements through uncertainty fusion techniques.

MCD and OHEM independently also showed meaningful improvements compared to the baseline. Both models achieved a mIoU of 92.90%, surpassing the Baseline's 92.7% by a small margin. MCD and OHEM also recorded higher overall accuracy values (96.2%) and mean accuracy (86.5% and 86.2%, respectively), demonstrating their robustness and effectiveness in fine-tuning.

5.2.5 Per Category Performance of Finetuned Models

The fine-tuned models demonstrated clear performance enhancements across all categories compared to their vanilla versions, reflecting the success of the refinement process. While MCD and OHEM consistently maintained their leadership in overall accuracy and per-class performance, the improvements in EDL, particularly for challenging and rare classes, were noteworthy.

For high-frequency classes such as Road, Building, and Sky, the fine-tuned models MCD and OHEM displayed slight gains over their vanilla configurations. EDL also exhibited improvements in these classes, particularly mitigating the performance drop seen in its vanilla version for Building, reinforcing its ability to effectively handle well-represented categories after fine-tuning.

In moderately frequent categories like Person and Car, MCD and OHEM retained their competitive edge, showcasing consistent improvements over the baseline. MCD achieved slightly higher stability across these classes, but EDL also demonstrated significant gains compared to its vanilla performance. For example, in Person, EDL showed a considerable leap in ac-

Classes	Accuracy			
	Baseline	**MCD**	**OHEM**	**EDL**
Road	99.1	99.2	99.2	99.0
Building	96.4	96.8	96.8	95.9
Vegetation	96.7	96.5	96.6	95.9
Car	97.6	97.7	97.7	97.3
SideWalk	91.8	91.7	92.0	91.4
Sky	98.4	98.5	98.5	98.2
Pole	73.5	73.7	73.7	72.3
Person	88.0	90.1	90.4	89.1
Terrain	73.3	72.9	70.9	73.1
Fence	64.7	65.5	64.5	65.8
Wall	67.6	68.4	69.8	71.7
Traffic Sign	88.2	88.1	88.1	86.6
Bicycle	88.7	86.9	88.2	86.9
Truck	88.1	89.3	88.4	88.2
Bus	92.6	92.6	92.5	92.6
Train	88.2	91.0	89.6	87.2
Traffic Light	84.8	84.9	85.0	84.2
Rider	79.9	80.0	77.1	73.4
Motor Cycle	80.1	79.1	79.7	76.4

Table 5.5: Per-Class Accuracy for Finetuned Models: The rows depict the 19 classes from the Cityscapes Dataset, organized by their frequency. Each row lists the corresponding accuracy values for the different models (Baseline, MCD, OHEM, and EDL) in the columns.

curacy, reducing the performance gap with the leading models and highlighting its ability to adapt effectively when fine-tuned.

The most striking improvements were observed in low-frequency and rare classes such as Pole, Wall, and Truck, where fine-tuning greatly enhanced the performance of all models. MCD stood out in these categories, achieving the highest accuracy, particularly for Pole and Truck. However, EDL and MCD also showed substantial growth, especially for Wall, where they overcame significant limitations observed in the vanilla models.

For particularly difficult classes such as Rider, Motorcycle, Traffic Light, and Bus, fine-tuning revealed contrasting dynamics. Rider, a challenging category due to its ambiguity, saw improvements across all models except

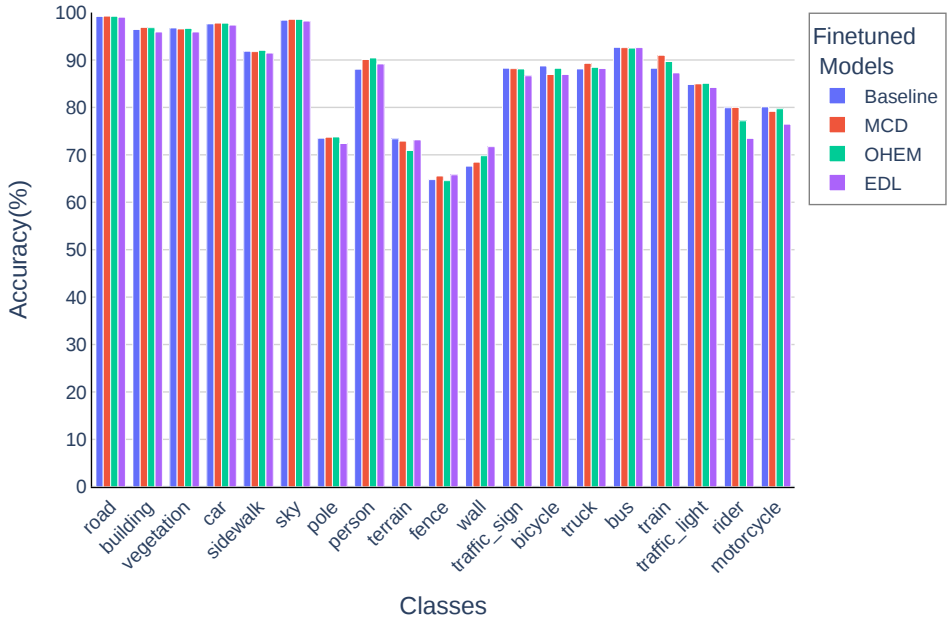


Figure 5.3: Accuracy of finetuned models across classes, ordered by descending class frequency in the dataset.

OHEM that showed a slight decline. MCD and OHEM led in absolute accuracy, but EDL's leap from its vanilla version was substantial, reflecting the impact of uncertainty-based learning. Bus and Traffic Light, despite being challenging categories, demonstrated the most significant improvements in EDL after fine-tuning. For Bus and Traffic Light, EDL transitioned from an almost unusable state in the vanilla version to performing on par with MCD and OHEM, achieving competitive accuracy. Similarly, for the Motor Cycle category, EDL's fine-tuned model closed the gap significantly, showing resilience in adapting to edge cases.

In summary, fine-tuning contributed to consistent gains across all models. MCD and OHEM maintained their strong overall performance, showing incremental but meaningful improvements in both frequent and rare categories. EDL, however, showcased the most dramatic progress, particularly for challenging classes where its initial performance lagged significantly.

In addition to quantitatively evaluating segmentation, we provide qualitative examples for both the vanilla and finetuned models, as shown in Figure 5.4. The first row displays the original input RGB image along with the

corresponding ground truth label. The following rows illustrate the evaluated models segmentation predictions, a probability map, an entropy map and the predictive uncertainty for both the vanilla and finetuned models. Overall, both model types perform exceptionally well, exhibiting minimal errors in large areas and for well-represented classes such as road, building, and sky. Additionally, a strong correlation exists among the confidence map, entropy map, and predictive uncertainty, suggesting that all models offer meaningful confidence levels and uncertainties. However, there are subtle differences between the models that are worth noting.

For instance, in the vanilla models of Figure 5.4, noticeable misclassifications occur, particularly with the distant rider next to cars on the road. Despite misclassifying the rider and bicycle as a car, these models fail to assign higher uncertainty and lower confidence to these errors. Additionally, all models struggle with the Pole category, often assigning medium confidence to uncertain pixels. However, EDL stands out by assigning meaningful uncertainty in these areas, effectively indicating when it is uncertain.

The vanilla EDL model, while demonstrating lower segmentation performance than other vanilla models as reflected in quantitative metrics, also exhibits underconfidence in its confidence map. It struggles to predict the rider and bicycle but compensates with high uncertainty and low confidence, demonstrating its ability to recognize uncertainty. In contrast, the finetuned MCD and OHEM models show only marginal improvements in segmentation performance. The most notable improvement is observed in the fine-tuned EDL model, which delivers better segmentation predictions. While the vanilla EDL model fails to correctly segment the rider, person on the road, and fence on the right side of the image, its finetuned counterpart performs slightly better in these areas.

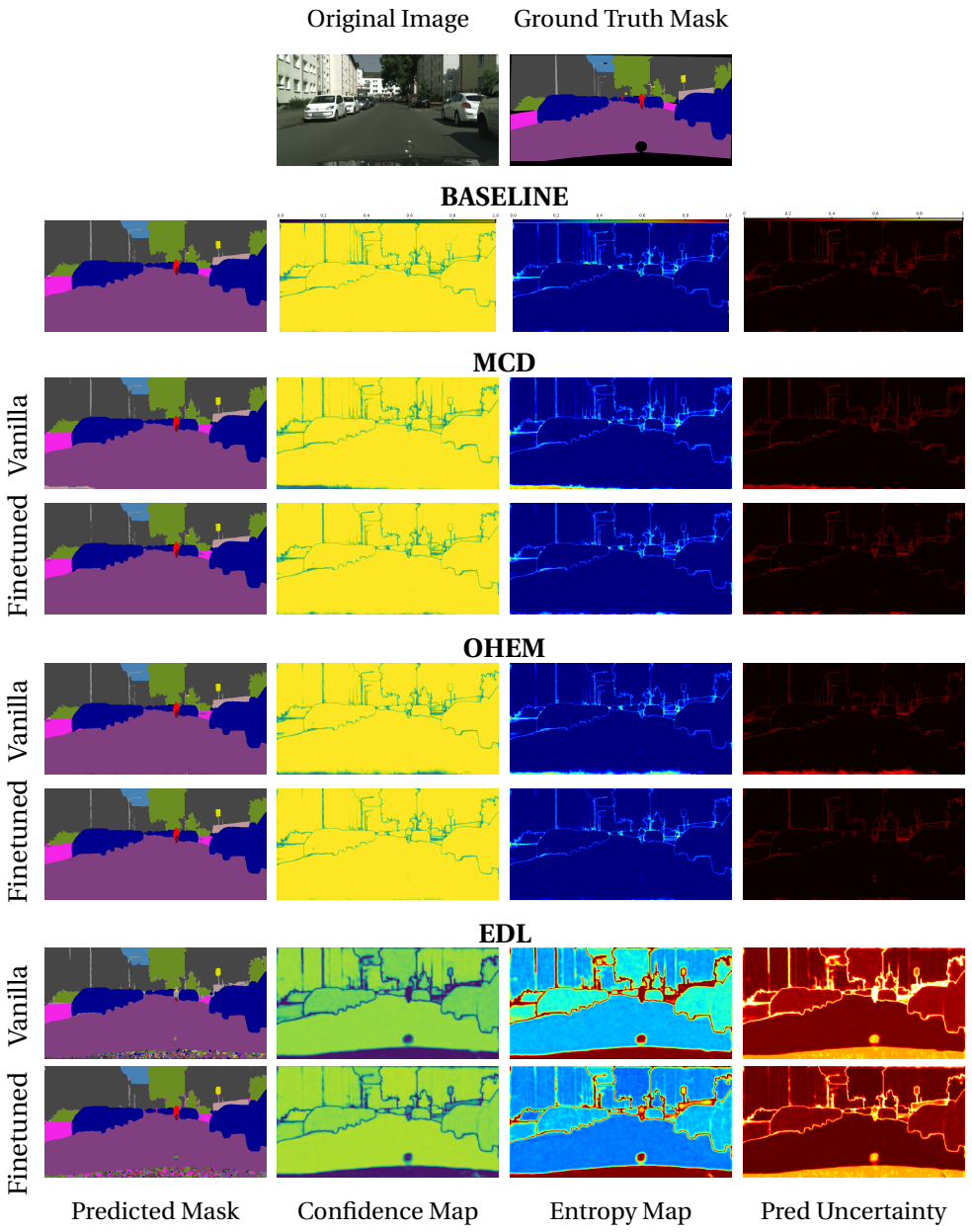


Figure 5.4: Predictions on the Cityscapes dataset. The first row displays the Original Image and the Ground Truth Mask. Each subsequent section corresponds to a specific model (Baseline, MCD, OHEM, EDL), with two rows: the first representing the vanilla model and the second representing the finetuned model. Within each section, the columns display the predicted segmentation mask, confidence map, entropy map, and predictive uncertainty map. In the confidence map, yellow indicates high confidence while blue indicates low confidence. In the entropy map, blue represents low entropy and red represents high entropy. Similarly, in the uncertainty map, brighter pixels indicate higher predictive uncertainty.

5.2.6 Calibration Performance

In this section, to evaluate the calibration performance, we present ECE metrics along with the reliability diagrams for the evaluated models in both their vanilla and fine-tuned configurations. Table 5.6 highlights that fine-tuning yields varying levels of improvement across models, with a clear trend of better-calibrated predictions for uncertainty-aware approaches. However, certain configurations exhibited mixed results, suggesting that the calibration gains depend heavily on the interplay between the model architecture and training strategies.

Interestingly, while models such as MCD+OHEM exhibited strong segmentation performance, their calibration gains were more modest. The ECE for MCD+OHEM increased slightly post-fine-tuning (from 0.266 to 0.281), suggesting potential challenges in managing mispredictions or overconfidence. This observation emphasizes the complexity of balancing segmentation accuracy and calibration in hybrid approaches. The standalone OHEM model saw a reduction in ECE from 0.292 (vanilla) to 0.273 (fine-tuned), showcasing its ability to better align prediction confidence with correctness by focusing on hard-to-learn examples. However, when combined with MCD, the calibration benefits were less pronounced when finetuned, indicating potential conflicts in how uncertainty is managed during training.

Models	**ECE ↓**	
	Vanilla	**Finetuned**
Baseline	0.282	
MCD	0.291	0.288
OHEM	0.292	0.273
EDL	**0.128**	**0.155**
MCD + OHEM	0.266	0.281
EDL + OHEM	**0.113**	**0.110**
EDL + UNC	**0.117**	**0.109**

Table 5.6: Calibration Performance of Models

Conversely, EDL-based models demonstrated robustness across configurations, achieving the most consistent calibration improvements. Even when fine-tuning introduced slight increases in ECE for vanilla EDL (0.128 to 0.155), these models still consistently achieve superior calibration

across all the models, far outperforming the baseline model (ECE of 0.282). The fine-tuned EDL-based variants further enhanced this advantage, with EDL+OHEM and EDL+UNC achieving the lowest ECE values of 0.110 and 0.109, respectively. These improvements highlight the effectiveness of EDL refining model confidence and reducing overconfidence.

5.2.6.1 Reliability Diagrams

To shed light further on the calibration dynamics observed through ECE metrics, we presented the reliability diagrams for the models in their vanilla and finetuned forms. These diagrams provide a visual representation of confidence calibration, highlighting where models exhibit overconfidence or underconfidence across their prediction range by providing insights into the relationship between model confidence and actual accuracy. As detailed in section 3.5, each reliability diagrams consist of two subplots: the top plot visualizes the accuracy of predictions within different confidence bins, while the bottom histogram illustrates the distribution of test examples across confidence bins. Ideally, a well-calibrated model should have accuracy matching confidence, meaning predictions align with the diagonal reference line. The red bars represent miscalibration, highlighting the gap between accuracy and confidence, while the black lines indicate the actual accuracy per bin.

The baseline model exhibits poor calibration, with an ECE of 0.282, indicating a substantial confidence-accuracy mismatch. The reliability diagram for the baseline model in Figure 5.5 shows that across all bins, the accuracy (black line) consistently falls below the diagonal, meaning the model is systematically overconfident. The confidence gap (red bars), representing the gap between accuracy and confidence, increases as confidence rises, particularly in higher confidence bins (0.6-1.0), confirming significant miscalibration and overconfidence. The histogram at the bottom reveals that most predictions fall in the 0.7-1.0 confidence range, suggesting that the model rarely produces low-confidence outputs and frequently makes high-confidence predictions that do not align with actual accuracy.

The vanilla MCD model does not exhibit any substantial improvement in calibration, as reflected in its higher ECE of 0.291. Its reliability dia-

gram shows a similar miscalibration pattern, where the accuracy (black line) consistently remains lower than confidence, falling below the diagonal across all bins and reflecting overconfidence. The confidence gap (red bars) remains large, particularly in the high-confidence bins 0.7-1.0, reinforcing that the model assigns high confidence to incorrect predictions. The histogram suggests most predictions are clustered in the high-confidence range (0.7-1.0), similar to the baseline model, highlighting a lack of calibration despite the Monte Carlo Dropout (MCD) approach.

The vanilla OHEM model follows a similar miscalibration trend with an ECE of 0.292, confirming overconfidence in predictions. The reliability diagram shows accuracy consistently lower than confidence, particularly in mid-to-high confidence bins, with large red bars indicating miscalibration. The histogram shows a right-skewed confidence distribution, where most predictions fall into higher confidence bins, reinforcing that the model assigns excessive confidence without properly reflecting uncertainty.

The EDL model demonstrates significantly improved calibration, achieving an ECE of 0.1287, which is substantially lower than the other vanilla models. As shown in the Figure 5.5, the accuracy (black line) closely follows the diagonal across most confidence bins, particularly in the mid-confidence range (0.3-0.7). This suggests that the model's confidence levels better align with its actual performance compared to the other vanilla models. The red bars, which indicate the calibration gap, are notably reduced compared to the other models, confirming improved calibration. However, some miscalibrations persist in the lower confidence bins, (0.0-0.3), where the accuracy line sits above the diagonal. This indicates slight underconfidence, meaning the model is more cautious when assigning low-confidence predictions. In contrast, at higher confidence levels (0.7-0.9), the model exhibits mild overconfidence, where its predicted confidence exceeds actual accuracy. A particularly notable miscalibration occurs in the highest confidence bin, (0.9-1.0), where accuracy drops significantly. This suggests that when the model is extremely confident, its predictions may sometimes be unreliable, indicating a potential failure mode in extreme certainty cases. The confidence histogram further supports these observations. Unlike the other vanilla models, which exhibit a strong skew toward high-confidence predictions, the EDL model shows a more uniform distribution across different confidence levels. This sug-

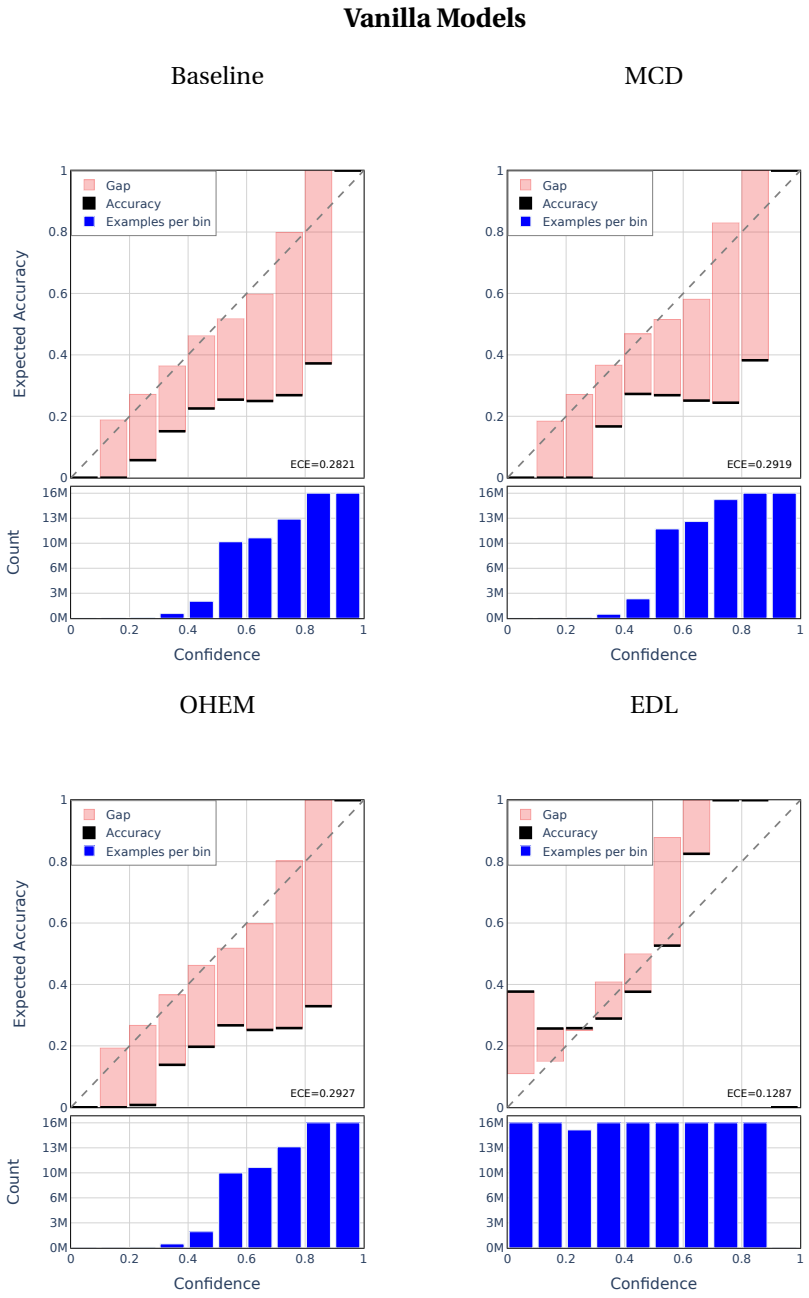


Figure 5.5: Reliability Diagrams of Vanilla Models

gests that it expresses uncertainty more effectively rather than defaulting to overconfident predictions. Additionally, the absence of predictions in the 0.9-1.0 confidence bin suggests that the model avoids making highly

confident predictions unless absolutely certain, which contributes to its improved reliability. Overall, the EDL model exhibits superior calibration performance over the remaining vanilla models, making it more reliable for probabilistic decision-making.

The reliability diagrams of fine-tuned models are depicted in Figure 5.6. The MCD model shows marginal improvement, with a slightly lower ECE of 0.288 compared to the vanilla model (0.291). The red bars remain significant in high-confidence bins, indicating persistent overconfidence, though slight reductions in calibration error. The confidence histogram remains similar to the vanilla version, meaning fine-tuning did not significantly impact how the MCD model assigns confidence. The OHEM model shows a slight improvement in calibration, with its ECE reduced to 0.278 from 0.292. The reliability diagram still exhibits overconfidence, but the red bars are very slightly smaller than in the vanilla model, suggesting marginal calibration improvements. The histogram remains skewed towards higher confidence values, indicating that while fine-tuning has helped slightly, the model remains overconfident.

The fine-tuned EDL model exhibits a slightly higher ECE of 0.1559, indicating a minor degradation in calibration compared to its vanilla counterpart (ECE=0.1287). This may be attributed to the fact that the models are initialized with baseline parameters during fine-tuning. However, it still outperforms the other models in terms of overall calibration. As observed in the Figure 5.6, the accuracy (black line) of the EDL model generally follows the diagonal across most confidence bins, suggesting that the model maintains a reasonable alignment between confidence and accuracy. However, compared to its vanilla version, the red calibration gaps have slightly increased, suggesting that fine-tuning has introduced some miscalibration. In the lower confidence bins (0.0-0.3), the model still appears underconfident. Meanwhile, in the mid confidence bins (0.4-0.7), overconfidence remains, where predicted confidence exceeds actual accuracy. Meanwhile, in the higher confidence bins (0.8-0.9), the accuracy line is also positioned above the diagonal, indicating underconfidence in this range. This suggests that the model underestimates its own accuracy, meaning it makes correct predictions more often than it expects. This underconfidence may have resulted from fine-tuning introducing regularization effects, making the model more conservative in assigning confidence scores. Unlike the

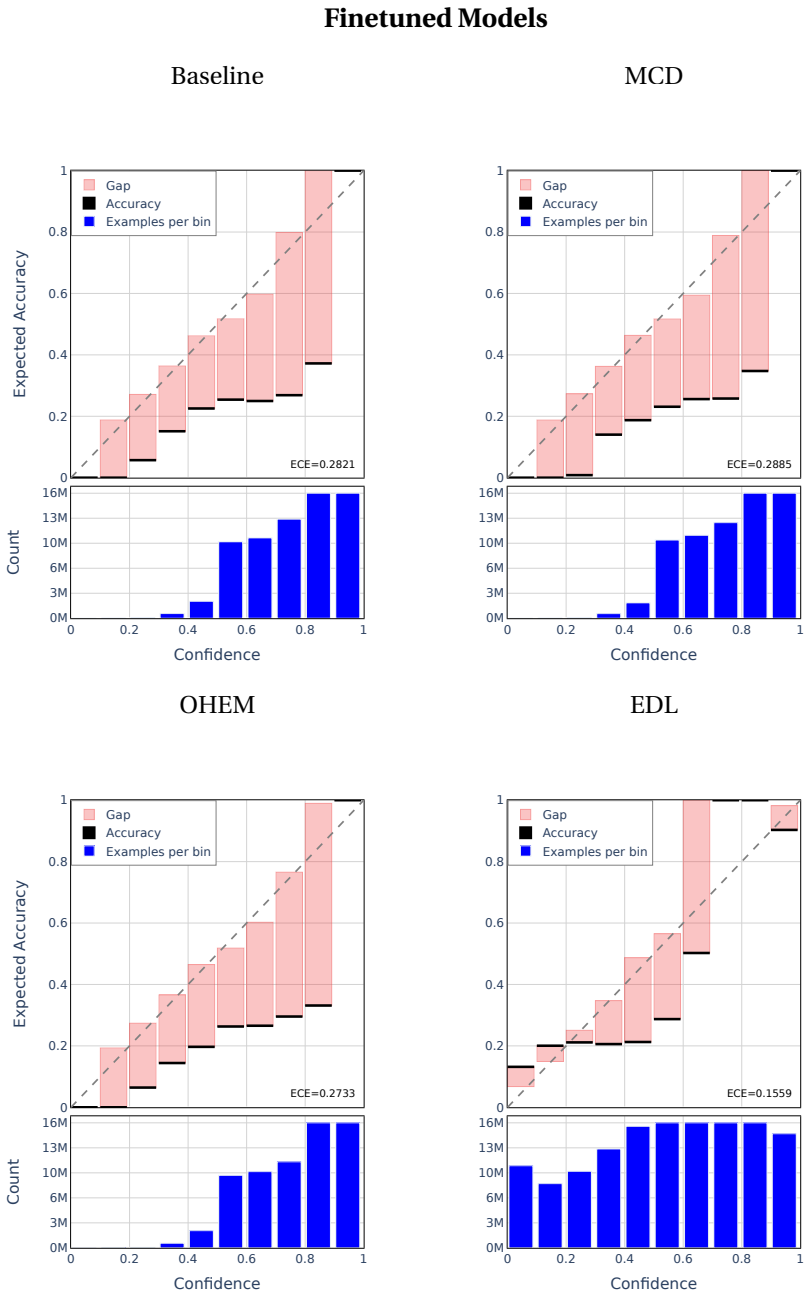


Figure 5.6: Reliability Diagrams of Finetuned Models

vanilla version, the fine-tuned model demonstrates improved accuracy at extreme confidence levels (0.9-1.0), although it is slightly overconfident.

The confidence histogram suggests that fine-tuning has shifted more predictions toward higher confidence levels, slightly reducing the previously observed uniform distribution. While the model still avoids placing too many predictions in the 0.9-1.0 confidence bin, it now has a higher concentration of predictions in the 0.7-0.9 range, reinforcing its overconfidence trend. Overall, while the fine-tuned EDL model remains the best-calibrated among all models, its increased ECE (0.1559) suggests that fine-tuning has introduced slight miscalibrations, particularly in the highest confidence range. Nevertheless, it continues to be the most reliable model than the other fine-tuned models in terms of calibration.

To further evaluate the calibration performance, we analyzed the variability of key model metrics of fine-tuned models on a per-image basis from the Cityscapes test set, as shown in Table 5.7. We computed essential metrics such as IoU, accuracy, and ECE for individual test images, aggregating the results to derive their mean and standard deviations (std). This approach provides a deeper understanding of model consistency across diverse image inputs.

Models	**Cityscapes**		
	IoU	**Accurcay**	**ECE**
Baseline	0.9307 ±0.0430	0.9602±0.0279	0.0123±0.0213
MCD	0.9323 ±0.0421	0.9615 ±0.0269	0.0116 ±0.0205
OHEM	0.9322 ±0.0424	0.9614 ±0.0271	0.0127 ±0.0218
EDL	0.9267 ±0.0418	0.9566 ±0.0281	0.1682 ±0.0202
MCD + OHEM	**0.9331 ±0.0407**	**0.9619 ±0.0262**	**0.0112 ±0.0201**
EDL + OHEM	0.9263 ±0.0423	0.9572 ±0.0277	0.2661 ±0.0239
EDL + UNC	0.9274 ±0.0419	0.9580 ±0.0273	0.2261 ±0.0212

Table 5.7: Variability Analysis of key model metrics across images from the Cityscapes Dataset for the fine-tuned models. The table presents mean±std for all the reported metrics.

We analyzed the calibration performance on the entire test set using the ECE metric in Table 5.6. In Table 5.7, we computed the ECE for each cityscapes test image to understand its variability.

The mean and standard deviations in the Table 5.7 reveal that the ECE metric exhibits low variability across images, with standard deviations generally below 0.025 for all the models. From the overall test set per-

spective and reliability diagrams, EDL based models show the lowest ECE, indicating superior calibration compared to all other evaluated models. However, a different trend emerges when analyzing per-image variability (Table 5.7). EDL based models exhibit significantly higher mean ECE and almost similar standard deviations across individual images, with EDL+OHEM(0.0239) demonstrating the most fluctuation. This suggests that while these models achieve strong overall calibration across the dataset, their confidence estimates vary more on a per-image basis. In contrast, MCD and MCD+OHEM achieve lower standard deviations for per-image ECE, at 0.0205 and 0.0201, respectively, along with reduced mean values, indicating more stable calibration across different samples, even though their overall ECE on the full dataset is higher.

This contrast between the overall ECE and per-image variability suggests that EDL based models calibrate well on average but may struggle with consistency across images. In contrast, methods like MCD provide steadier per-image calibration, even if their overall ECE is not as low. To better understand these trends, it would be helpful to examine how image complexity affects both the mean and standard deviation of ECE. Additionally, analyzing the impact of data augmentation techniques on calibration consistency could help identify specific areas for improvement. This deeper analysis could be valuable for future research, revealing the strengths and weaknesses of various models and guiding strategies to enhance overall model calibration on the Cityscapes dataset.

Along with the predictions, we also analyzed the misclassification statistics of the fine-tuned models to understand how each model assigns confidence to its misclassified pixels. We observed that the EDL model concentrated most of its misclassified pixels in the low-confidence range, indicating better uncertainty estimation and calibration. In contrast, the Baseline model assigned the majority of its misclassified pixels to the high-confidence range, despite being incorrect, highlighting overconfidence in its predictions. Surprisingly, a similar trend was observed for the MCD and OHEM models, suggesting that these uncertainty estimation techniques do not significantly mitigate overconfidence in misclassified pixels.

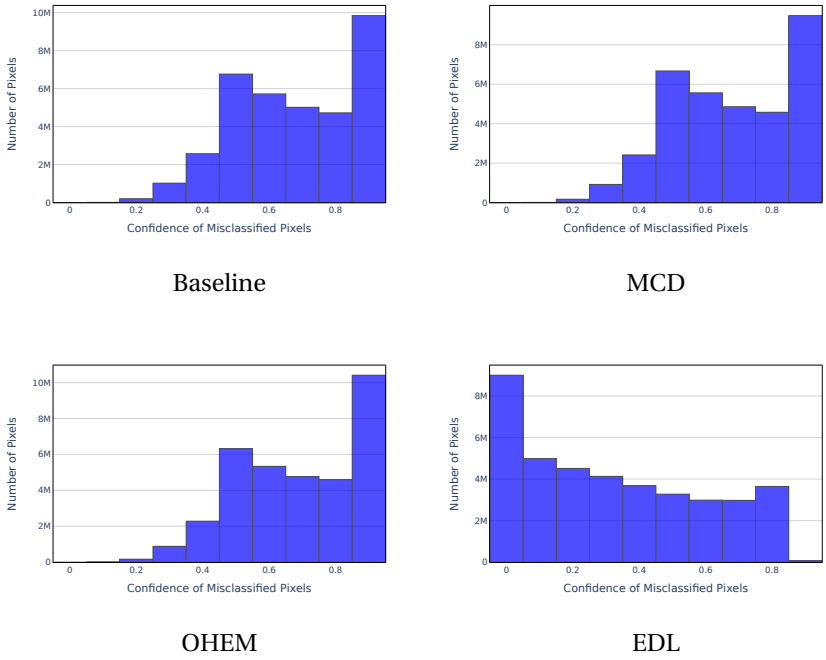


Figure 5.7: Misclassification Distributions on Cityscapes Dataset of finetuned Models. The x-axis represents the confidence of misclassified pixels arranged in 10 bins and the y-axis represents the number of pixels that fall into that confidence bin

5.2.7 OOD Performance evaluation

The Out of Distribution (OOD) performance of both vanilla and fine-tuned models are summarized in Tables Table 5.8 and Table 5.9, respectively. We evaluate the models on two OOD datasets, namely Fishyscapes and LostandFound, using the False Positive Rate at 95 % True Positive Rate (FPR95) and Area under the Receiver Operating Characteristic Curve (AUROC) as key metrics.

In the vanilla setting Table 5.8, the baseline model demonstrates reasonable performance on Fishyscapes, achieving a solid AUROC of 76.43, though its FPR95 is relatively high (40.28). While methods like MCD and OHEM show slight improvements in AUROC, with OHEM showing the most notable gain. However, these models show only minor reductions in FPR95, with values remains close to the baseline, suggesting limited improvements in distinguishing OOD samples. MCD+OHEM performs

the worst on Fishyscapes, with an FPR95 of 45.31, demonstrating a higher tendency to misclassify OOD instances as ID. The EDL model, on the other hand, stands out by achieving a better balance between AUROC and FPR95, particularly showing a notable reduction in FPR95 (from 40.28 to 34.67) while maintaining a comparable AUROC (75.77). This indicates that EDL effectively preserves ID samples while still correctly detecting OOD instances, making it a better-balanced approach for reducing false positives.

Vanilla Models	**OOD Performance**			
	Fishyscapes		**LostandFound**	
	FPR95 ↓	**AUROC** ↑	**FPR95** ↓	**AUROC** ↑
Baseline	40.28	76.43	66.84	68.03
MCD	41.94	76.68	73.98	67.59
OHEM	39.16	**77.29**	77.69	67.21
EDL	**34.67**	75.77	67.99	70.07
MCD + OHEM	45.31	75.37	**59.58**	**70.37**
EDL + OHEM	39.26	74.31	71.55	69.65
EDL + UNC	42.05	72.51	75.44	68.41

Table 5.8: OOD Performance of Vanilla Models on Fishyscapes and LostandFound Datasets

On the LostandFound dataset, the baseline model performs relatively poorly, with an AUROC of 68.03 and a FPR95 of 66.84. Notably, MCD and OHEM models exhibit increased FPR95, indicating a decline in their ability to detect OOD instances. MCD+OHEM significantly improves FPR95, making it the best-performing vanilla model on LostandFound. However, its AUROC of 70.37 remains comparable to its peers, suggesting that while it excels at managing false positives, it does not substantially outperform others in overall discrimination capability. EDL model, however, remains more consistent, achieving a better trade-off between AUROC (70.07) and FPR95 (67.99), demonstrating its robustness in handling OOD data. Interestingly, the variants of the EDL model, namely EDL+OHEM and EDL+UNC, do not maintain this level of performance. Instead, they result in deteriorated outcomes, reflected in higher FPR95 values. This finding reveals that integrating specific techniques may result in unforeseen trade-offs in OOD detection.

Upon fine-tuning, several models exhibit clear improvements as presented in Table 5.9. For instance, the fine-tuned MCD demonstrates both a reduction in FPR95 and an increase in AUROC on Fishyscapes, indicating better handling of OOD data. OHEM fine-tuning provides a moderate AUROC increase (76.85) on Fishyscapes, but its FPR95 also increased slightly (41.53), meaning it still struggles with false positives. On the other hand, MCD+OHEM benefits from fine-tuning, reducing FPR95 from 45.31 to 36.85 on Fishyscapes.

Finetuned Models	**OOD Performance**			
	Fishyscapes		**LostandFound**	
	FPR95 ↓	**AUROC ↑**	**FPR95 ↓**	**AUROC ↑**
Baseline	40.28	76.43	66.84	68.03
MCD	37.22	77.88	60.72	69.09
OHEM	41.53	76.85	66.10	67.76
EDL	**31.54**	78.25	62.51	70.93
MCD + OHEM	36.85	77.86	64.30	68.13
EDL + OHEM	32.04	78.45	**60.13**	**72.40**
EDL + UNC	31.80	**79.13**	69.46	72.19

Table 5.9: OOD Performance of Finetuned Models on Fishyscapes and LostandFound Datasets

Conversely, EDL based models continue to perform the best on Fishyscapes dataset after fine-tuning, with substantial reductions in FPR95 and a marked increase in AUROC, underscoring its enhanced ability to distinguish OOD instances after fine-tuning. EDL alone achieves the lowest FPR95 (31.54) and highest AUROC (78.25) on Fishyscapes, while EDL+OHEM achieves an even better balance, reducing FPR95 from 39.26 to 32.04 and boosting AUROC from 74.31 to 78.45, confirming that OHEM and EDL complement each other when fine-tuned whereas EDL+UNC achieves the highest AUROC (79.13).

For the LostandFound dataset, fine-tuning leads to significant improvements across multiple models, particularly in FPR95, which measures how often Out of Distribution (OOD) instances are misclassified as in-distribution when the TPR is 95%. The MCD model shows a notable reduction in FPR95 from 73.98 to 60.72, highlighting its improved ability to separate Out of Distribution (OOD) from in-distribution data. Similarly, EDL improves its FPR95 from 67.99 to 62.51, along with an increase

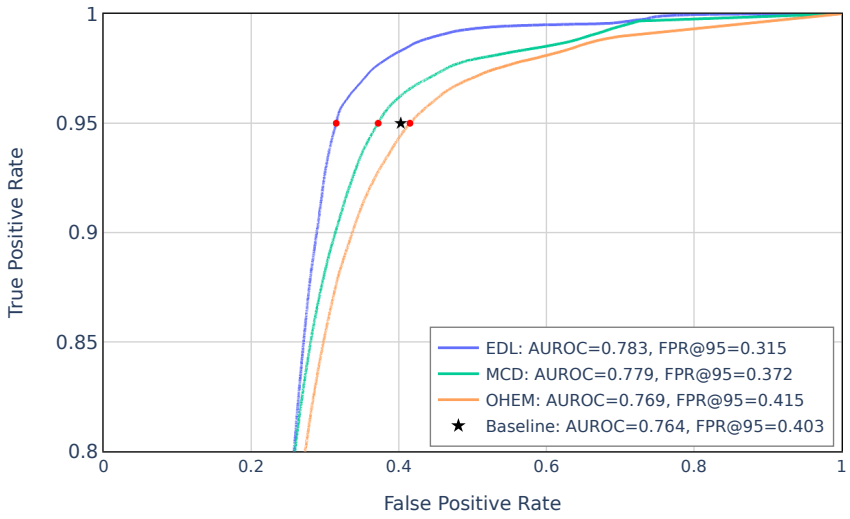


Figure 5.8: ROC Curves of the Finetuned Models on Fishyscapes Dataset

in AUROC from 70.07 to 70.93, demonstrating a more reliable ranking of OOD instances. The OHEM model, which initially performed worse in terms of FPR95 (77.69), benefits from fine-tuning, reducing this metric to 66.10. However, its AUROC remains relatively stable, shifting only slightly from 67.21 to 67.76, indicating that while fine-tuning helps in reducing the number of false positives, its impact on overall detection ranking is minimal. Notably, the EDL+OHEM combination achieves one of the best improvements, with FPR95 decreasing from 71.55 to 60.13 and AUROC rising from 69.65 to 72.40, confirming that integrating hard example mining with uncertainty-aware techniques enhances OOD detection when finetuned.

Interestingly, while most models improve, EDL+UNC presents a unique case. Though it achieves the better AUROC of 72.19, its FPR95 increases slightly to 69.46, suggesting that while the model becomes more confident in ranking OOD instances correctly, it may still struggle with certain OOD cases that are challenging to distinguish from ID data. These results emphasize the importance of fine-tuning in refining the models ability to differentiate between OOD and In-distribution (ID) instances, with

uncertainty-aware techniques like EDL showing the most consistent benefits.

The Out of Distribution (OOD) performance of the models is further illustrated through predictions made on the Fishyscapes dataset and LostAndFound datasets. Figure 5.9 and 5.10 compares the Baseline with vanilla and finetuned versions of the MCD, OHEM, and EDL, showcasing their outputs in terms of predicted masks, probability maps, entropy maps, and uncertainty maps. The uncertainty map or predictive uncertainty map is generated using MCD during inference for all models, except for EDL based models, where uncertainty map is derived implicitly from the model itself. These visualizations illustrate the model's ability to differentiate between In-distribution (ID) and Out of Distribution (OOD) instances, with noticeable improvements observed after fine-tuning.

In both figures, the first row presents the original RGB image from the Cityscapes validation dataset, along with its corresponding ground truth mask. This is followed by the Fishyscapes RGB image, which contains overlaid OOD instances (such as cats and dogs) that were not present in the Cityscapes dataset on which the models were trained. These additions test the models robustness in identifying OOD objects while maintaining accurate segmentation of ID regions.

From Figure 5.9, the Baseline model demonstrates relatively strong segmentation performance for most ID categories but struggles significantly with OOD detection. Although the model misclassifies the cat as a person, the confidence map reveals high confidence across the entire image, including the OOD object, which should ideally have low confidence. This overconfidence is further reinforced by the entropy map, which displays low entropy in OOD regions, suggesting that the model is incorrectly certain about its misclassification. The uncertainty map also confirms this trend, showing minimal uncertainty in regions where it should be significantly higher, indicating poor OOD awareness.

Additionally, it is evident that all vanilla models struggle with segmenting long-distance objects, particularly for categories such as persons, traffic signs, poles, and small vehicles. This issue is noticeable in the predicted masks, where these distant objects appear blurred, misclassified, or missing altogether. As highlighted in Table 5.3, all models exhibit poor perfor-

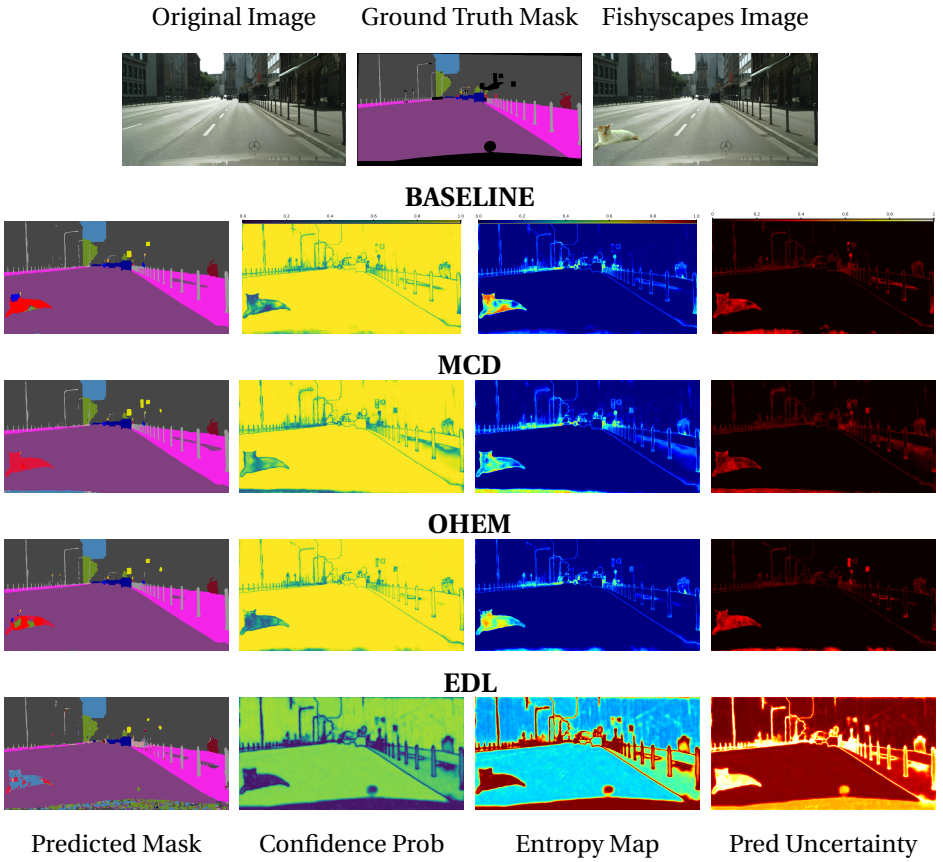


Figure 5.9: Predictions of Vanilla models on Fishyscapes Dataset. The first row shows the input RGB Image, Ground Truth Mask, and the Fishyscapes image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and a Predictive Uncertainty Map.

mance in detecting the Pole category and in preserving object boundaries. The Baseline model, in particular, tends to over-smooth distant objects, leading to a loss of fine details in segmentation, which negatively impacts the overall performance.

The MCD model improves segmentation for some ID classes, especially in larger structures, but it exhibits extreme overconfidence in the OOD object (cat). The confidence map assigns very high confidence to the cat, even more than the Baseline, which indicates that MCD struggles with OOD separation. The entropy map and the uncertainty map fails to highlight

the OOD region as uncertain, reinforcing the models tendency to misclassify OOD objects with high certainty.

The OHEM model, while also struggling with OOD detection, performs slightly better than MCD in this regard. It assigns lower confidence to the cat compared to both Baseline and MCD, as seen in the confidence map. The entropy and uncertainty maps also reflects slightly increased uncertainty in the OOD region, suggesting that OHEM makes some progress in distinguishing OOD objects.

In the vanilla EDL model, segmentation quality is noticeably weaker, with blurry boundaries for roads, sidewalks, and long-distance cars, indicating struggles with sharp transitions. The confidence map further confirms this, as EDL assigns lower confidence to ID objects compared to other models. This aligns with the entropy and uncertainty maps, which shows higher uncertainty not just for OOD regions but also for ID objects, suggesting that the model is underconfident even in ID cases. However, EDL performs well in OOD detection. Unlike other vanilla models, which assign high confidence to the OOD object (cat), EDL correctly lowers its confidence in the confidence map, making the cat more distinguishable from the ID background. The entropy map highlights this improvement by assigning higher entropy to the cat, signaling that the model recognizes it as uncertain. The uncertainty map also reinforces this, clearly marking the OOD object, which other models fail to do effectively.

In the finetuned models Figure 5.10, both MCD and OHEM show similar segmentation performance, with slightly sharper edges for roads and sidewalks. However, they remain overconfident in OOD detection and continue to struggle with misclassification of distant objects, particularly the person on a bicycle next to the cars. The confidence map still assigns high confidence to the cat, and the entropy and uncertainty maps does not effectively capture the uncertainty in OOD regions.

The fine-tuned EDL model shows notable segmentation improvements compared to its vanilla counterpart, though boundaries remain blurry, and ID regions exhibit some underconfidence. For OOD detection, EDL retains its advantage, with the confidence map more distinctly separating the cat as an OOD object. Additionally, the entropy and uncertainty maps clearly highlight OOD regions, while also better capturing the uncertainty

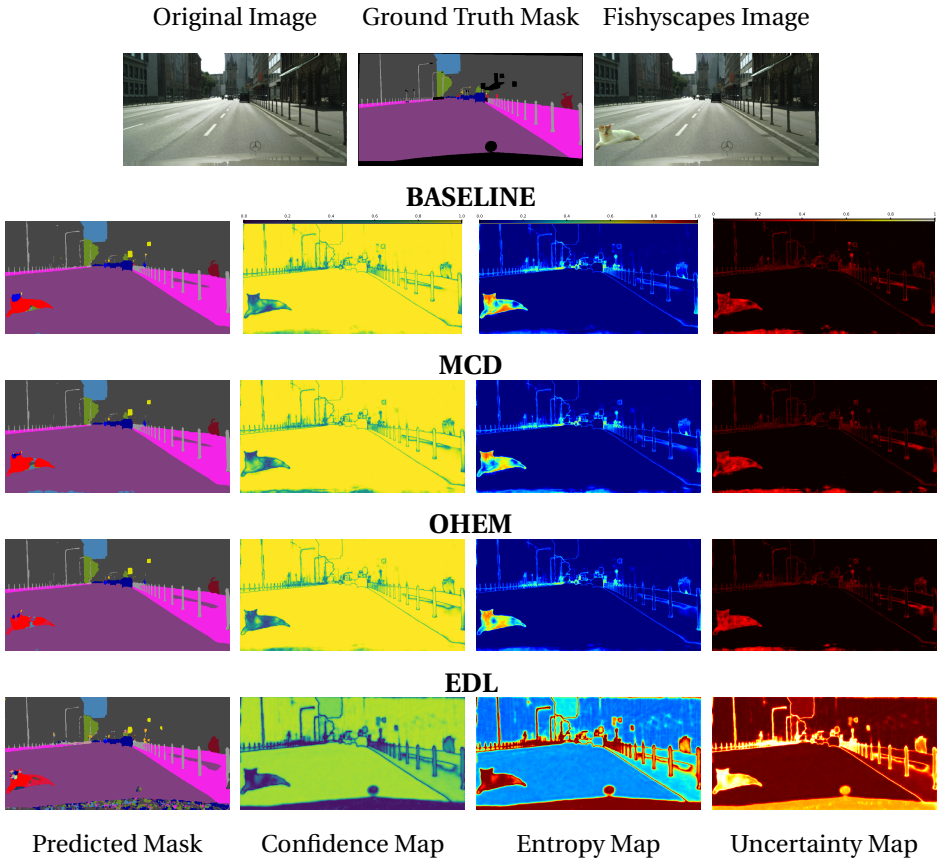


Figure 5.10: Predictions of Finetuned models on Fishyscapes dataset. The first row shows the Input RGB Image, Ground Truth Mask, and the Fishyscapes image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and an Uncertainty Map.

in distinct objects like the rider and bicycle, reinforcing EDLs robustness in uncertainty estimation despite segmentation limitations.

The entropy distributions shown in Figure 5.11 and Figure 5.12 further illustrate the impact of fine-tuning on the model's confidence and uncertainty. In these figures, we present the entropy levels for In-distribution (ID) pixels and Out of Distribution (OOD) pixels for all the fine-tuned models, evaluated against the baseline for both the Fishyscapes and LostAndFound datasets.

MCD and OHEM show a slight increase in entropy for OOD instances after fine-tuning, but the distribution remains skewed, with many OOD pix-

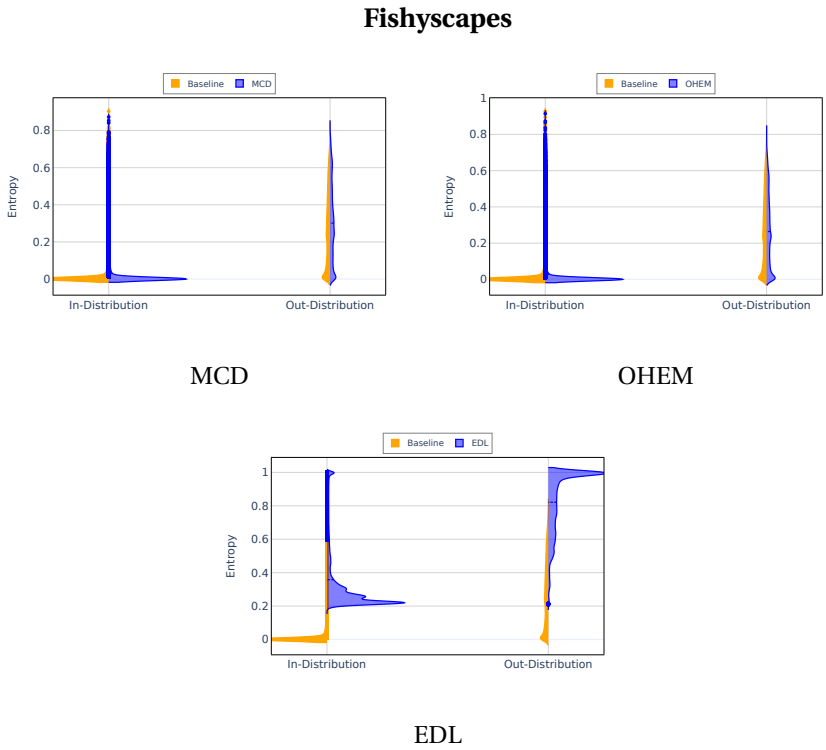


Figure 5.11: Entropy Distributions of Baseline (Yellow) and Finetuned Models (Blue) on the Fishyscapes Dataset

els still assigned low entropy, indicating persistent overconfidence in incorrect classifications. In contrast, EDL exhibits a significant shift in entropy for Out of Distribution (OOD) regions, with the fine-tuned model (blue) consistently showing higher entropy compared to the baseline (yellow) across both Fishyscapes and LostAndFound datasets. Notably, EDL concentrates most of its OOD distribution within the high entropy range (0.81.0), further confirming its improved uncertainty representation. This improvement suggests that EDL better separates ID and OOD pixels after fine-tuning, reinforcing its strength in uncertainty estimation. However, for In-distribution (ID) regions, EDL continues to assign slightly higher entropy than MCD and OHEM, reflecting residual underconfidence in segmentation.

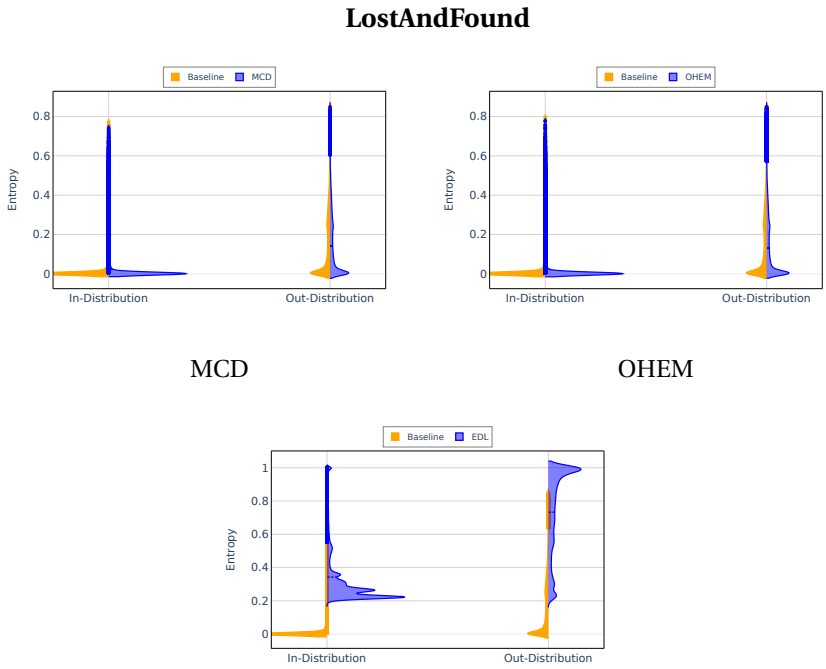


Figure 5.12: Entropy Distributions of Baseline (Yellow) and Finetuned Models (Blue) on the LostAndFound Dataset

5.2.8 Additional Results

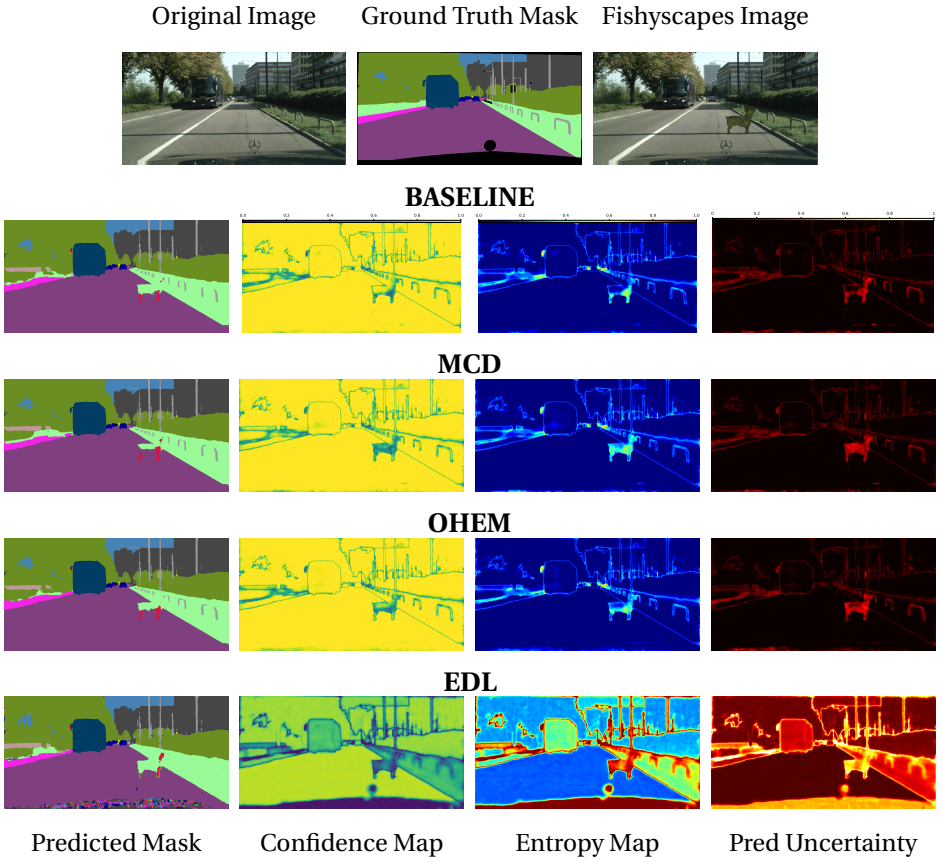


Figure 5.13: Predictions of Finetuned models on Fishyscapes dataset. The first row shows the LostAndFound RGB input image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and an Uncertainty Map.

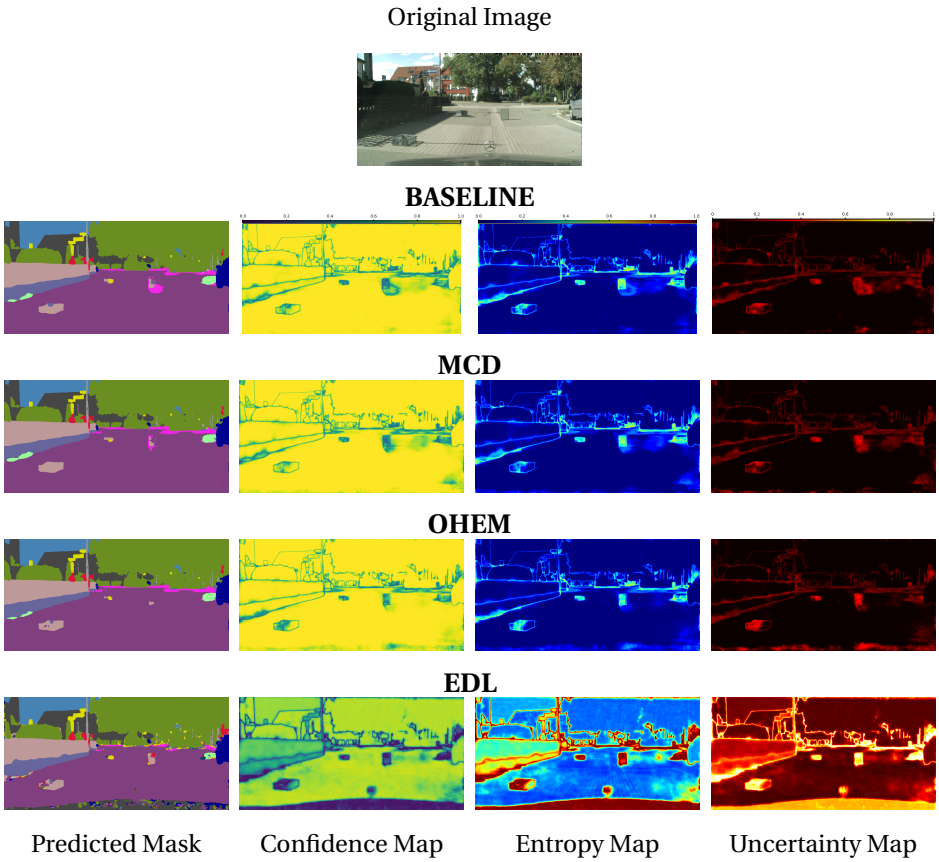


Figure 5.14: Predictions of Finetuned models on LostAndFound dataset. The first row shows the LostAndFound RGB input image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and an Uncertainty Map.

We collected per-image metrics from the cityscapes test set to assess the statistical significance of performance differences, including IoU, ECE, and Accuracy. We then performed the Oxford Wilcoxon signed-rank test to compare each of our evaluated models (MCD, OHEM, and EDL) against the baseline model across the three key evaluation metrics that were computed on a per-image basis over the cityscapes test set. This non-parametric test was chosen as it does not assume a normal distribution and is well-suited for paired model performance comparisons. The results confirmed that the evaluated models are statistically different from the baseline, with a significant improvement in all the metrics, reportedly IoU, Accuracy, and ECE ($p < 0.05$).

Method	Metric	Median	IQR	p-value	Significant?
Baseline	IoU	0.9399	0.0345	-	-
	Accuracy	0.9667	0.0212	-	-
	ECE	0.0056	0.0113	-	-
MCD	IoU	0.9423	0.0358	3.66e-10	✓
	Accuracy	0.9682	0.0212	1.95e-12	✓
	ECE	0.0053	0.0092	2.80e-04	✓
OHEM	IoU	0.9422	0.0357	1.30e-10	✓
	Accuracy	0.9681	0.0208	8.16e-12	✓
	ECE	0.0061	0.0113	2.04e-06	✓
EDL	IoU	0.9371	0.0365	1.35e-42	✓
	Accuracy	0.9635	0.0224	1.34e-50	✓
	ECE	0.1696	0.0187	2.07e-83	✓

Table 5.10: Oxford Wilcoxon signed-rank statistical significance test on the Cityscapes Test set.

6

Discussion and Limitations

In this chapter, we first review the principal findings from our experiments and delve into their implications concerning our three main research questions in the initial section.

6.1 Discussion

RQ1: How can uncertainty estimations be effectively incorporated into the training regimes of semantic segmentation models as guidance signals, and what impact do they have on the segmentation performance of the systems?

Our first research question examines the impact of incorporating uncertainty-aware guidance signals into the training regimes of semantic segmentation models. A thorough review of the existing literature and our experimental investigations revealed that incorporating guidance signals into semantic segmentation model training regimes presents performance advantages and challenges. Our experiments focused on the integration of uncertainty estimation techniques such as Monte Carlo Dropout (MCD), Online Hard Example Mining (OHEM), and Evidential Deep Learning (EDL) into the training processes of semantic segmentation models. Our results highlight the varying effectiveness of different uncertainty-aware methods and their impact on segmentation performance.

One key observation is that Monte Carlo Dropout (MCD) provided a stable segmentation performance but did not significantly outperform the baseline model. This suggests that while MCD introduces a form of uncertainty awareness, its impact on guiding the learning process is limited unless combined with additional techniques. Evidential Deep Learning

(EDL), on the other hand, exhibited challenges in handling low-frequency and ambiguous classes. The extremely low accuracy in certain classes (e.g., Bus and Traffic Light) indicates that EDL struggles to provide reliable guidance for rare categories. However, fine-tuning considerably improved EDL's performance, suggesting that it benefits from extended training and refinement. Online Hard Example Mining (OHEM) emerged as a more effective strategy for guiding the segmentation learning process, particularly for underrepresented classes, by explicitly focusing on difficult samples, ensuring the model prioritizes hard-to-learn uncertain regions. Interestingly, the combination of MCD and OHEM yielded the highest segmentation performance, demonstrating that uncertainty estimation is most effective when used in conjunction with strategies that directly address challenging samples. This suggests that uncertainty-aware methods alone may not be sufficient for optimal segmentation performance but can serve as complementary signals when integrated. Overall, our findings indicate that while uncertainty estimation can be a valuable tool in guiding the training process, its effectiveness in enhancing segmentation performance depends on the specific approach used in the training pipeline.

RQ2: Does the incorporation of guidance signals in the training process enhance the reliability of semantic segmentation models compared to conventional training regimes, particularly in terms of predictive confidence and reliability?

Incorporating guidance signals in the training process significantly enhances the reliability of semantic segmentation models compared to conventional training regimes. This improvement is particularly evident in terms of predictive confidence and overall reliability, as demonstrated by calibration performance metrics and reliability diagrams. Models trained with uncertainty awareness, especially those using EDL, consistently achieved lower Expected Calibration Error (ECE) scores over the whole test set, indicating that their confidence estimates align more closely with actual predictive accuracy. This improvement is particularly noticeable in uncertain regions, where conventional models tend to exhibit overconfidence. Reliability diagrams further reinforce that the segmentation model guided by EDL training regime maintains a more balanced confidence distribution, reducing both underconfidence in incorrect predictions and

overconfidence in incorrect ones. The fine-tuned variants of EDL, such as EDL+OHEM and EDL+UNC, further enhanced these improvements, achieving the lowest calibration errors and demonstrating the most consistent performance. This improved reliability is crucial for safety-critical applications, where overconfident errors can lead to high-risk failures. However, the analysis also revealed a nuanced aspect. While EDL generally improves calibration, the per-image analysis showed increased variability in ECE for EDL based models compared to other training regimes. This indicates a possible trade-off where EDL might enhance calibration in certain situations but could also lead to inconsistencies in others. This observation calls for a more detailed exploration to understand the factors influencing this variability, such as image complexity and data augmentation techniques.

Although EDL based models showed the most significant improvements in overall reliability, other uncertainty-aware approaches also demonstrated benefits. For example, MCD and OHEM also exhibited moderate enhancements in overall calibration performance after fine-tuning, though not to the same extent as EDL. This heightened reliability when using uncertainty guidance signals during training makes these models particularly suitable for real-world deployment, especially in applications where accurate and trustworthy predictions are paramount.

RQ3: To what extent does employing guidance signals during segmentation model training improve the robustness in detecting Out of Distribution (OOD) objects? How effectively can an uncertainty-guided segmentation model differentiate between ID and OOD instances?

The experimental results on Fishyscapes and LostAndFound datasets provide strong evidence that uncertainty-guided training, particularly using EDL, significantly improves OOD detection in segmentation models. EDL-based models consistently outperformed baseline and the other evaluated uncertainty training regimes, achieving higher AUROC scores and lower FPR95 values, indicating superior ability to distinguish OOD objects from ID samples.

Uncertainty heatmaps clearly demonstrated that OOD instances correspond to high-uncertainty regions in EDL models, effectively reducing false positives in ID classifications. This was visually apparent in confi-

dence, entropy, and uncertainty maps, where EDL models highlighted OOD regions as uncertain, unlike other approaches. In contrast, MCD and OHEM often exhibited overconfidence in OOD regions, leading to higher false positive rates. The relatively poor performance of MCD and OHEM on the LostandFound dataset can be attributed to the dataset's characteristics and the limitations of these techniques. MCD's dropout-based approach and OHEM's focus on hard examples may not effectively capture the subtle distributional shifts present in real-world street scenes with tiny, distant OOD objects. This misalignment resulted in increased FPR95 and lower AUROC scores for these models.

Fine-tuning further enhanced EDL-based models performance, showing consistent improvements across datasets. Entropy distributions also revealed that fine-tuned EDL models concentrated most of their OOD distribution within the high entropy range, confirming their improved ability to distinguish OOD instances. However, EDL models exhibited some limitations, such as blurry boundaries for ID regions and occasional underconfidence in certain ID areas. These findings highlight that while EDL-based uncertainty estimation significantly improves OOD detection robustness, there may be trade-offs with segmentation performance that warrant further investigation. The results underscore the importance of selecting appropriate uncertainty guidance techniques aligned with dataset characteristics and the specific challenges of OOD detection in segmentation tasks.

6.2 Limitations

While our study provides valuable insights into the effectiveness of uncertainty guidance in semantic segmentation, several limitations should be acknowledged. Firstly, the uncertainty-guided training experiments were conducted on a single dataset (Cityscapes), which may limit the generalizability of our findings and require further examination. Future work should evaluate the proposed methods on diverse datasets and model architectures to assess their robustness across various scenarios.

Secondly, while improving model performance, the fine-tuning process introduced slight miscalibrations in some models, particularly in the highest confidence range. This suggests that further research is needed to optimize fine-tuning strategies for maintaining calibration while enhancing

segmentation accuracy. Thirdly, while demonstrating strong overall calibration, the EDL model exhibited some limitations. The per-image analysis revealed increased variability in calibration performance across different images within the dataset. Additionally, EDL models showed a slight decrease in segmentation accuracy for certain classes, particularly those with fewer training examples (minority classes). Future research should investigate techniques to mitigate these limitations, such as incorporating class-balancing strategies within the EDL framework or exploring alternative training objectives that prioritize both calibration and accuracy for underrepresented classes.

Lastly, the computational complexity of incorporating uncertainty estimations, especially MCD and EDL, may pose challenges for real-time applications. Future research should investigate more efficient implementations of these techniques to make them viable for practical deployment.

In conclusion, our study demonstrates that incorporating uncertainty estimations as guidance signals can significantly enhance the semantic segmentation model's performance, reliability, and robustness. While our findings are promising, addressing the identified limitations will be crucial for advancing the field and ensuring the practical applicability of these methods. Potential areas for future work are detailed in 7.2 to address these limitations and propel the research forward. This discussion aims to outline pathways for overcoming the current constraints and expanding the research's scope and impact within the domain of semantic segmentation.

7

Conclusions and Future Work

7.1 Conclusions

The primary objective of this thesis is to investigate the integration of uncertainty signals as a guiding mechanism in training semantic segmentation models, with the goal of enhancing their performance, reliability, and robustness, particularly in detecting Out of Distribution (OOD) instances. By analyzing different uncertainty estimation techniques, including Monte Carlo Dropout (MCD), Online Hard Example Mining (OHEM), and Evidential Deep Learning (EDL), we examined their impact on segmentation performance, model calibration, and the ability to identify OOD samples.

Our experiments on the Cityscapes, Fishyscapes, and LostandFound datasets showed that uncertainty-guided models, particularly those based on EDL, achieved significant improvements in OOD detection and overall reliability. EDL consistently outperformed baseline and conventional approaches in distinguishing between In-distribution (ID) and OOD samples, demonstrating its strong ability to capture and represent uncertainty.

Interestingly, while other uncertainty techniques like MCD and OHEM based models improved segmentation performance over the baseline, they did not perform as well in OOD detection. EDL, on the other hand, stood out as an exception. It significantly enhanced OOD robustness and reliability but struggled with segmentation performance. However, finetuning EDL achieved a balance between strong segmentation performance and reliable uncertainty estimation. One of the key takeaways from this work is the importance of well-calibrated models in real-world applications. Integrating uncertainty, especially through EDL led to models

that were not only more reliable but also better at estimating confidence in their predictions. Reliability diagrams reinforced these findings, showing that EDL based models had a more balanced confidence distribution, helping to reduce overconfidence in incorrect predictions. This is crucial for applications where trust in model outputs directly impacts decision-making.

However, while incorporating uncertainty improved OOD detection and confidence calibration, we also noticed trade-offs, particularly in segmentation performance. Some models struggled with boundary sharpness and fine detail preservation. While our results highlight the potential of uncertainty-guided training, they also open avenues for future research, including the development of novel loss functions, adaptive uncertainty modulation dynamically during training, and architectural modifications to optimize the trade-off between segmentation accuracy and OOD robustness. Ultimately, this thesis sheds light on the role of uncertainty in semantic segmentation and provides a foundation for building more robust and reliable vision systems. By embracing uncertainty-guided learning, we can push the boundaries of model interpretability and trustworthiness, paving the way for safer and more effective AI applications in the real world.

7.2 Future Work

While this research demonstrates the benefits and challenges of integrating uncertainty estimation into semantic segmentation training, several avenues remain for future exploration. One key direction is optimizing the trade-off between segmentation accuracy and OOD detection, particularly in preserving boundary sharpness and fine details. This could be achieved by designing novel loss functions or architectural modifications that balance uncertainty guidance with fine-grained segmentation precision.

Another promising direction is adaptive uncertainty modulation, where the influence of uncertainty signals is dynamically adjusted based on data characteristics. This could help models focus more on ambiguous regions while maintaining strong segmentation performance in well-defined areas. Beyond methodological improvements, applying uncertainty-guided

training to other vision tasks, such as instance segmentation or depth estimation, could provide valuable insights into the generalizability of these techniques. Furthermore, exploring real-world deployment scenarios, such as in autonomous driving or medical imaging, would help assess the practical impact of uncertainty-aware models in safety-critical applications.

Finally, as deep learning continues to evolve, integrating self-supervised or semi-supervised learning or even contrastive learning with uncertainty estimation could improve performance in data-scarce environments. This would enable models to learn from unlabeled or partially labeled data while maintaining reliable uncertainty estimates. Future research can further refine uncertainty-aware semantic segmentation by addressing these challenges, ultimately paving the way for more robust and trustworthy deep-learning models.

A

List of Figures

2.1 The process of semantic segmentation. 12

2.2 Encoder Decoder Architecture 16

2.3 Vision Transformer Architecture 20

3.1 Reliability Diagram 44

4.1 Distribution of Cityscapes classes 46

4.2 Sample RGB images and their corresponding segmentation masks from the cityscapes dataset. 48

4.3 Sample RGB images and their associated segmentation masks from the Fishyscapes dataset. In the mask, blue indicates In-distribution (ID) pixels the model was trained on, red signifies Out of Distribution (OOD) pixels, and black represents the background areas. 49

4.4 Sample RGB images and their associated segmentation masks from the LostAndFound dataset. In the mask, blue indicates In-distribution (ID) pixels the model was trained on, red signifies Out of Distribution (OOD) pixels, and black represents the background areas. 50

4.5 Baseline Architecture 51

4.6 Architecture of Mix Vision Transformer 52

4.7 Workflow of Feature Pyramid Network Decoder 56

4.8 Architecture of Monte Carlo Dropout Based Uncertainty Signal Training 60

4.9 Architecture of OHEM Based Uncertainty Signal Training . . 62

4.10 Architecture of EDL Based Uncertainty Signal Training 64

5.1 IoU Calculation 70

5.2 Accuracy of vanilla models across classes, ordered by descending class frequency in the dataset. 78

5.3 Accuracy of finetuned models across classes, ordered by descending class frequency in the dataset. 82

5.4 Predictions on the Cityscapes dataset. The first row displays the Original Image and the Ground Truth Mask. Each subsequent section corresponds to a specific model (Baseline, MCD, OHEM, EDL), with two rows: the first representing the vanilla model and the second representing the finetuned model. Within each section, the columns display the predicted segmentation mask, confidence map, entropy map, and predictive uncertainty map. In the confidence map, yellow indicates high confidence while blue indicates low confidence. In the entropy map, blue represents low entropy and red represents high entropy. Similarly, in the uncertainty map, brighter pixels indicate higher predictive uncertainty. . 84

5.5 Reliability Diagrams of Vanilla Models 88

5.6 Reliability Diagrams of Finetuned Models 90

5.7 Misclassification Distributions on Cityscapes Dataset of finetuned Models. The x-axis represents the confidence of misclassified pixels arranged in 10 bins and the y-axis represents the number of pixels that fall into that confidence bin 93

5.8 ROC Curves of the Finetuned Models on Fishyscapes Dataset 96

5.9 Predictions of Vanilla models on Fishyscapes Dataset. The first row shows the input RGB Image, Ground Truth Mask, and the Fishyscapes image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and a Predictive Uncertainty Map. 98

5.10 Predictions of Finetuned models on Fishyscapes dataset. The first row shows the Input RGB Image, Ground Truth Mask, and the Fishyscapes image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and an Uncertainty Map. 100

5.11 Entropy Distributions of Baseline (Yellow) and Finetuned Models (Blue) on the Fishyscapes Dataset 101

5.12 Entropy Distributions of Baseline (Yellow) and Finetuned Models (Blue) on the LostAndFound Dataset 102

5.13 Predictions of Finetuned models on Fishyscapes dataset. The first row shows the LostAndFound RGB input image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and an Uncertainty Map. 103

5.14 Predictions of Finetuned models on LostAndFound dataset. The first row shows the LostAndFound RGB input image. Each subsequent row corresponds to a specific model (Baseline, MCD, OHEM, EDL), displaying a Predicted Mask, Confidence Map, Entropy Map, and an Uncertainty Map. 104

B

List of Tables

4.1 Description of Cityscapes Dataset categories 47

5.1 Segmentation Performance of Vanilla Models 74

5.2 Evaluation of Segmentation Performance Under Different Configurations 76

5.3 Per-Class Accuracy of Vanilla Models: The rows depict the 19 classes from the Cityscapes Dataset, organized by their frequency. Each row lists the corresponding accuracy values for the different models (Baseline, MCD, OHEM, and EDL) in the columns. 77

5.4 Segmentation Performance of Finetuned Models 79

5.5 Per-Class Accuracy for Finetuned Models: The rows depict the 19 classes from the Cityscapes Dataset, organized by their frequency. Each row lists the corresponding accuracy values for the different models (Baseline, MCD, OHEM, and EDL) in the columns. 81

5.6 Calibration Performance of Models 85

5.7 Variability Analysis of key model metrics across images from the Cityscapes Dataset for the fine-tuned models. The table presents mean±std for all the reported metrics. 91

5.8 OOD Performance of Vanilla Models on Fishyscapes and LostandFound Datasets 94

5.9 OOD Performance of Finetuned Models on Fishyscapes and LostandFound Datasets 95

5.10 Oxford Wilcoxon signed-rank statistical significance test on the Cityscapes Test set. 105

Acronyms

AI	Artificial Intelligence
AUROC	Area under the Receiver Operating Characteristic Curve
BNNs	Bayesian Neural Networks
CNNs	Convolutional Neural Networks
CV	Computer Vision
DL	Deep Learning
DNNs	Deep Neural Networks
DST	DempsterShafer Theory of Evidence
ECE	Expected Calibration Error
EDL	Evidential Deep Learning
FCNs	Fully Convolutional Networks
FFN	Feed-Forward Network
FPN	Feature Pyramid Network
FPR95	False Positive Rate at 95 % True Positive Rate
ID	In-distribution
IoU	Intersection Over Union
MCD	Monte Carlo Dropout
mIoU	Mean Intersection Over Union
MiT	Mix Transformer encoder
ML	Machine Learning
MLE	Maximum Likelihood Estimation
NLP	Natural Language Processing
OHEM	Online Hard Example Mining
OOD	Out of Distribution

RoIs	regions of interest
RPN	Region Proposal Network
SOTA	state-of-the-art
std	standard deviations
ViT	Vision Transformer

C

Bibliography

[ABDAR et al. 2021] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya, V. Makarenkov und S. Nahavandi. **A review of uncertainty quantification in deep learning: Techniques, applications and challenges**. Information Fusion, Vol. 76:243297, 2021.

[AMODEI et al.] **Concrete Problems in AI Safety**.

[BADRINARAYANAN et al.] **SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation**.

[BAI et al. 2023] S. Bai, J. Chen, Y. Yang und Y. Liu. **Multi-Dimension Compression of Feed-Forward Network in Vision Transformers**. Pattern Recognition Letters, Vol. 176:56–61, 2023.

[BIAN et al.] **Pyramid Network with Online Hard Example Mining for Accurate Left Atrium Segmentation**.

[BLUM et al. 2019] H. Blum, P.-E. Sarlin, J. Nieto, R. Siegwart und C. Cadena. **The Fishyscapes Benchmark: Measuring Blind Spots in Semantic Segmentation**. arXiv preprint arXiv:1904.03215, 2019.

[BLUNDELL et al.] **Weight Uncertainty in Neural Networks**.

[BUSLAEV et al. 2020] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin und A. A. Kalinin. **Albumentations: Fast and Flexible Image Augmentations**. Information, Vol. 11(2):125, 2020.

[CARION et al.] **End-to-End Object Detection with Transformers**.

[CHAN et al.] **Entropy Maximization and Meta Classification for Out-Of-Distribution Detection in Semantic Segmentation**.

[CHEN et al.] **Semantic Image Segmentation with Deep Convolutional Nets and Fully Connected CRFs**.

[CHEN et al.] **DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs**.

[CHEN et al.] **Rethinking Atrous Convolution for Semantic Image Segmentation**.

[CHU et al.] **Conditional Positional Encodings for Vision Transformers**.

[CORDTS et al.] **The Cityscapes Dataset for Semantic Urban Scene Understanding**.

[DAVIES et al.] **Knowledge from Uncertainty in Evidential Deep Learning**.

[DAVIS und GOADRICH 2006] J. Davis und M. Goadrich. **The relationship between Precision-Recall and ROC curves**. In: Proceedings of the 23rd International Conference on Machine Learning, ICML '06, p. 233240. 2006, Association for Computing Machinery, New York, NY, USA.

[DE JORGE et al.] **Reliability in Semantic Segmentation: Are We on the Right Track?**

[DECHESNE et al. 2021] C. Dechesne, P. Lassalle und S. Lefèvre. **Bayesian U-Net: Estimating Uncertainty in Semantic Segmentation of Earth Observation Images**. Remote Sensing, Vol. 13(19), 2021.

[DEGROOT und FIENBERG 1983] M. H. Degroot und S. E. Fienberg. **The Comparison and Evaluation of Forecasters.** The Statistician, Vol. 32:12–22, 1983.

[DOSOVITSKIY et al. 2020] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit und N. Houlsby. **An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale**. CoRR, Vol. abs/2010.11929, 2020.

[EVERINGHAM et al.] **The Pascal Visual Object Classes Challenge 2007 (VOC2007) Results**. URL: `http://host.robots.ox.ac.uk:8080/pascal/VOC/voc2007/index.html`.

[FARABET et al. 2013] C. Farabet, C. Couprie, L. Najman und Y. LeCun. **Learning Hierarchical Features for Scene Labeling**. IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 35(8):1915–1929, 2013.

[FAWCETT 2006] T. Fawcett. **An introduction to ROC analysis**. Pattern Recognition Letters, Vol. 27(8):861–874, 2006. ROC Analysis in Pattern Recognition.

[GAL und GHAHRAMANI] **Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning**.

[GAWLIKOWSKI et al.] **A Survey of Uncertainty in Deep Neural Networks**.

[GIRSHICK et al.] **Rich feature hierarchies for accurate object detection and semantic segmentation**.

[GOODFELLOW et al.] **Explaining and Harnessing Adversarial Examples**.

[GUO et al.] **On Calibration of Modern Neural Networks**.

[GUPTA et al.] **Learning Rich Features from RGB-D Images for Object Detection and Segmentation**.

[GUSTAFSSON et al.] **Evaluating Scalable Bayesian Deep Learning Methods for Robust Computer Vision**.

[HE et al.] **Masked Autoencoders Are Scalable Vision Learners**.

[HE et al.] **Deep Residual Learning for Image Recognition**.

[HENDRYCKS et al.] **Unsolved Problems in ML Safety**.

[HENDRYCKS und GIMPEL] **A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks**.

[HOCHREITER 1998] S. Hochreiter. **The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions**. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, Vol. 06(02):107–116, 1998.

[HOLDER und SHAFIQUE 2021] C. J. Holder und M. Shafique. **Efficient Uncertainty Estimation in Semantic Segmentation via Distillation**. In: 2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 2021, pp. 3080–3087.

[ISLAM et al.] **How Much Position Information Do Convolutional Neural Networks Encode?**

[JOSPIN et al. 2022] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine und M. Bennamoun. **Hands-On Bayesian Neural NetworksA Tutorial for Deep Learning Users**. IEEE Computational Intelligence Magazine, Vol. 17(2):2948, 2022.

[KENDALL und GAL] **What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?**

[KRIZHEVSKY et al. 2012] A. Krizhevsky, I. Sutskever und G. E. Hinton. **ImageNet Classification with Deep Convolutional Neural Networks**. In: F. Pereira, C. Burges, L. Bottou und K. Weinberger, Eds., Advances in Neural Information Processing Systems, Vol. 25. 2012, Curran Associates, Inc.

[KRUGER und DUNNING 1999] J. Kruger und D. Dunning. **Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments**. Journal of Personality and Social Psychology, Vol. 77(6):1121–1134, 1999.

[LAKSHMINARAYANAN et al.] **Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles**.

[LANDGRAF et al.] **U-CE: Uncertainty-aware Cross-Entropy for Semantic Segmentation**.

[LANDGRAF et al. 2023] S. Landgraf, K. Wursthorn, M. Hillemann und M. Ulrich. **DUDES: Deep Uncertainty Distillation Using Ensembles for Semantic Segmentation**. arXiv preprint arXiv:2303.09843, 2023b.

[LANDGRAF et al. 2024] S. Landgraf, K. Wursthorn, M. Hillemann und M. Ulrich. **DUDES: Deep Uncertainty Distillation using Ensembles for Semantic Segmentation**. PFG Journal of Photogrammetry, Remote Sensing and Geoinformation Science, Vol. 92(2):101114, 2024.

[LECUN et al. 2015] Y. LeCun, Y. Bengio und G. Hinton. **Deep Learning**. Nature, Vol. 521(7553):436–444, 2015.

[LECUN et al. 1989] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard und L. D. Jackel. **Backpropagation Applied to Handwritten Zip Code Recognition**. Neural Computation, Vol. 1(4):541–551, 1989.

[LEE et al.] **Efficient Ensemble Model Generation for Uncertainty Estimation with Bayesian Approximation in Segmentation**.

[LEE et al. 2018] K. Lee, H. Lee, K. Lee und J. Shin. **Training Confidence-calibrated Classifiers for Detecting Out-of-Distribution Samples**. arXiv preprint arXiv:1711.09325, 2018.

[LEIBIG et al. 2017] C. Leibig, V. Allken, M. S. Ayhan, P. Berens und S. Wahl. **Leveraging uncertainty information from deep neural networks for disease detection**. Scientific Reports, Vol. 7(1):17816, 2017.

[LI et al.] **Fully Convolutional Instance-aware Semantic Segmentation**.

[LIN et al.] **Feature Pyramid Networks for Object Detection**.

[LIU et al.] **Swin Transformer: Hierarchical Vision Transformer using Shifted Windows**.

[LONG et al. 2017] J. Long, E. Shelhamer und T. Darrell. **Fully Convolutional Networks for Semantic Segmentation**. IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 39(4):640–651, 2017.

[LOSHCHILOV und HUTTER] **Online Batch Selection for Faster Training of Neural Networks**.

[LOSHCHILOV und HUTTER] **Decoupled Weight Decay Regularization**.

[LV et al.] **Importance-Aware Image Segmentation-based Semantic Communication for Autonomous Driving**.

[MACKAY 1992] D. J. C. MacKay. **A practical Bayesian framework for backpropagation networks**. Neural Comput., Vol. 4(3):448472, 1992.

[MICIKEVICIUS et al.] **Mixed Precision Training**.

[MINAEE et al.] **Image Segmentation Using Deep Learning: A Survey**.

[MONTEIRO et al.] **Stochastic Segmentation Networks: Modelling Spatially Correlated Aleatoric Uncertainty**.

[MÜLLER et al.] **Towards a Guideline for Evaluation Metrics in Medical Image Segmentation**.

[NAIR et al.] **Exploring Uncertainty Measures in Deep Networks for Multiple Sclerosis Lesion Detection and Segmentation**.

[NG et al. 2023] M. Ng, F. Guo, L. Biswas, S. E. Petersen, S. K. Piechnik, S. Neubauer und G. Wright. **Estimating Uncertainty in Neural Networks for Cardiac MRI Segmentation: A Benchmark Study**. IEEE Transactions on Biomedical Engineering, Vol. 70(6):19551966, 2023.

[NGUYEN et al.] **Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images**.

[NICULESCU-MIZIL und CARUANA 2005] A. Niculescu-Mizil und R. Caruana. **Predicting good probabilities with supervised learning**. In: Proceedings of the 22nd international conference on Machine learning, pp. 625–632. 2005, ACM.

[NOH et al.] **Learning Deconvolution Network for Semantic Segmentation**.

[OVADIA et al.] **Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift**.

[PAKDAMAN NAEINI et al. 2015] M. Pakdaman Naeini, G. Cooper und M. Hauskrecht. **Obtaining Well Calibrated Probabilities Using Bayesian Binning**. Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 29(1), 2015.

[PINGGERA et al. 2016] P. Pinggera, S. Ramos, S. Gehrig, U. Franke, C. Rother und R. Mester. **Lost and found: detecting small road hazards for self-driving vehicles**. 2016, In: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS).

[PLATT 1999] J. C. Platt. **Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods**. In: Advances in Large Margin Classifiers, pp. 61–74. 1999, MIT Press.

[POHLEN et al.] **Full-Resolution Residual Networks for Semantic Segmentation in Street Scenes**.

[RAMACHANDRAN et al.] **Stand-Alone Self-Attention in Vision Models**.

[RONNEBERGER et al.] **U-Net: Convolutional Networks for Biomedical Image Segmentation**.

[RUSSAKOVSKY et al.] **ImageNet Large Scale Visual Recognition Challenge**.

[SANDER et al. 2019] J. Sander, B. D. de Vos, J. M. Wolterink und I. Igum. **Towards increased trustworthiness of deep learning segmentation methods on cardiac MRI**. In: E. D. Angelini und B. A. Landman, Eds., Medical Imaging 2019: Image Processing, p. 44. 2019, SPIE.

[SCHMIDT-MENGIN et al. 2022] M. Schmidt-Mengin, T. Soulier, M. Hamzaoui, A. Yazdan-Panah, B. Bodini, N. Ayache, B. Stankoff und O. Colliot. **Online hard example mining vs. fixed oversampling strategy for segmentation of new multiple sclerosis lesions from longitudinal FLAIR MRI**. Frontiers in Neuroscience, Vol. 16, 2022.

[SENSOY et al.] **Evidential Deep Learning to Quantify Classification Uncertainty**.

[SHRIVASTAVA et al.] **Training Region-based Object Detectors with Online Hard Example Mining**.

[SIMONYAN und ZISSERMAN] **Very Deep Convolutional Networks for Large-Scale Image Recognition**.

[SRIVASTAVA et al. 2014] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever und R. Salakhutdinov. **Dropout: a simple way to prevent neural networks from overfitting**. J. Mach. Learn. Res., Vol. 15(1):19291958, 2014.

[SULTANA et al. 2020] F. Sultana, A. Sufian und P. Dutta. **Evolution of Image Segmentation using Deep Convolutional Neural Network: A Survey**. Knowledge-Based Systems, Vol. 201202:106062, 2020.

[SZEGEDY et al.] **Intriguing properties of neural networks**.

[UIJLINGS et al. 2013] J. R. Uijlings, K. E. Sande, T. Gevers und A. W. Smeulders. **Selective Search for Object Recognition**. Int. J. Comput. Vision, Vol. 104(2):154171, 2013.

[VAN AMERSFOORT et al.] **Uncertainty Estimation Using a Single Deep Deterministic Neural Network**.

[VASWANI et al.] **Attention Is All You Need**.

[WANG et al. 2019] G. Wang, W. Li, M. Aertsen, J. Deprest, S. Ourselin und T. Vercauteren. **Aleatoric uncertainty estimation with test-time augmentation for medical image segmentation with convolutional neural networks**. Neurocomputing, Vol. 338:3445, 2019.

[WEI et al. 2016] Y. Wei, X. Liang, Y. Chen, Z. Jie, Y. Xiao, Y. Zhao und S. Yan. **Learning to segment with image-level annotations**. Pattern Recognition, Vol. 59:234–244, 2016. Compositional Models and Structured Learning for Visual Recognition.

[WU et al.] **Bridging Category-level and Instance-level Semantic Image Segmentation**.

[XIE et al. 2021] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Álvarez und P. Luo. **SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers**. CoRR, Vol. abs/2105.15203, 2021.

[YAGER und LIU 2010] R. R. Yager und L. Liu. **Classic Works of the Dempster-Shafer Theory of Belief Functions**. Springer Publishing Company, Incorporated, 1st Edn., 2010.

[YAO et al. 2021] Z. Yao, Y. Cao, Y. Lin, Z. Liu, Z. Zhang und H. Hu. **Leveraging Batch Normalization for Vision Transformers**. In: 2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 2021, pp. 413–422.

[ZADROZNY und ELKAN 2001] B. Zadrozny und C. Elkan. **Obtaining Calibrated Probability Estimates from Decision Trees and Naive Bayesian Classifiers**. In: Proceedings of the Eighteenth International Conference on Machine Learning (ICML), pp. 609–616. 2001, Morgan Kaufmann.

[ZADROZNY und ELKAN 2002] B. Zadrozny und C. Elkan. **Transforming classifier scores into accurate multiclass probability estimates**. In:

Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '02, p. 694699. 2002, Association for Computing Machinery, New York, NY, USA.

[ZHANG und GONG] **Positional Label for Self-Supervised Vision Transformer**.

[ZHENG et al.] **Rethinking Semantic Segmentation from a Sequence-to-Sequence Perspective with Transformers**.

[ZHENG et al.] **Rethinking Semantic Segmentation from a Sequence-to-Sequence Perspective with Transformers**.

Declaration of Academic Integrity

I hereby declare that I have written the present work myself and did not use any sources or tools other than the ones indicated.

Datum: .....

(Signature)